GPGPU

Chronology of GPGPU Computing & Related Studies

Hyunil Shin, Prof. Hyeonsang Eom
Distributed Computing Systems Laboratory
Department of Computer Science and Engineering
Seoul National University, Korea (Rep. of)

2026. 10. 05.

Overall

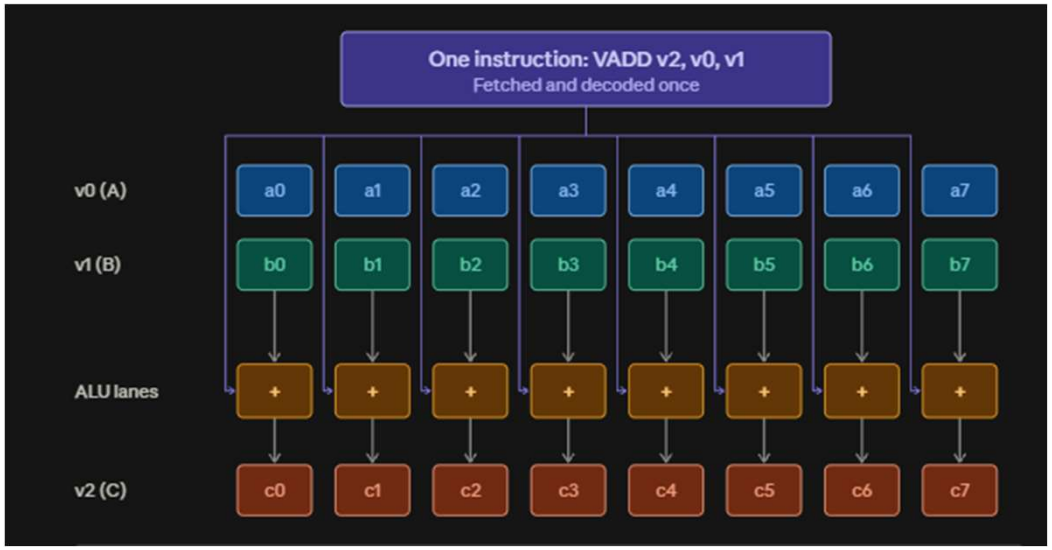
- Evolution of GPGPU through GPU architectures
  - From Tesla to Ampere
- Focused on architectural evolution & platform updates
- Mainly NVIDIA GPUs
  - OpenCL for adversary ideas

Traditional GPU

- Fixed Pipelines to process graphics computation

- Single Instruction Multiple Data (SIMD)
  - Single same instructions to multiple data

[*]

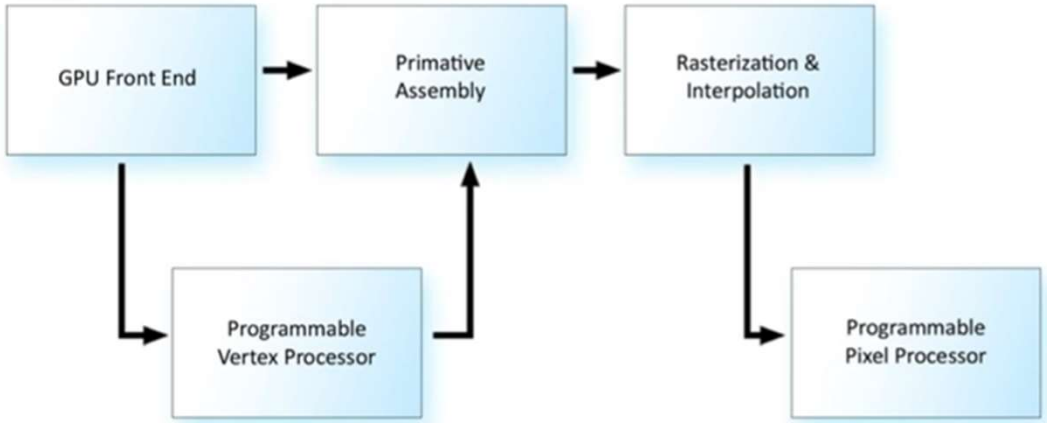


- Computing vertex / pixel data

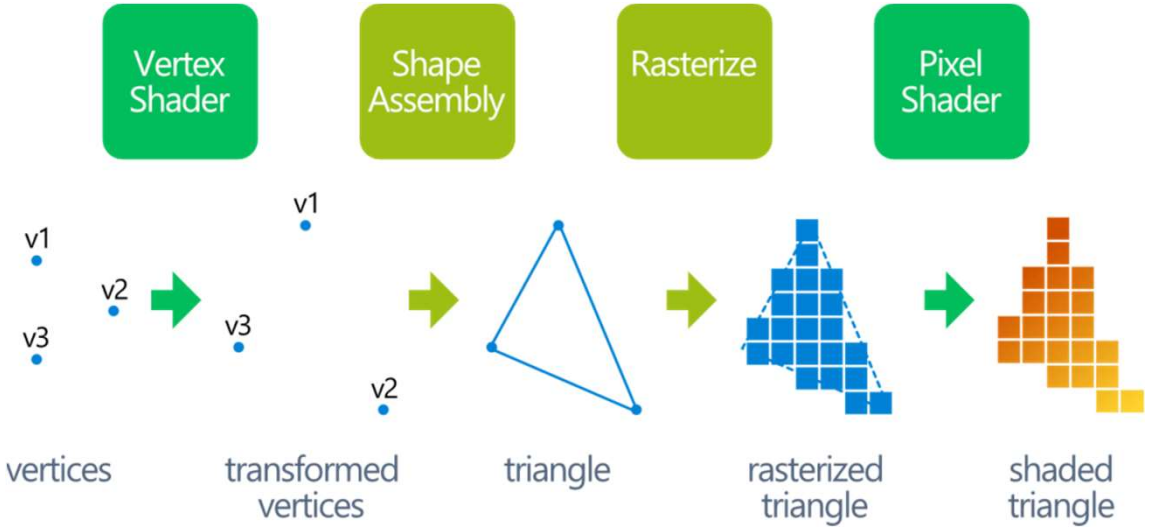
Traditional GPU

- Graphics pipeline

[27]

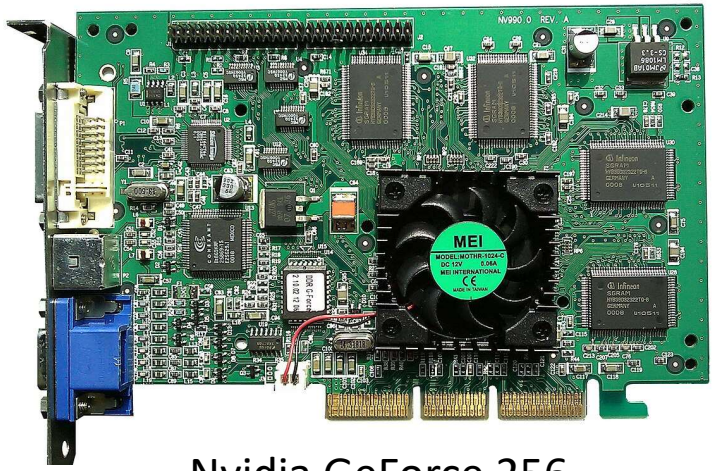


[27]



Fixed Function Pipeline (1990s)

[32]



Nvidia GeForce 256

- Pre-determined hardware stages
  - New hardware for new functions
- Calculating vertex shaders & pixel fragments
- Resources cannot be shared between stages

GeForce 3

[33]



- Programmable vertex shaders
  - For custom lighting & animation
- Mostly Fixed-Functions
- 128 instructions limit
  - Only vertex part can be programmed

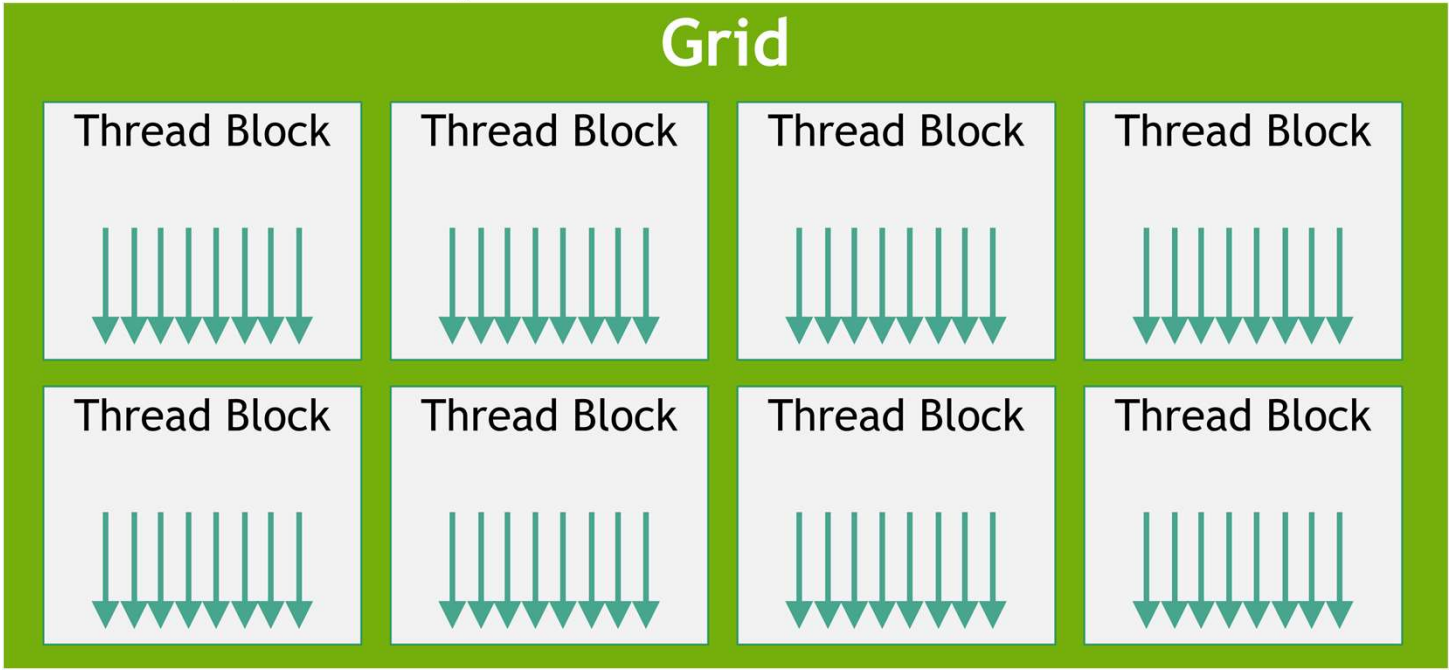
Terminology – Hardware

- **Streaming Multiprocessor (SM)**
  - Unified graphics and computing multiprocessor
  - Usually consisting of
    - Multiple cores
    - Shared memory
    - L1 cache
- **Raster Operation Pipeline (ROP)**
  - Graphics component for display

Terminology – Software

- **Thread**
  - Smallest unit of execution
  - Running the same code with different position value (threadID, blockID)

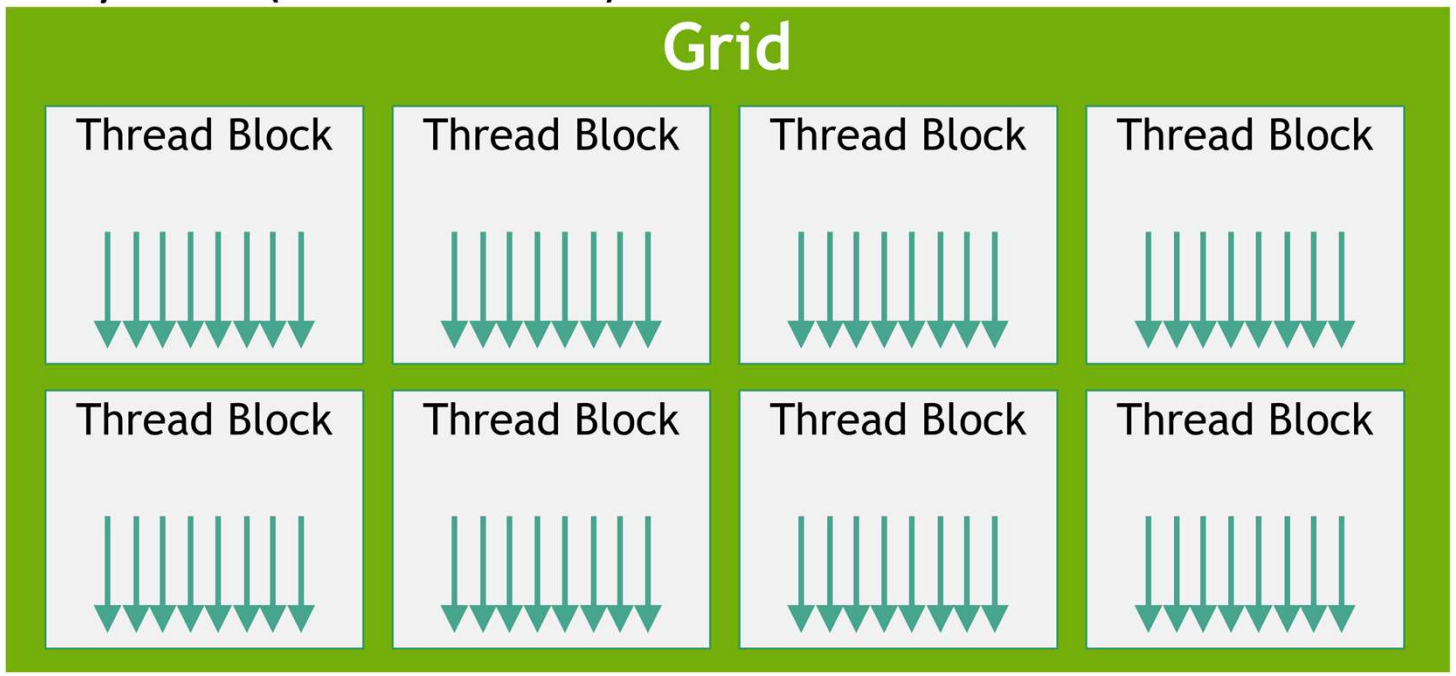
[28]



Terminology – Software

- **Thread Block (TB), Grid**
  - TB: Group of threads, Grid: Multiple TBs in 2D/3D shape
  - Threads in the same block can share shared memory, and can be synced (1 TB in 1 SM)

[28]



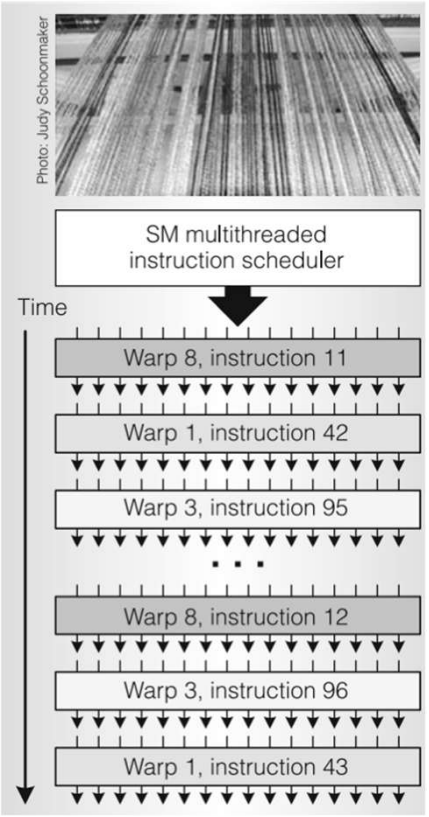
Terminology – Software (Cont'd)

- **Kernel**
  - Function / Program on GPU
  - Can be represented as Grid
- **Stream**
  - Series of works (queue)
    - Kernel launches, Memory copies
- **Context**
  - Execution environment
  - Multiple streams, Address space, Memory allocation

Terminology – Software (Cont'd)

- **Single Instruction Multi-Thread Model (SIMT)**
  - Process instruction on multiple independent threads
    - Not just data planes
    - Thread-level parallel programs
  - **1 Warp = 32 parallel threads**
    - Granularity of scheduling
  - Each SM executes an active warp from a pool of several warps
    - Size of pool differs through architecture
  - Instruction unit selects warp at each instruction time
    - If warp is ready to execute
  - Issuing next instruction to active threads
    - Inactive threads due to branching/prediction

[1]

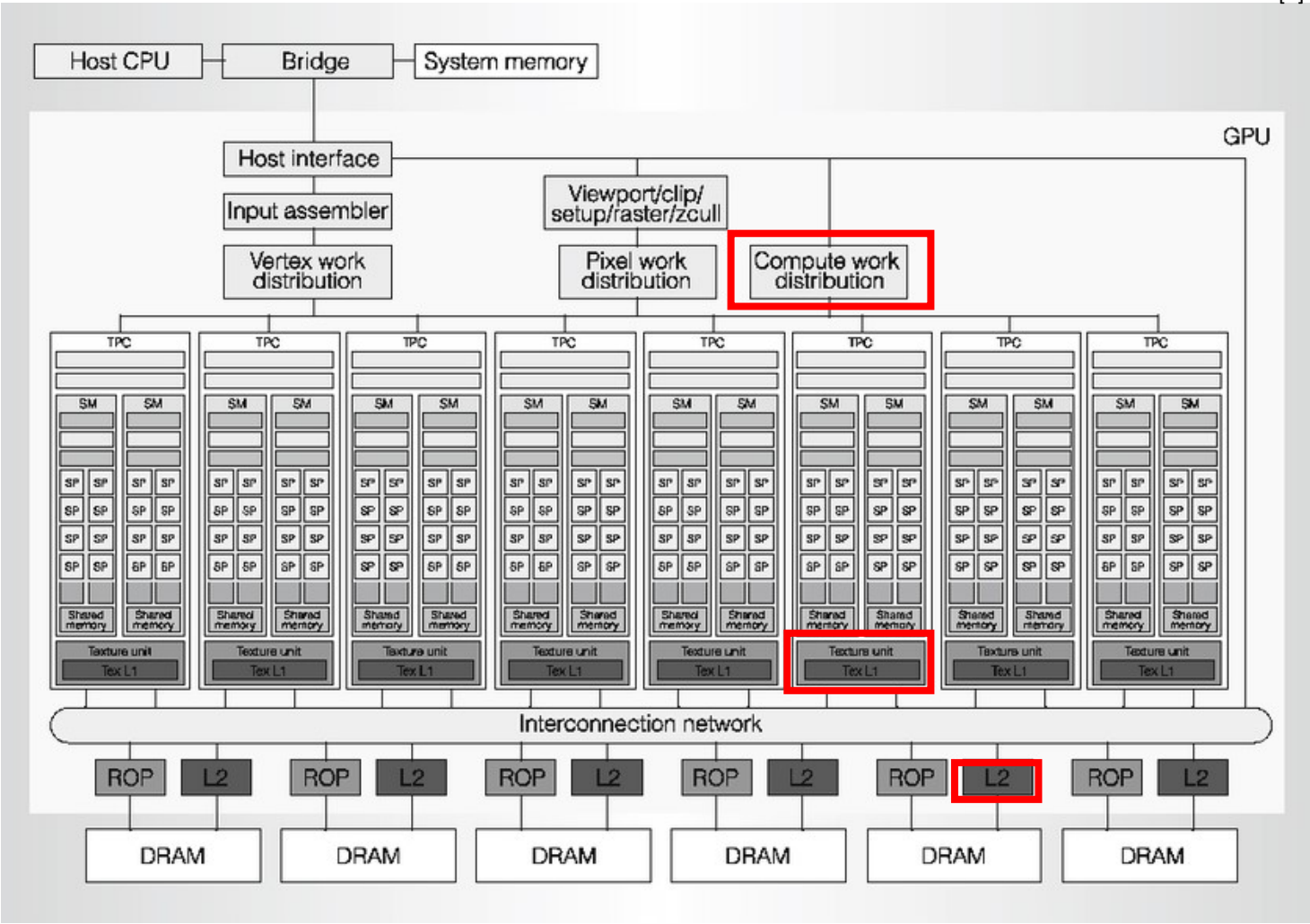


History of Architecture

- Tesla
- Fermi
- Kepler
- Maxwell
- Pascal
- Ampere
- Hopper
- Blackwell

Tesla (2008)

[1]


Host CPU
Bridge
System memory
GPU
Host interface
Input assembler
Vertex work distribution
Viewport/clip/ setup/raster/zcull
Pixel work distribution
Compute work distribution
TPC
SM
SP
Shared memory
Texture unit
Tex L1
Interconnection network
ROP
L2
DRAM

Tesla (2008)

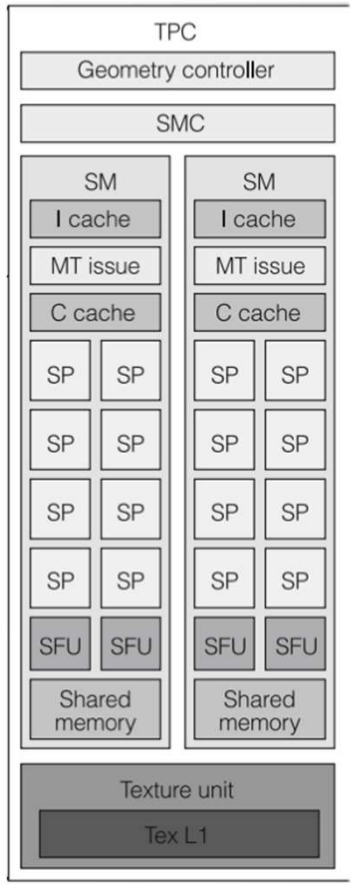
- **Specs**

Chip	Fabric	TPC#	SM / TPC	Core/SM	Total # of Cores	Max # of warps in SM	Max # of threads in pool	Max # of warps in GPU	L1 Cache (per TPC)	L2 Cache (per memory partition)
G80	90nm	8	2	8	128	24	768	384	24KB texture	32KB texture
G92	65nm	8	2	8	128	24	768	384	24KB texture	32KB texture
GT200	65nm	10	3	8	240	32	1,024	960	24KB texture	32KB texture

Tesla (2008) (Cont'd)

- **SM**
  - Texture/Processor Cluster (TPC)
    - Each attached with ROP, L2
    - Each containing 2~3 SMs
  - 8 Streaming processors (SPs)
    - Floating-point operations (FP32, FP64)
  - 2 Special function units (SFUs)
    - Single-precision transcendental
    - Planar attribute interpolation (graphic)

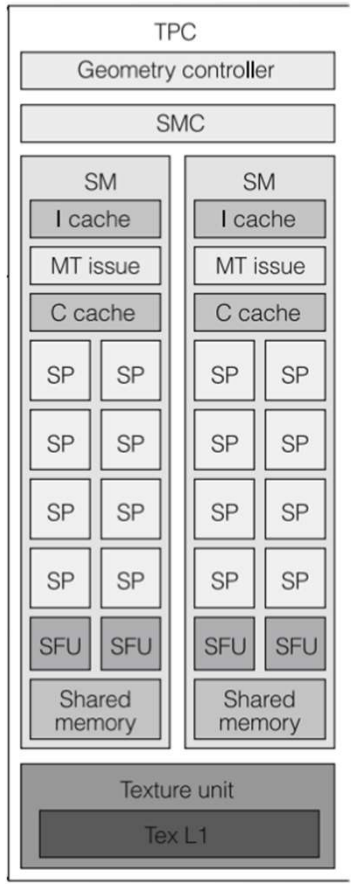
[1]



Tesla (2008) (Cont'd)

- **SM**
  - Multithreaded instruction fetch and issue unit (MT issue)
  - Instruction cache, Constant cache
  - SM Controller (SMC)
    - Load balancing

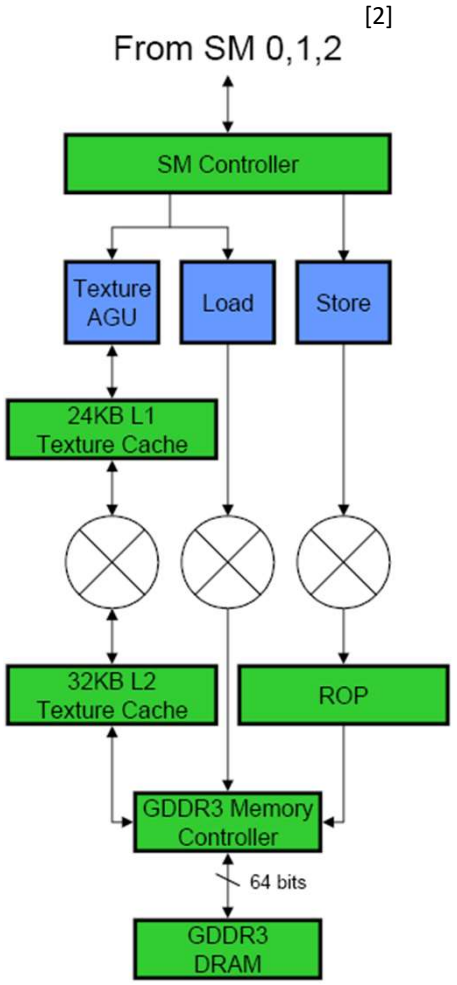
[1]



Tesla (2008) (Cont'd)

- **Memory**
  - ROP
    - Final processing for aid in anti-aliasing
  - Cache
    - L1, L2 texture cache (read only)

[2]

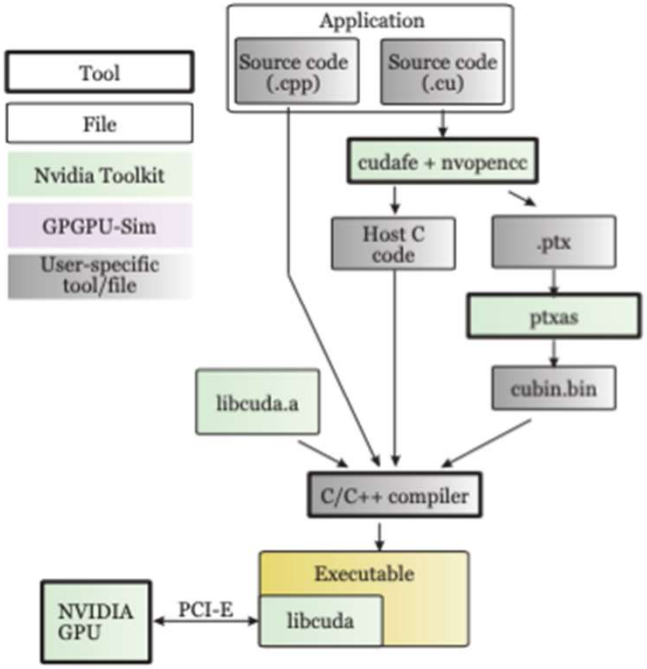


Tesla (2008) (Cont'd)

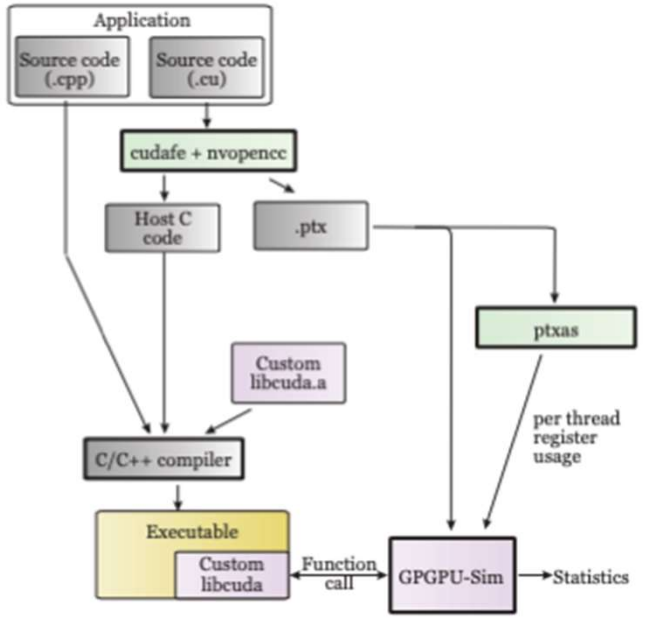
- **CUDA (Compute Unified Device Architecture)**
  - Platform for general-purpose parallel computing
  - CUDA 1.0 (CUDA C)
    - Base programming model for GPU
  - CUDA 1.1
    - Atomic functions
    - Asynchronous memcpy/streams/events
  - CUDA 2.0
    - FP64, 3D texture
    - cuda-gdb, Visual profiler (nvprof)

Related Studies (Tesla) (Cont'd)

- **Simulator**
  - Analyzing CUDA Workloads Using a Detailed GPU Simulator (GPGPU-sim) [15]



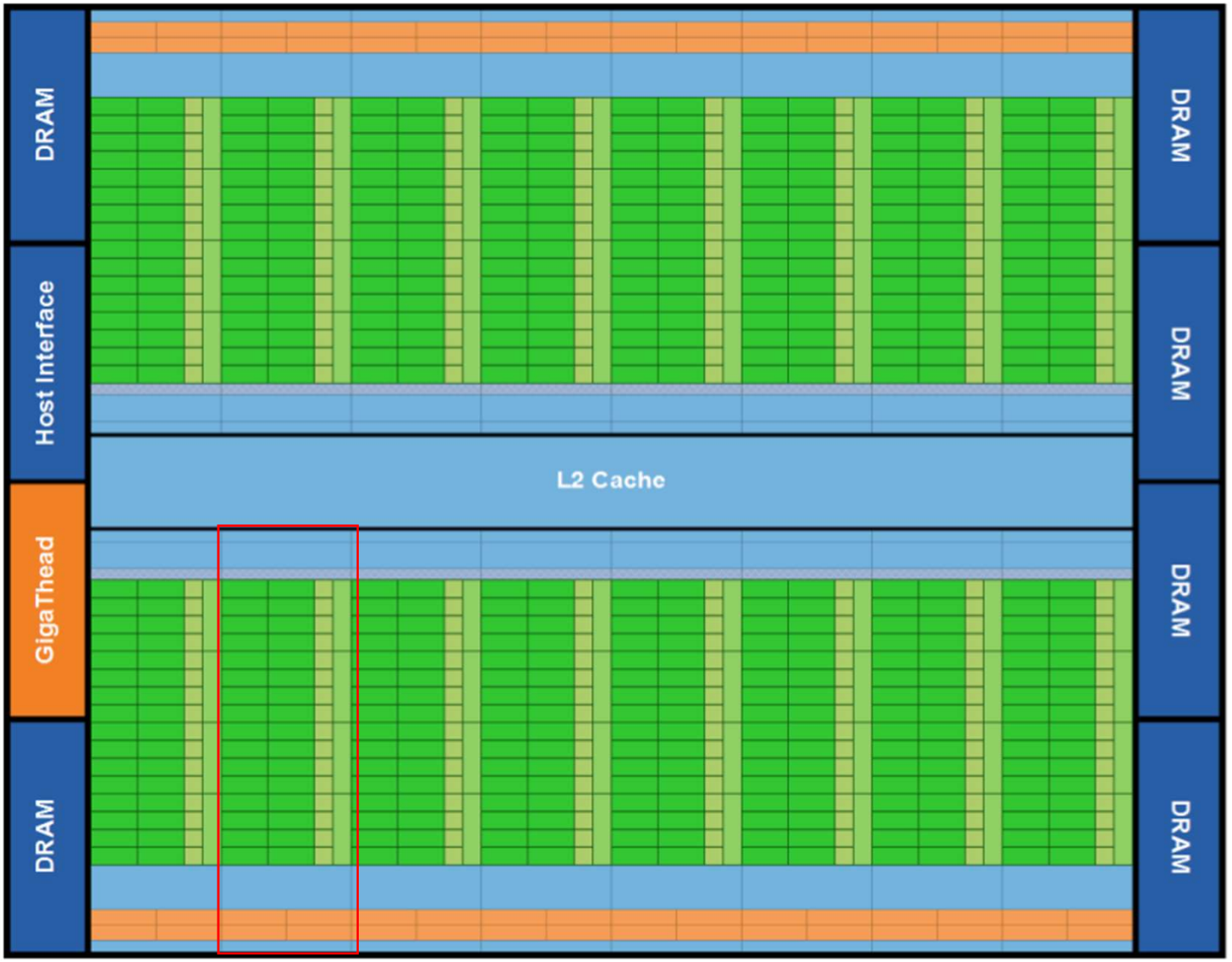
(a) CUDA Flow with GPU Hardware



(b) GPGPU-Sim

Fermi (2010)

[3]


DRAM
Host Interface
GigaThead
DRAM
DRAM
DRAM
DRAM
DRAM
DRAM
L2 Cache

Fermi (2010)

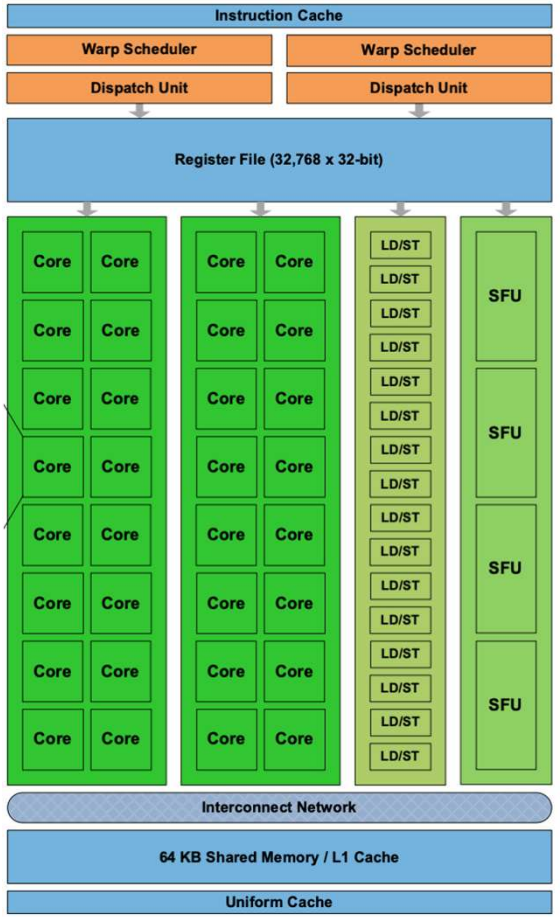
- **Updated specs**
  - No TPC -> GPCs (Graphics Processing Clustsers)
  - Configurable L1/Shared memory
  - L2 uniform cache

Archi.	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core/ SM	Total # of Cores	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Tesla	GT200	65nm	—	10	30	3	—	8	240	1	32	1,024	960	24KB texture (per TPC)	—	32KB texture (per partition)
Fermi	GF100	40nm	4	—	16	—	4	32	512	2	48	1,536	768	64KB Total, Configurable (16/48 or 48/16)		768KB
Fermi	GF110	40nm	4	—	16	—	4	32	512	2	48	1,536	768	64KB Total, Configurable (16/48 or 48/16)		768KB
Fermi	GF104	40nm	2	—	8	—	4	48	384	2	48	1,536	384	64KB Total, Configurable (16/48 or 48/16)		512KB

Fermi (2010) (Cont'd)

- **SM**
  - 32 CUDA cores
  - 4 SFUs
  - 16 Load/Store units
    - Address calculated 16 threads/clock
  - 64KB Shared/L1 cache
    - Configurable
- **L2 cache**
  - Uniformed into single L2 cache
  - Support all operations (load,store,texture)

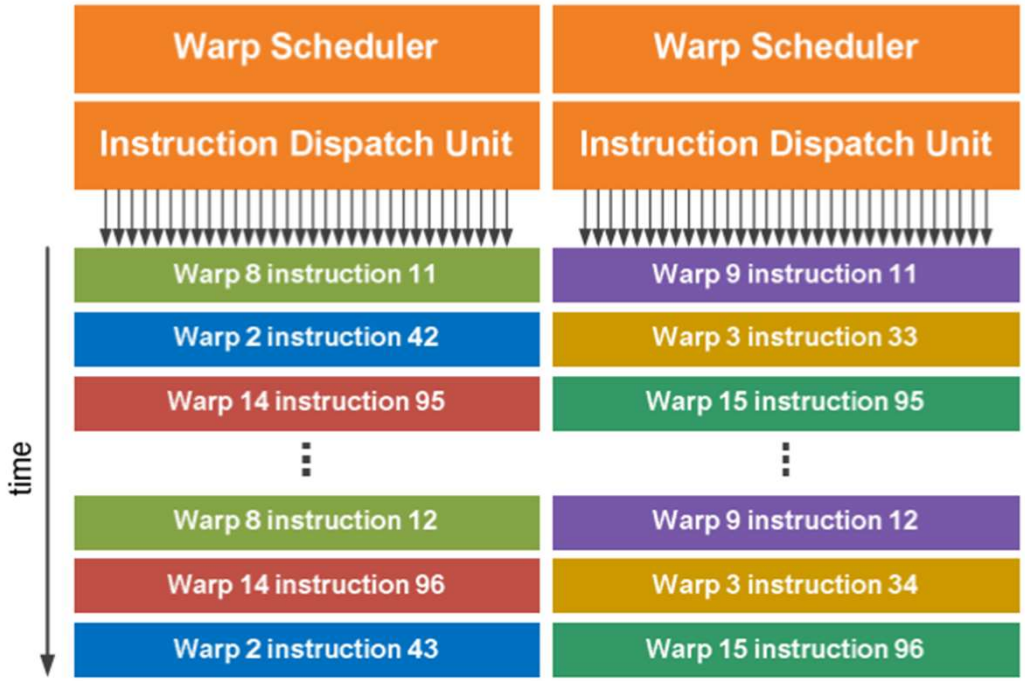
[3]



Fermi (2010) (Cont'd)

- **SM**
  - Dual warp scheduler
    - 2 warps can be issued & executed
      - Half of SM cores to a warp

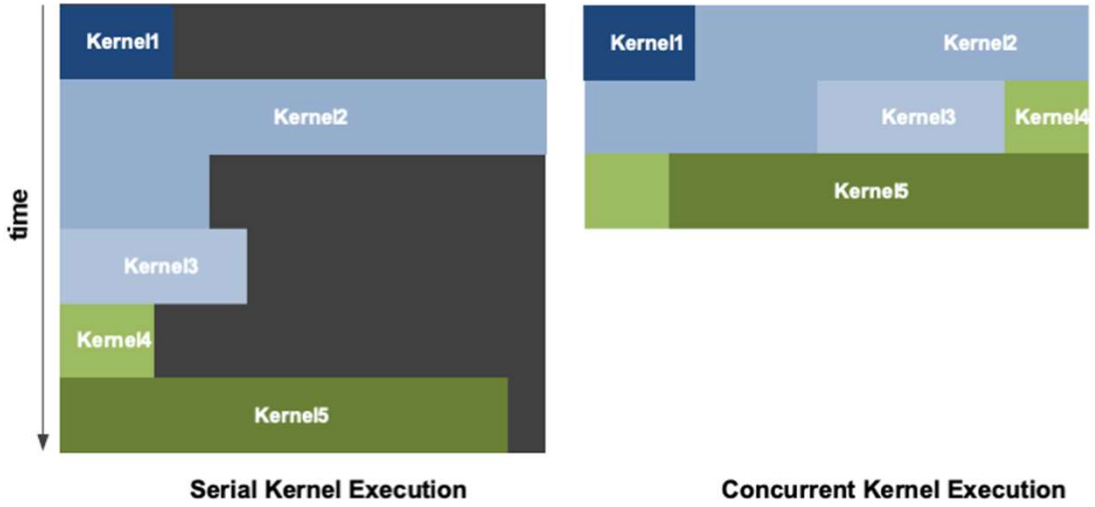
[3]



Fermi (2010) (Cont'd)

- **Architecture**
  - First Error Correcting Code (ECC) protection
  - GigaThread thread scheduler
    - Concurrent kernel execution

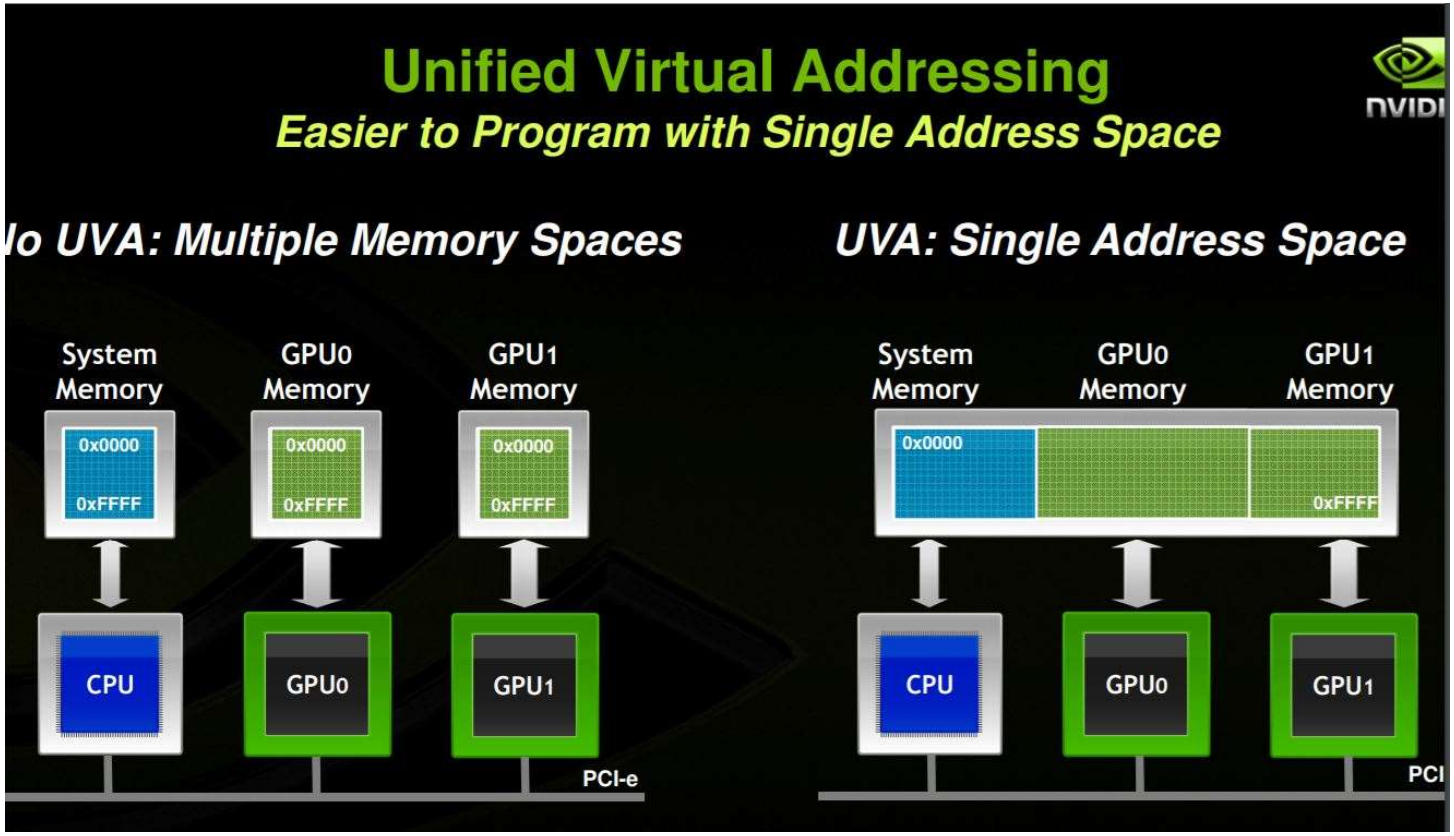
[3]



Fermi (2010) (Cont'd)

- **CUDA Updates**
  - Unified Virtual Addressing
    - Mapping memory address as unified manner

[4]



Fermi (2010) (Cont'd)

- **CUDA Updates**
  - cuSPARSE, cuRAND
    - Pre-built CUDA kernels for sparse matrix multiplication & random numbers
  - 64-bit address expansion
  - LLVM as Compiler backend

Kepler (2012)

[5]



Kepler (2012)

- **Updated specs**
  - Read only L1 cache
  - Explosive increase of cores & SMs

Archi.	Chip	Fabric	GPC#	Total SMs	SM / GPC	Core/ SM	Total # of Cores	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Fermi	GF104	40nm	2	8	4	48	384	2	48	1,536	384	8	64KB Total, Configurable (16/48 or 48/16)		512KB
Kepler	GK104	28nm	4	8	2	192	1,536	4	64	2,048	512	16	64KB Total, Configurable (16/48, 32/32, 48/16) 48KB Read-only L1 Data Cache		512KB
Kepler	GK110	28nm	5	15	3	192	2,880	4	64	2,048	960	16	64KB Total, Configurable (16/48, 32/32, 48/16) 48KB Read-only L1 Data Cache		1,536KB
Kepler	GK210	28nm	5	15	3	192	2,880	4	64	2,048	960	16	128KB Total, Configurable (16~112 / 112~16) 48KB Read-only L1 Data Cache		1,536KB

Kepler (2012) (Cont'd)

- **Architecture**
  - 192 CUDA cores
    - Single-precision
  - 64 Double-Precision (DP) units
  - 32 SFUs
  - 32 Load/Store units
  - Read-only data cache (48KB)

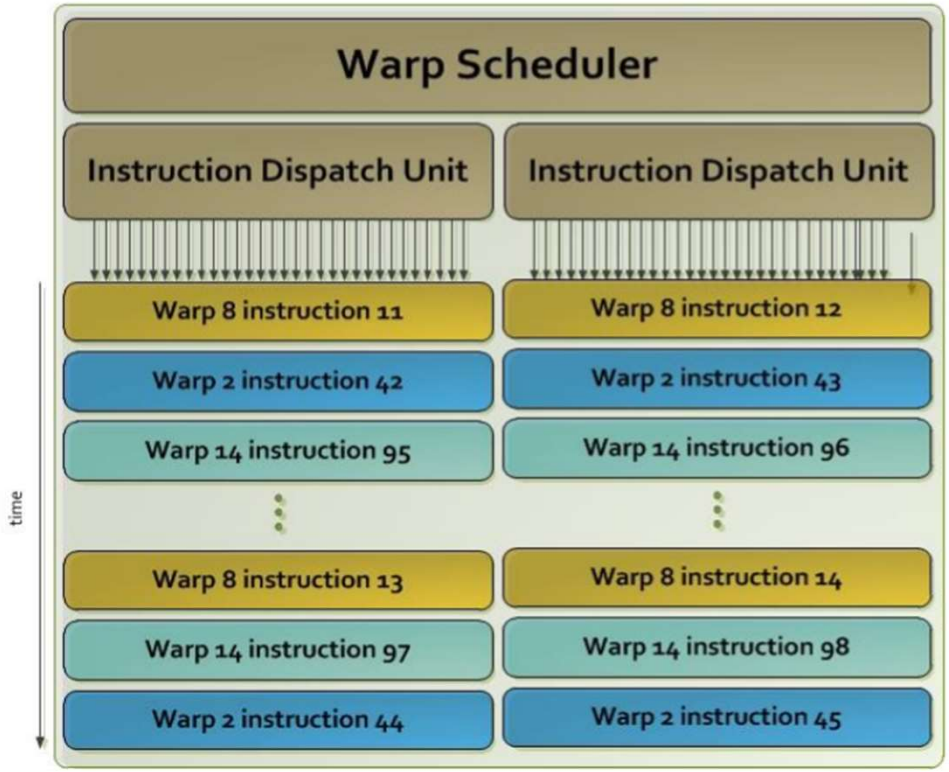
[5]



Kepler (2012) (Cont'd)

- **Architecture**
  - 4 warp schedulers
    - Each scheduler with 2 dispatches

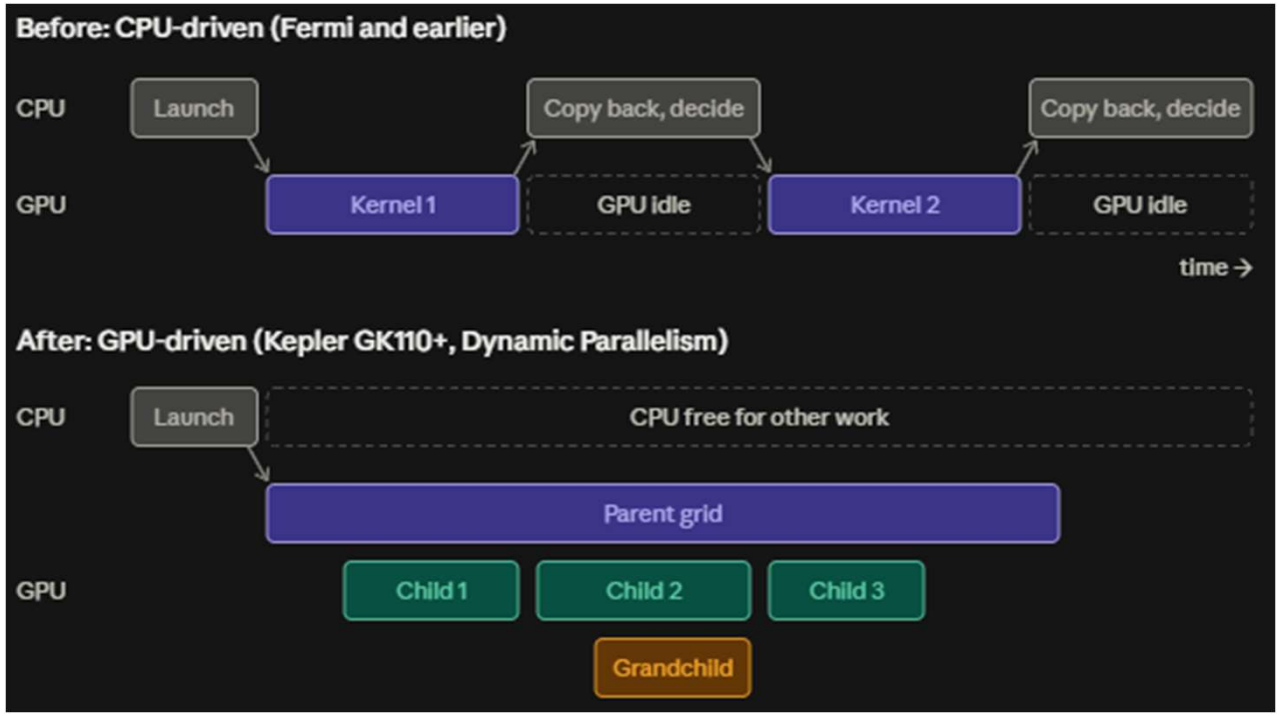
[5]



Kepler (2012) (Cont'd)

- **Architecture**
  - Dynamic Parallelism

[*]



Kepler (2012) (Cont'd)

- **Architecture**
  - Dynamic Parallelism

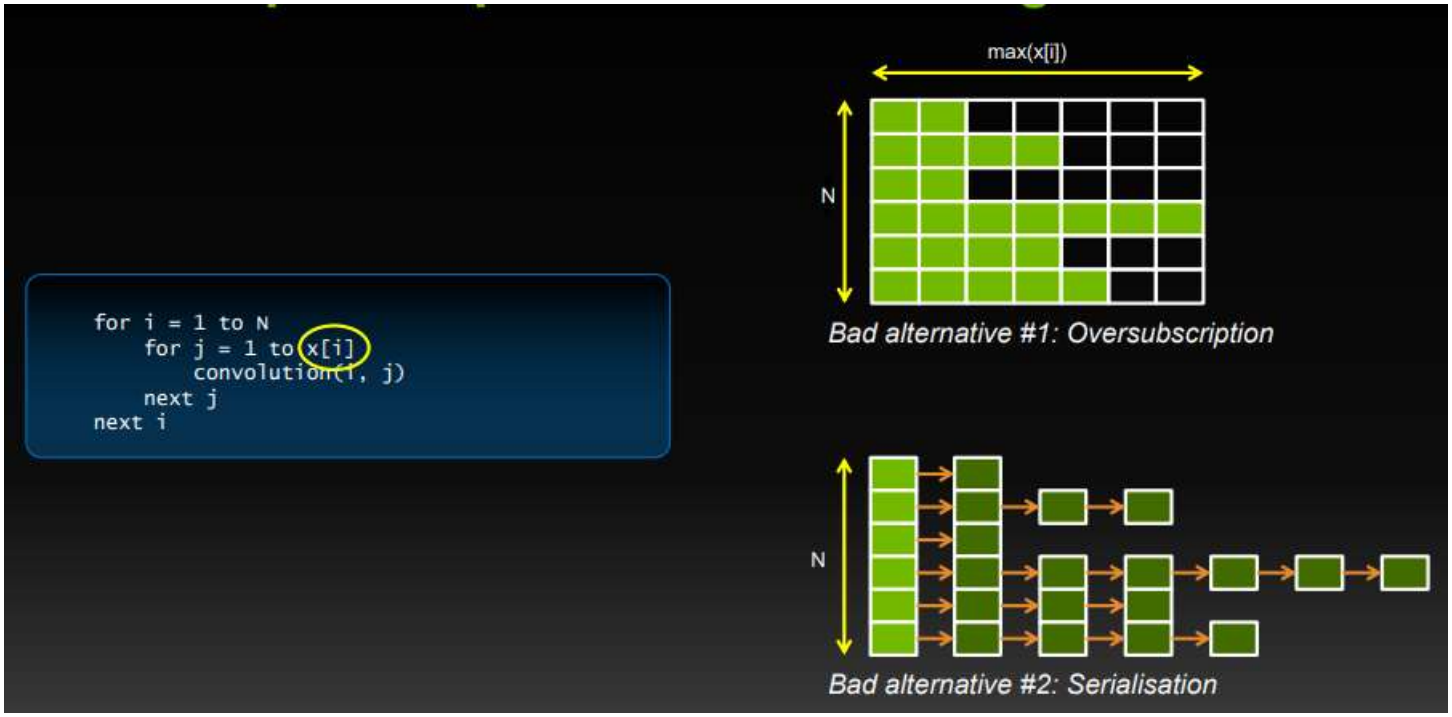
[6]



Kepler (2012) (Cont'd)

- **Architecture**
  - Dynamic Parallelism

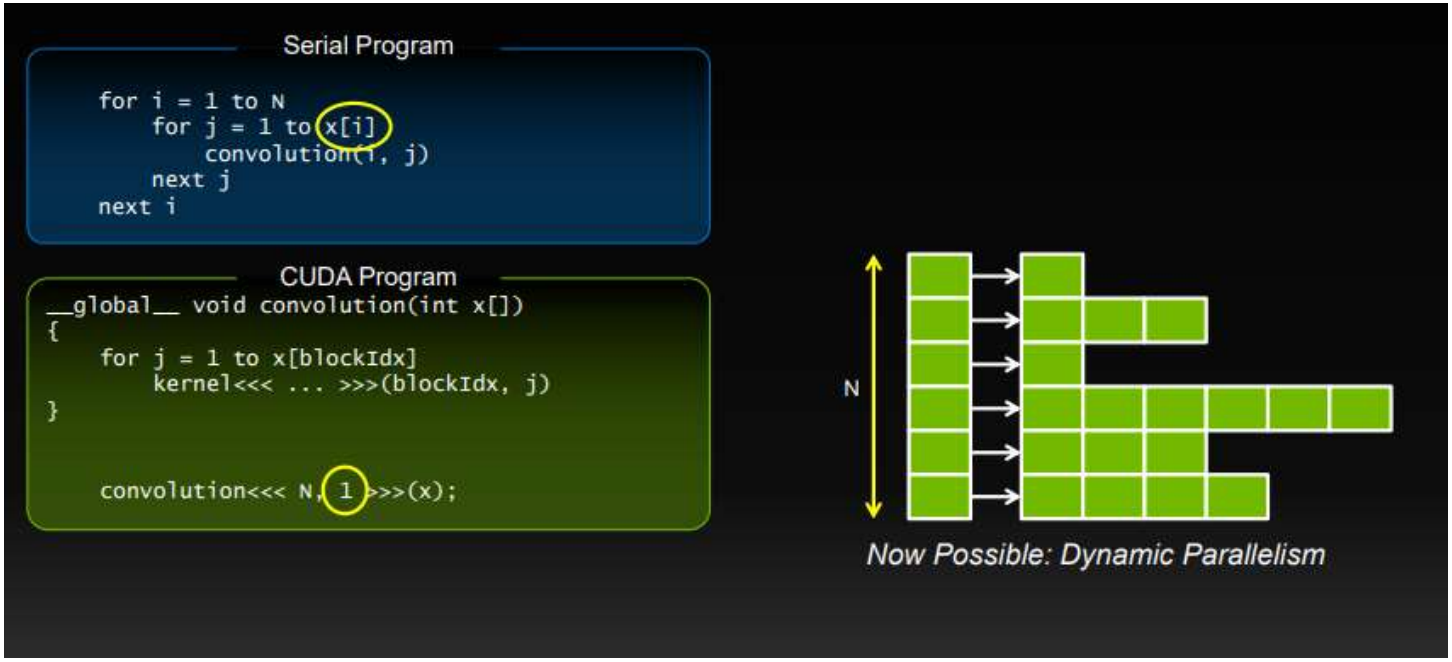
[6]



Kepler (2012) (Cont'd)

- **Architecture**
  - Dynamic Parallelism

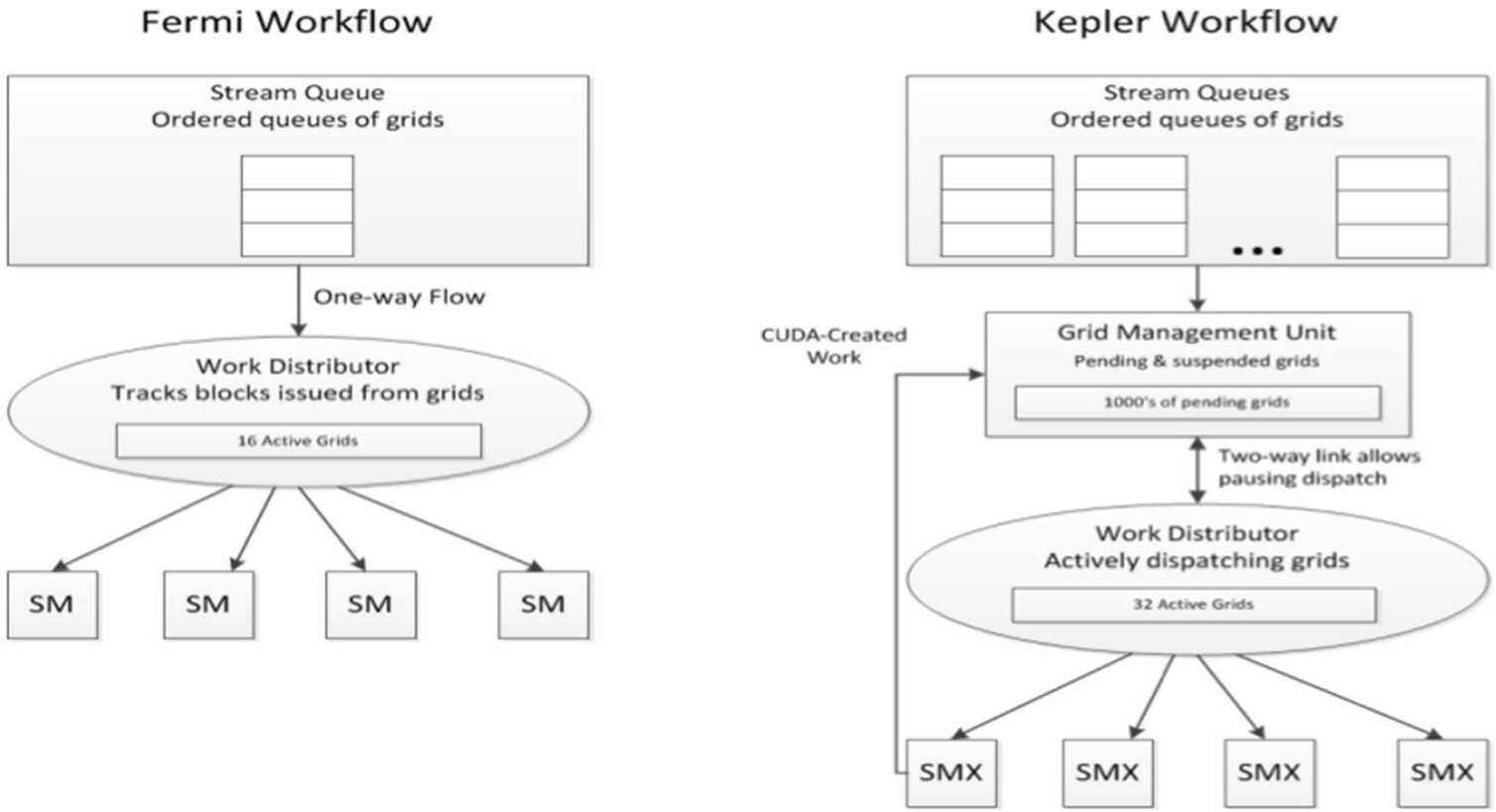
[6]



Kepler (2012) (Cont'd)

- **Architecture**
  - Grid Management Unit

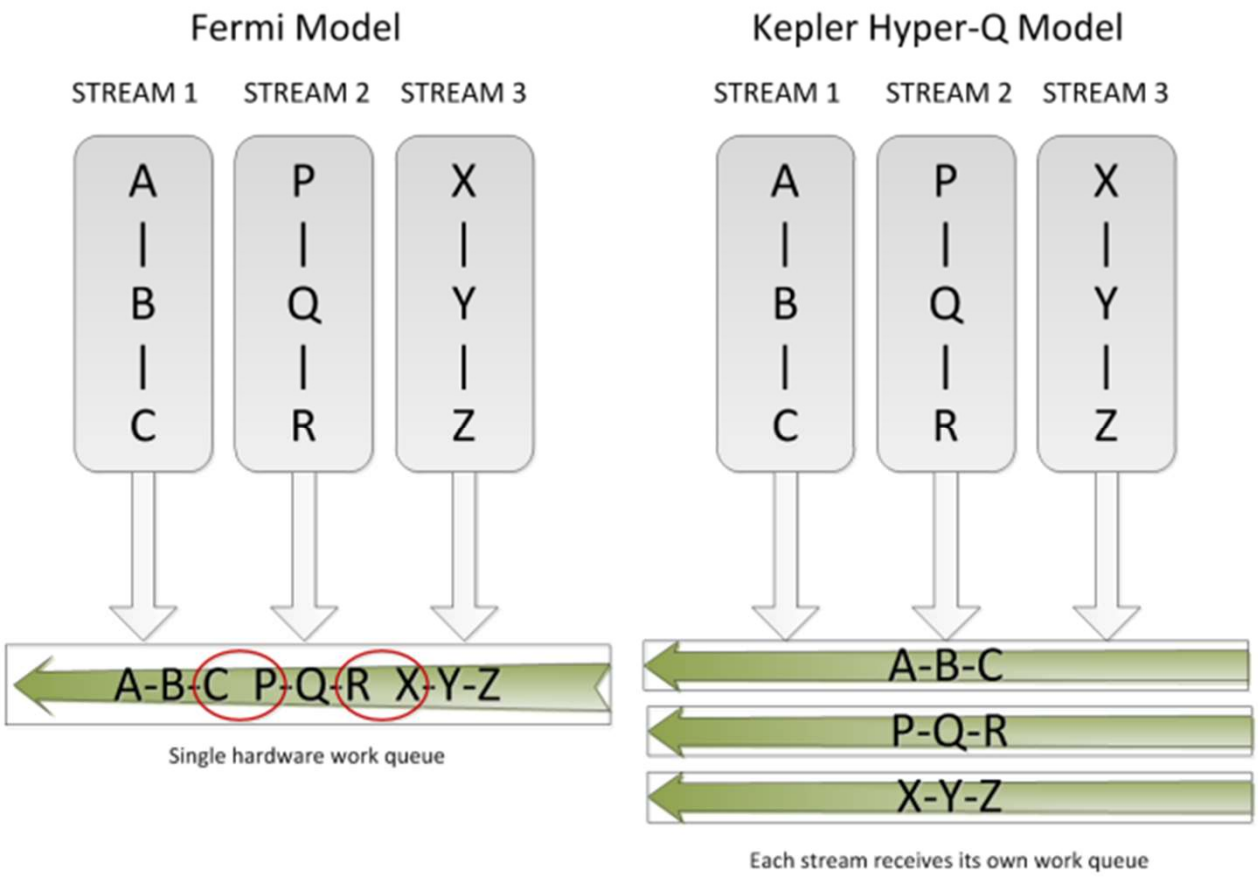
[5]



Kepler (2012) (Cont'd)

- **Architecture**
  - Hyper-Q

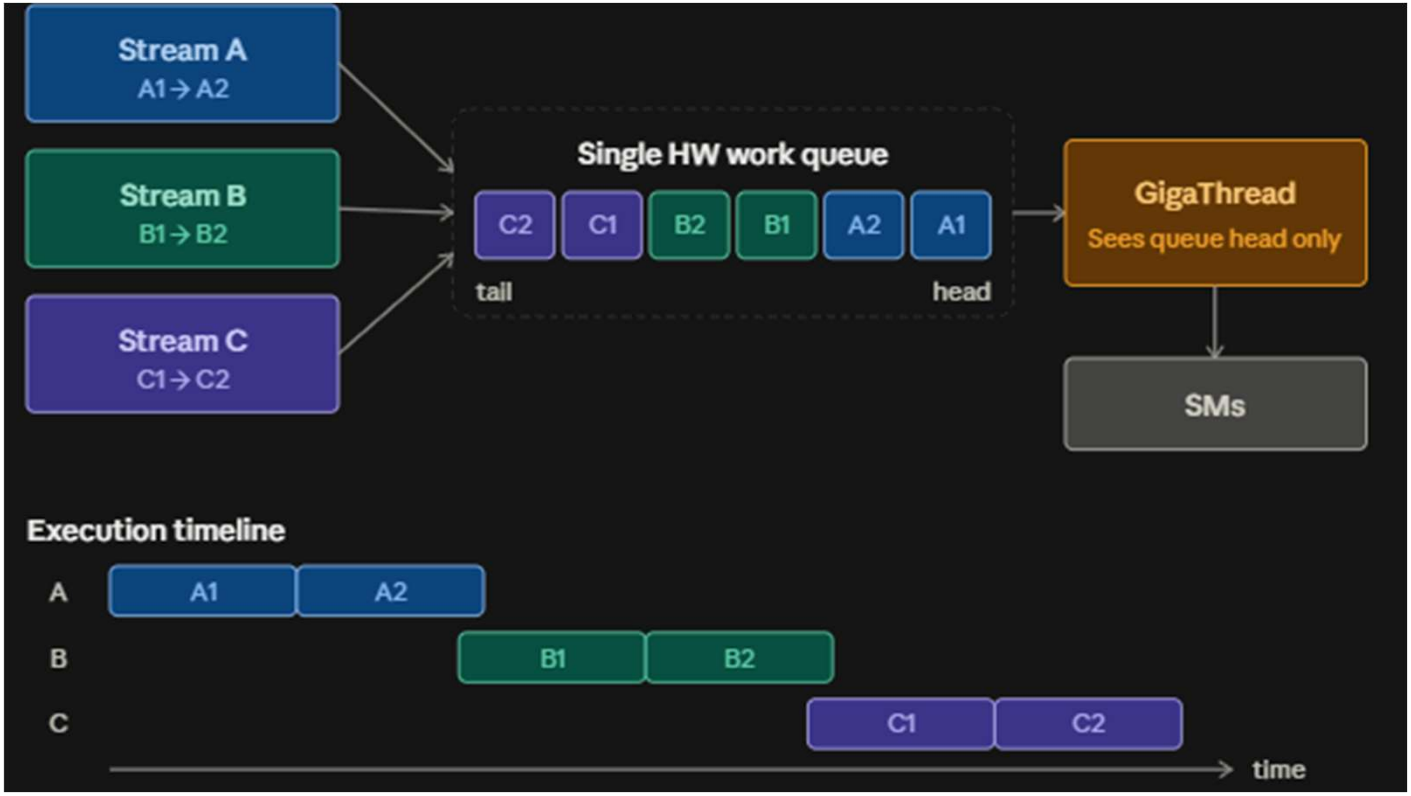
[5]



Kepler (2012) (Cont'd)

- **Architecture**
  - Hyper-Q

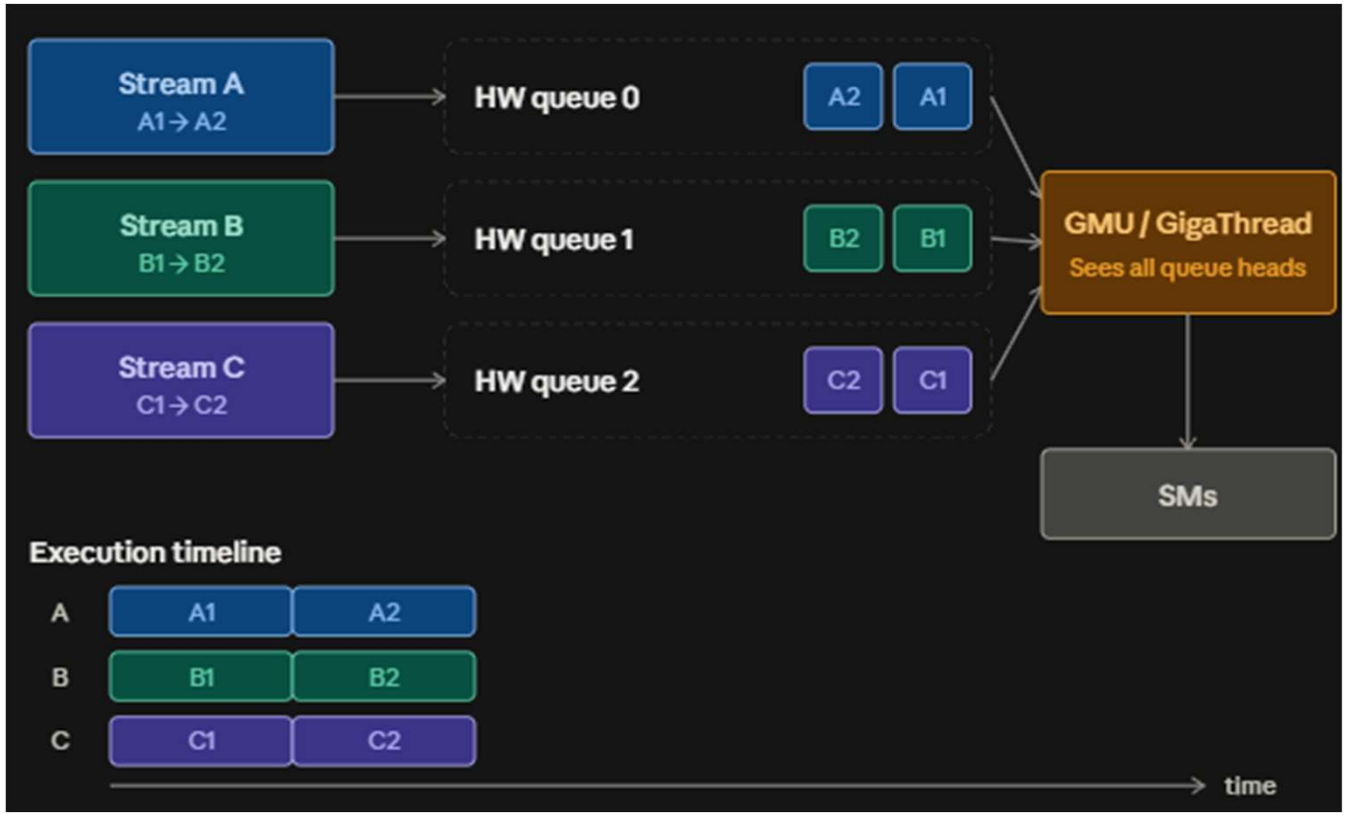
[*]



Kepler (2012) (Cont'd)

- **Architecture**
  - Hyper-Q

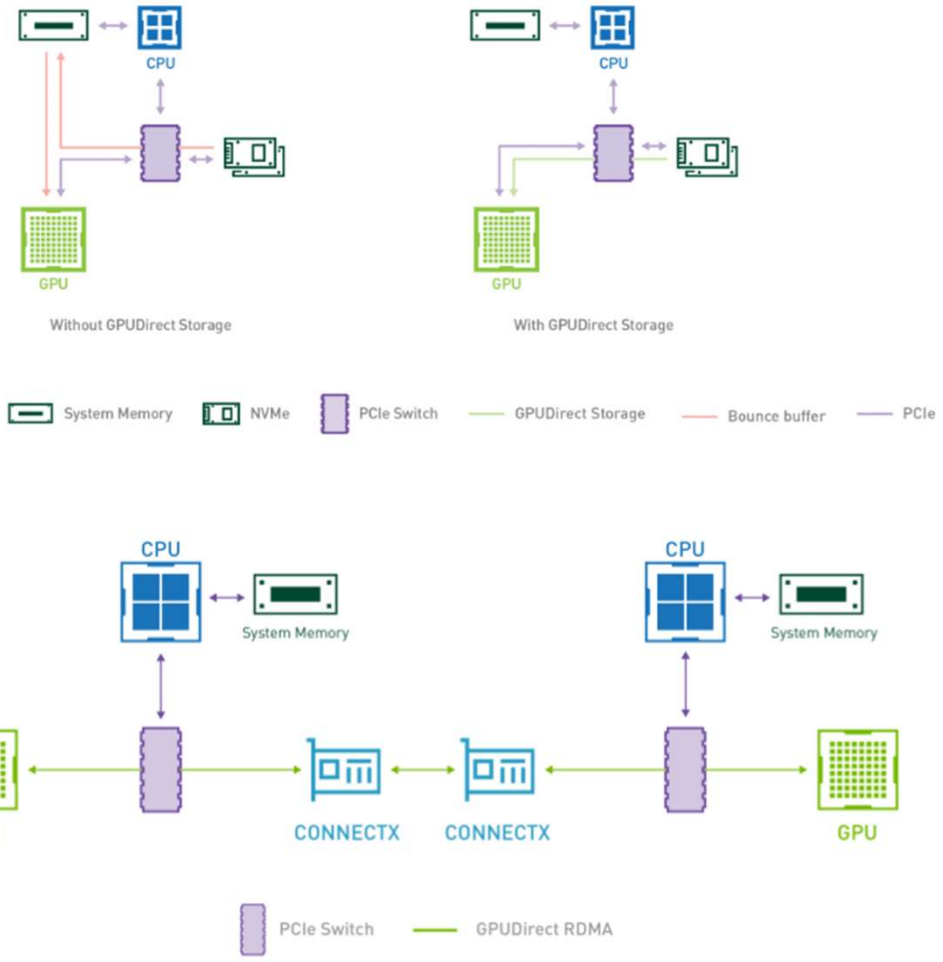
[*]



Kepler (2012) (Cont'd)

- **Architecture**
  - GPUDirect

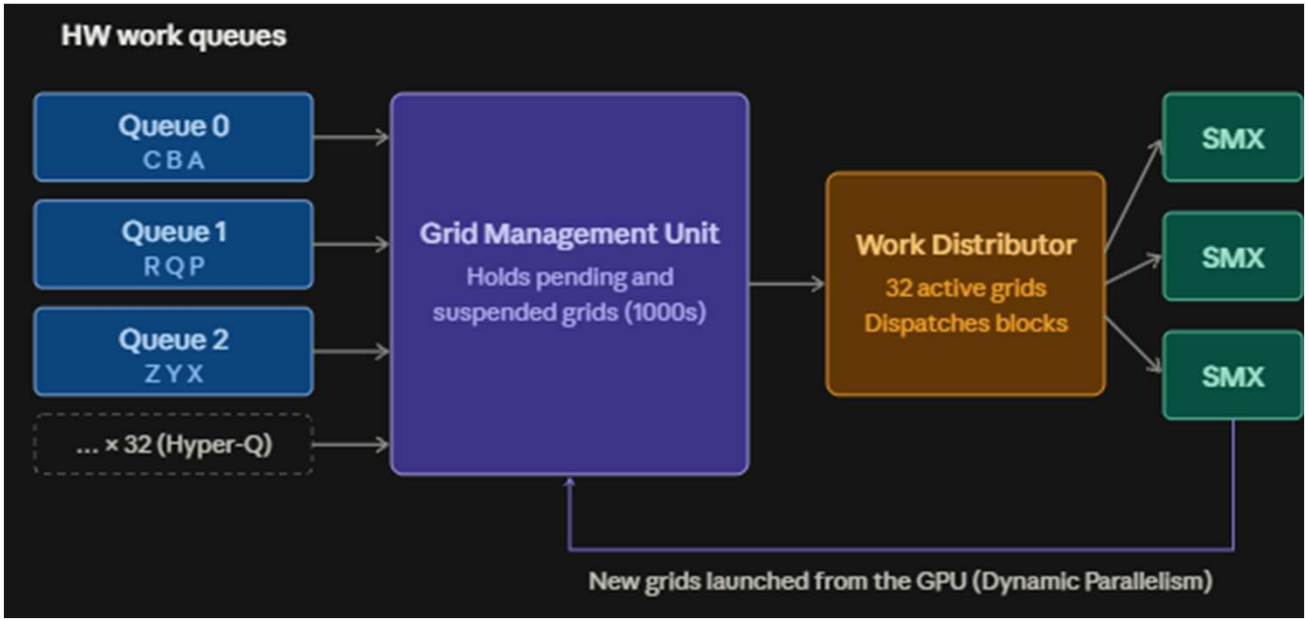
[5]



Kepler (2012) (Cont'd)

- **Architecture**
  - Grid Management Unit
    - For Dynamic Parallelism

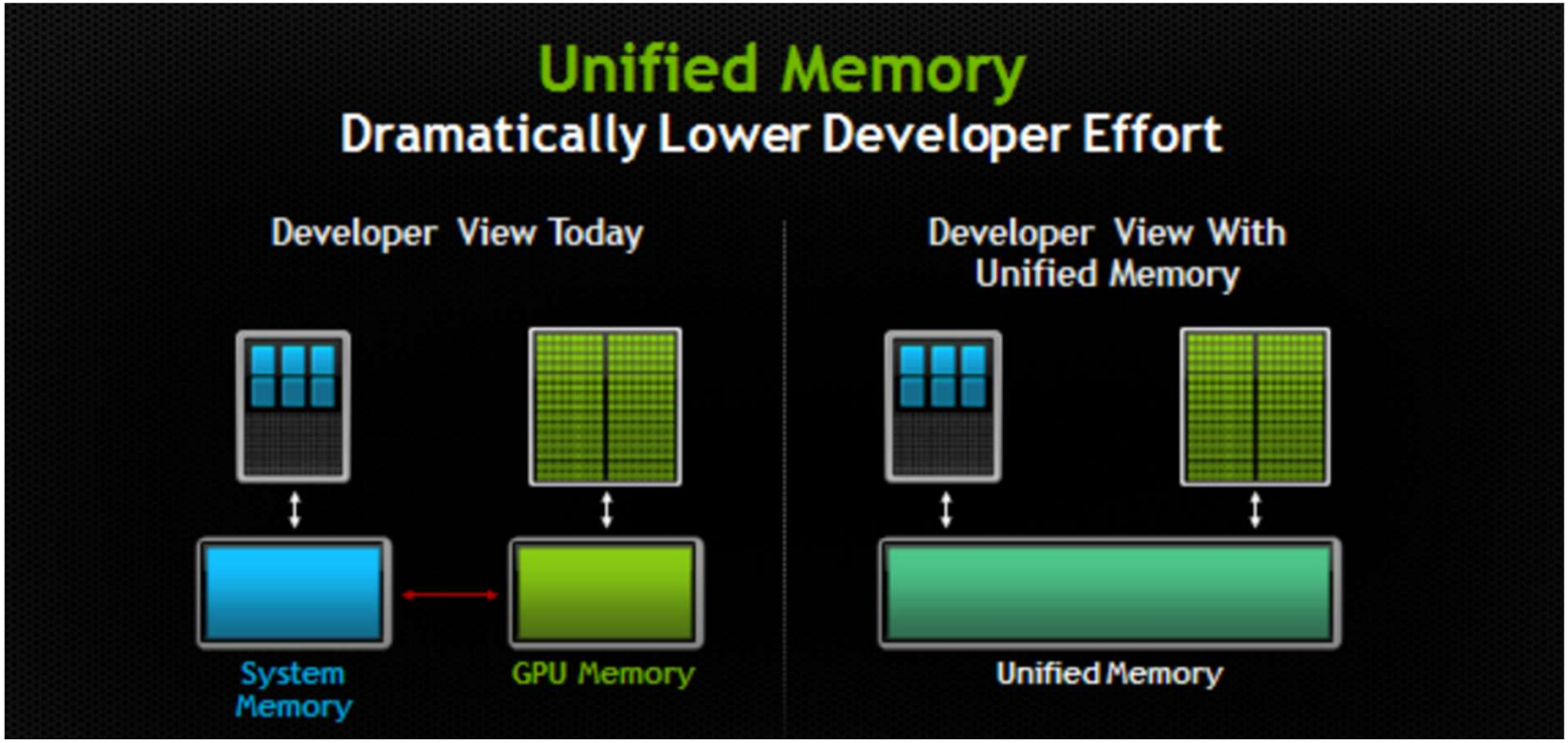
[*]



Kepler (2012) (Cont'd)

- **CUDA**
  - Unified Memory
    - Managing memory&copy as unified view
    - cudaMallocManaged

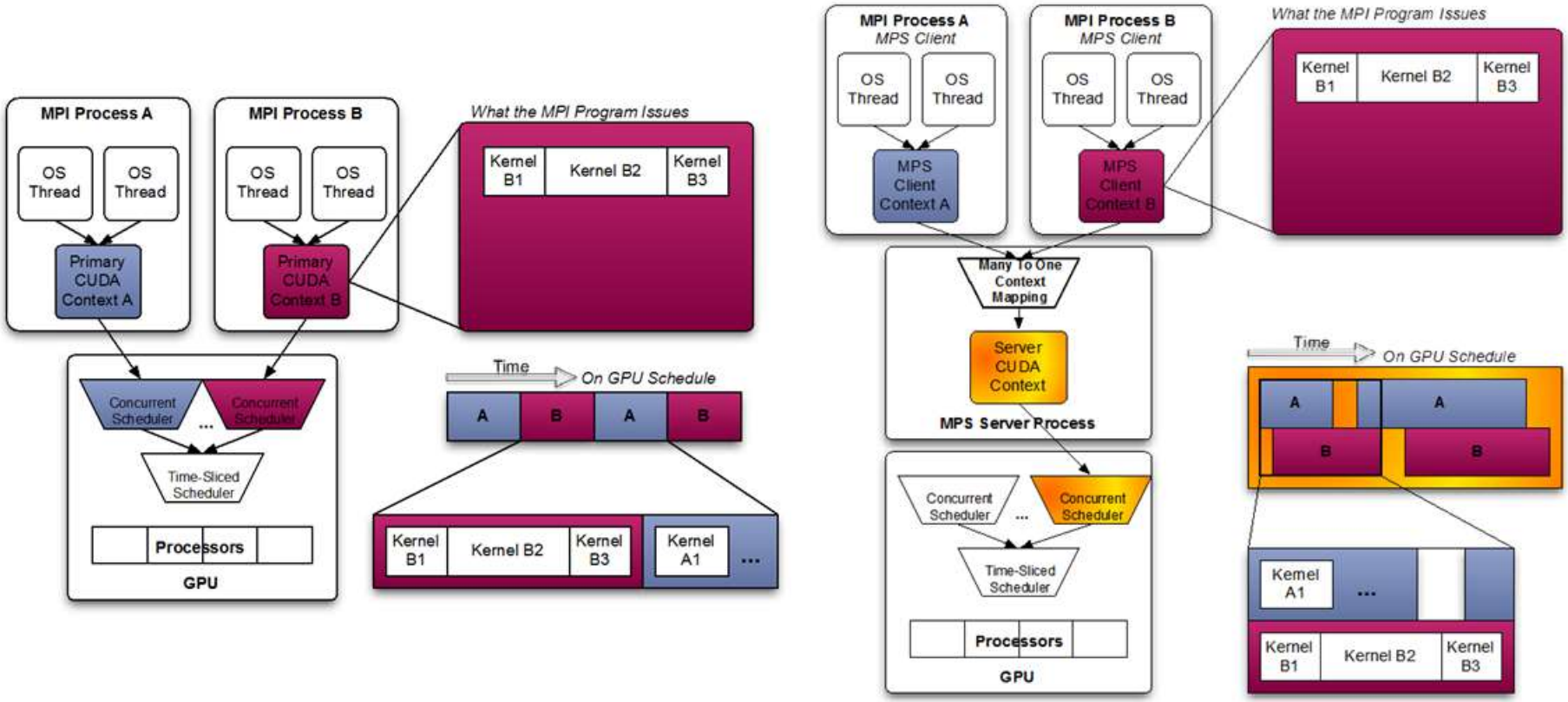
[5]



Kepler (2012) (Cont'd)

- **CUDA**
  - MPS (CUDA Proxy)

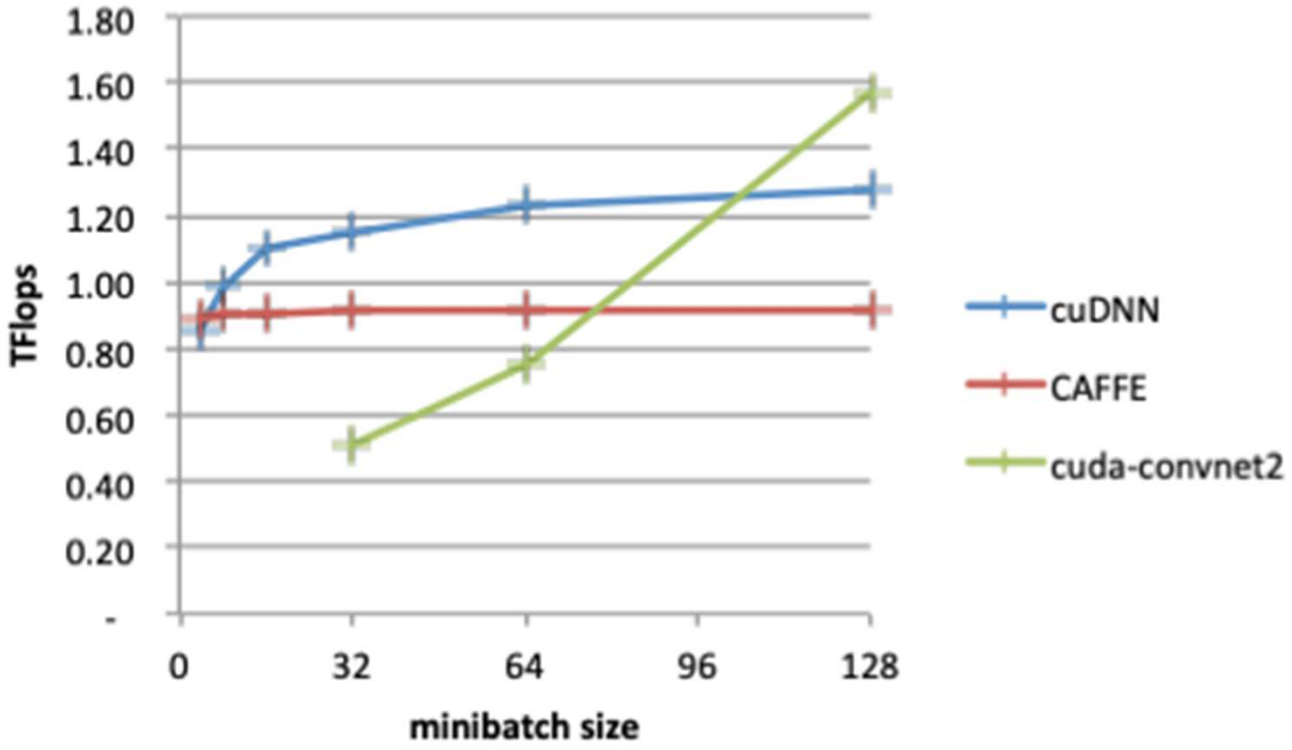
[7]



Kepler (2012) (Cont'd)

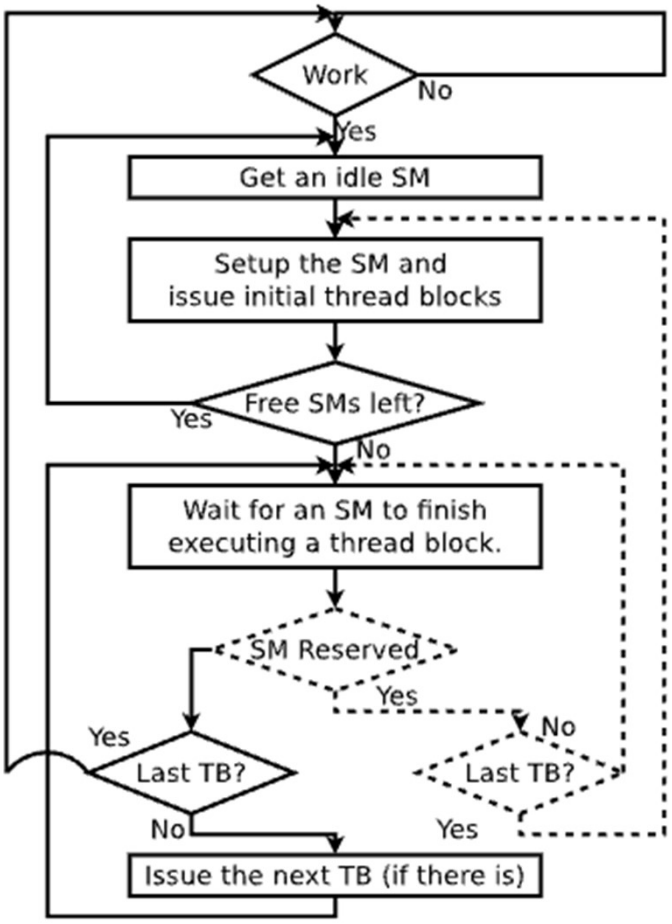
- **CUDA**
  - cuDNN (CUDA Deep Neural Network Library)
    - Library for deep learning computations

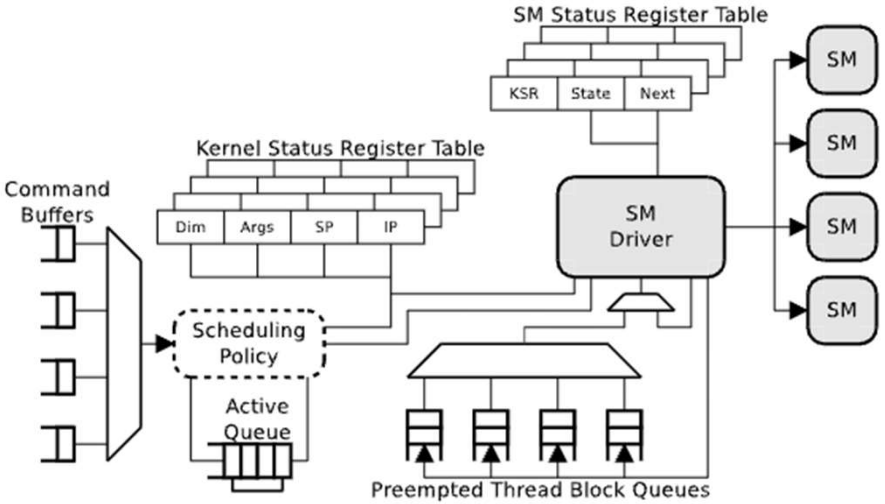
[8]



Related Studies (Kepler)

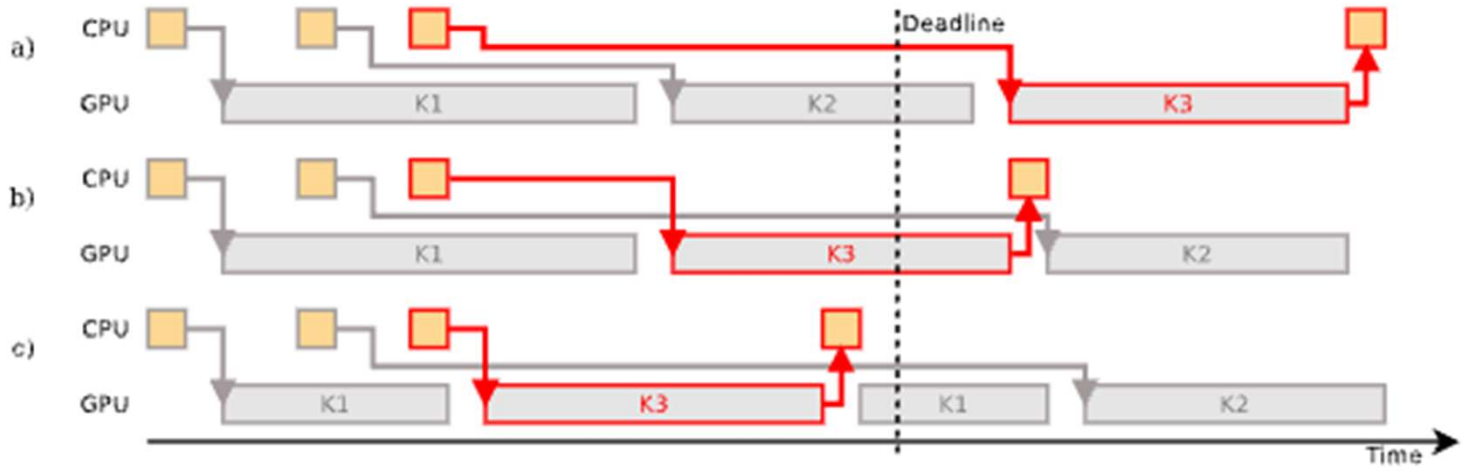
- **Hyper-Q**
  - Enabling preemptive multiprogramming on GPUs [35]





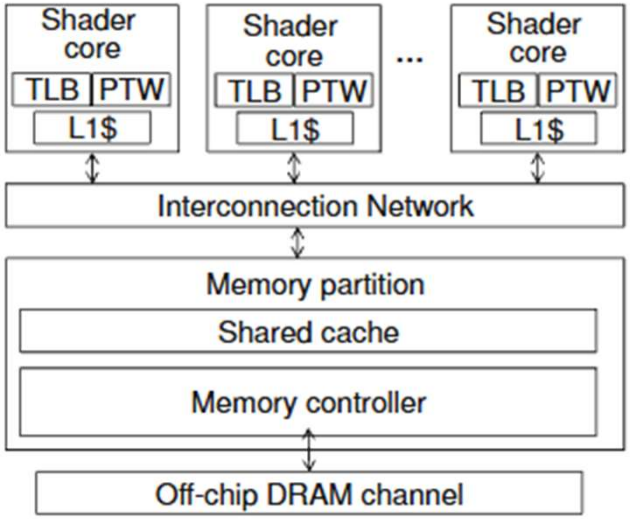
Related Studies (Kepler)

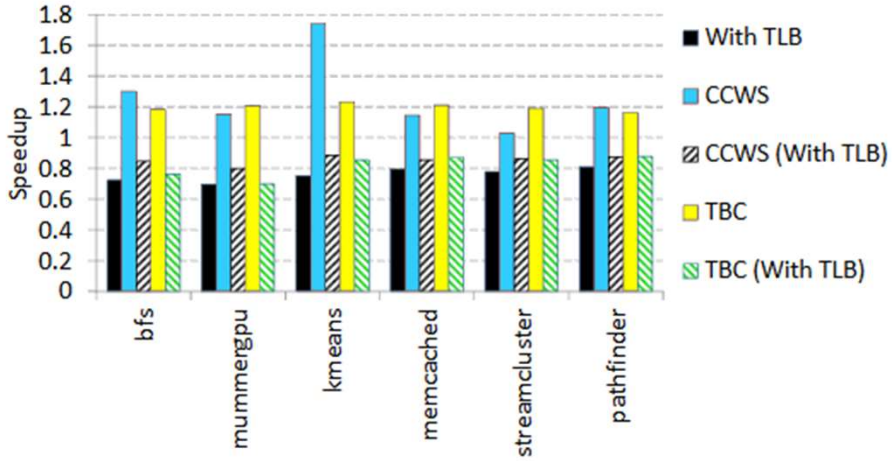
- **Hyper-Q**
  - Enabling preemptive multiprogramming on GPUs [35]



Related Studies (Kepler) (Cont'd)

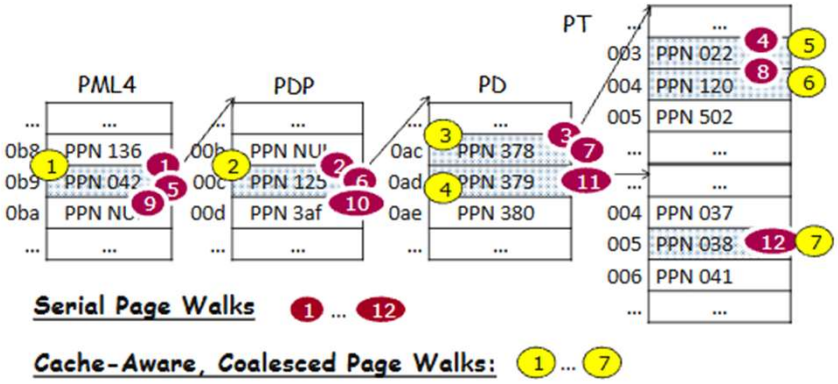
- **Unified Memory**
  - Architectural Support for Address Translation on GPUs [36]
    - Focus on performance degrade by Naïve TLB
    - Requirement of MMU for GPU

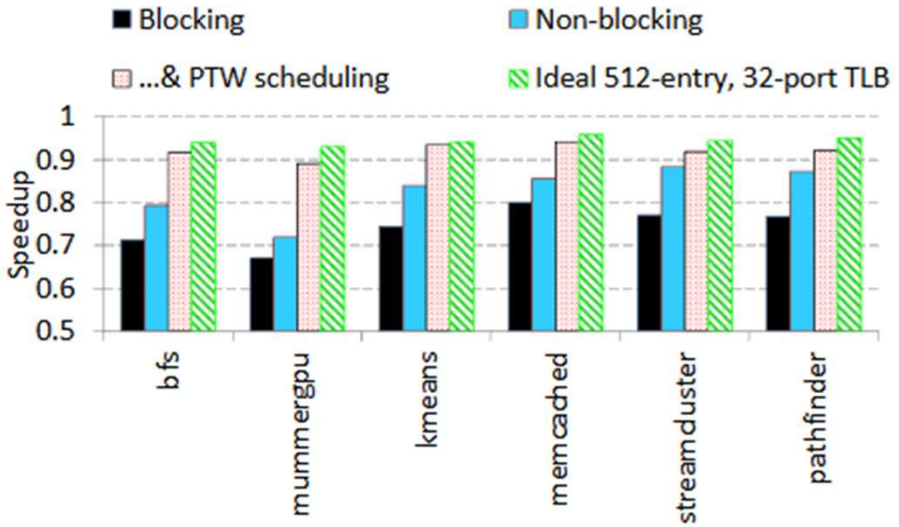




Related Studies (Kepler) (Cont'd)

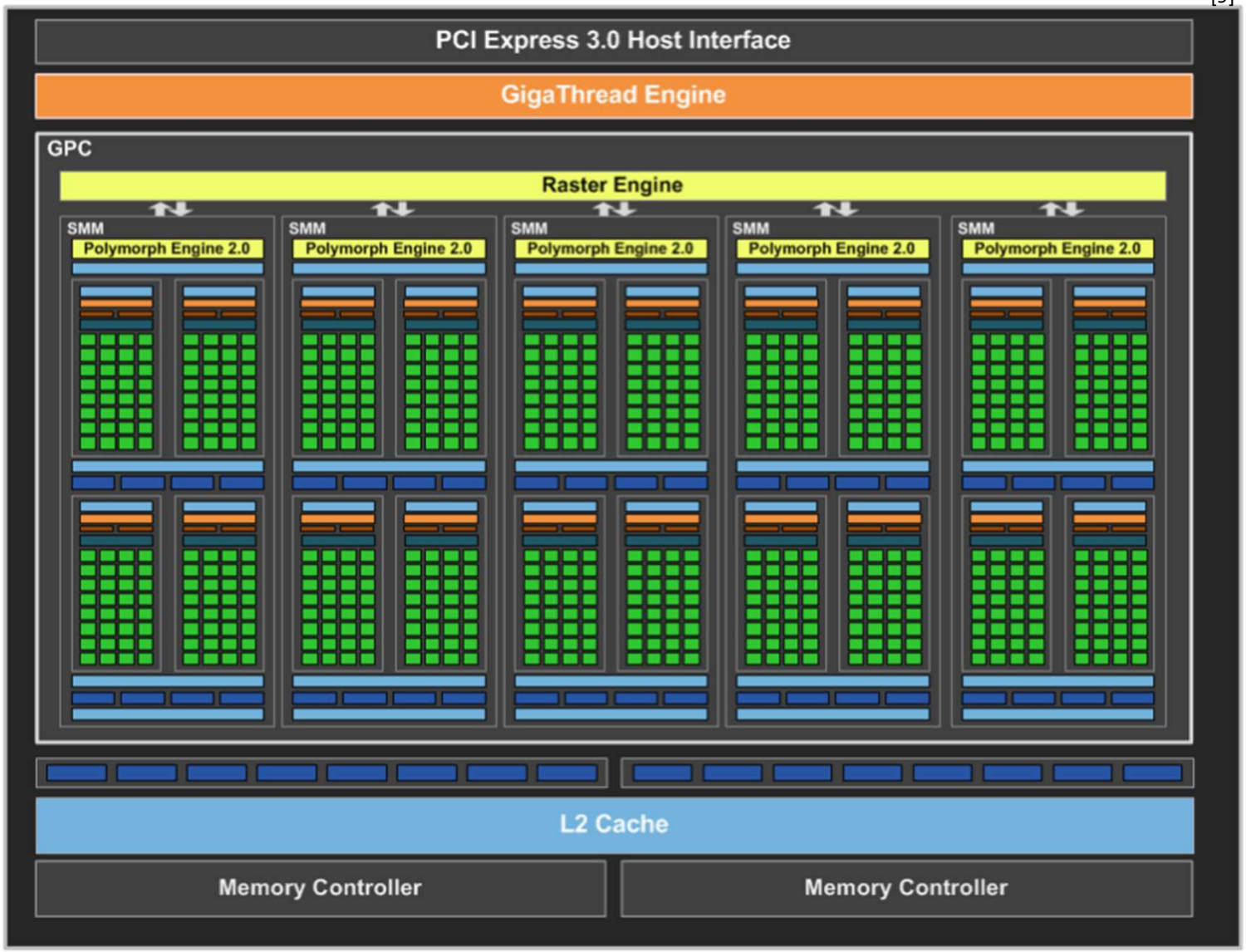
- **Unified Memory**
  - Architectural Support for Address Translation on GPUs [36]
    - Focus on performance degrade by Naïve TLB
    - Requirement of MMU for GPU





Maxwell (2014)

[9]



Maxwell (2014)

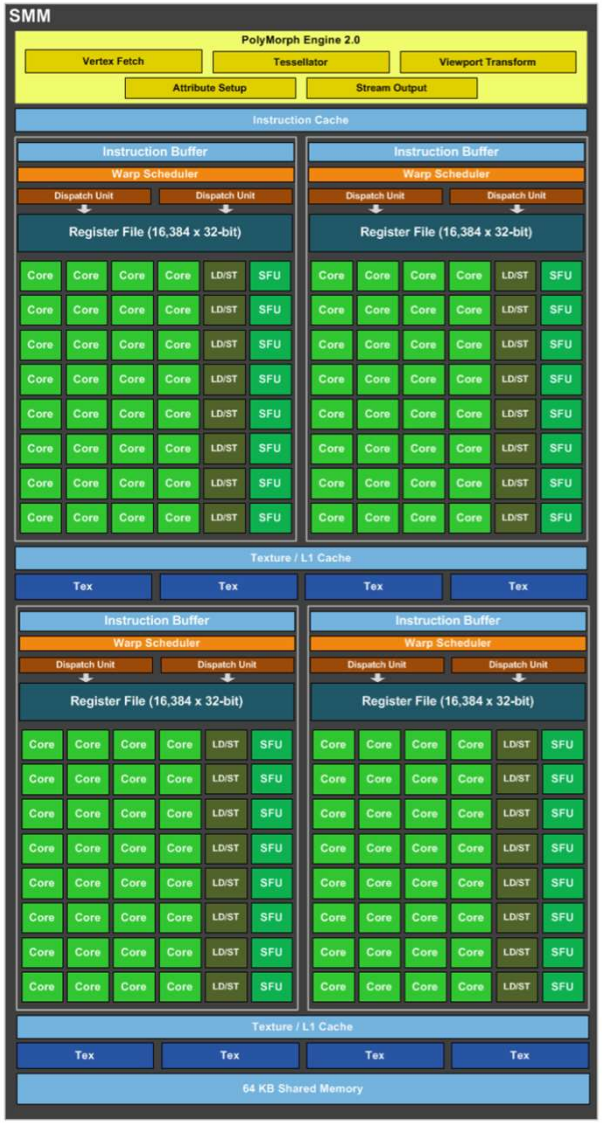
- **Updated specs**
  - Subsections in SM
  - Separated L1 / Shared memory
  - Embedded GPU chip

Archi.	Chip	Fabric	GPC#	Total SMs	SM / GPC	Core/ SM	Total # of Cores	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Kepler	GK210	28nm	5	15	3	192	2,880	4	64	2,048	960	16	128KB Total, Configurable (16~112 / 112~16) 48KB Read-only L1 Data Cache		1,536KB
Maxwell	GM107	28nm	1	5	5	128	640	4	64	2,048	320	32	24KB ×2	64KB	2,048KB
Maxwell	GM204	28nm	4	16	4	128	2,048	4	64	2,048	1,024	32	24KB ×2	96KB	2,048KB
Maxwell	GM200	28nm	6	24	4	128	3,072	4	64	2,048	1,536	32	24KB ×2	96KB	3,072KB

Maxwell (2014) (Cont'd)

- **SM**
  - 4 quadrant sections
    - 2 sections mapped with different L1 Cache
    - Higher utilization & power efficiency
  - Separated L1, Shared memory
  - Reduced FP32 core (128 core/SM)
    - 32core with warp scheduler, register
  - Reduced FP64 to 1/32 (4 core/SM)
  - FP16 support

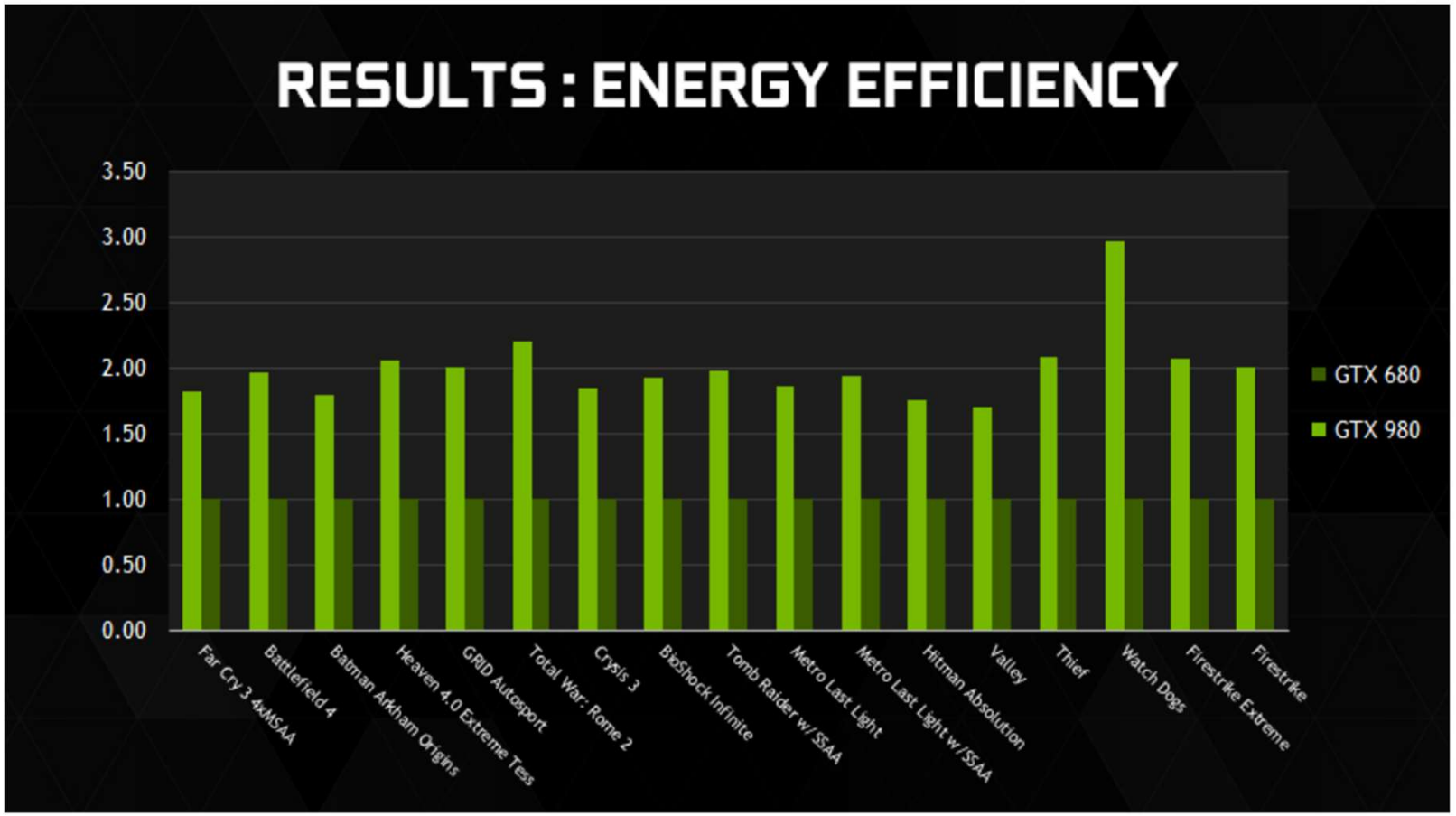
[9]



Maxwell (2014) (Cont'd)

- **Architecture**
  - Better power efficiency

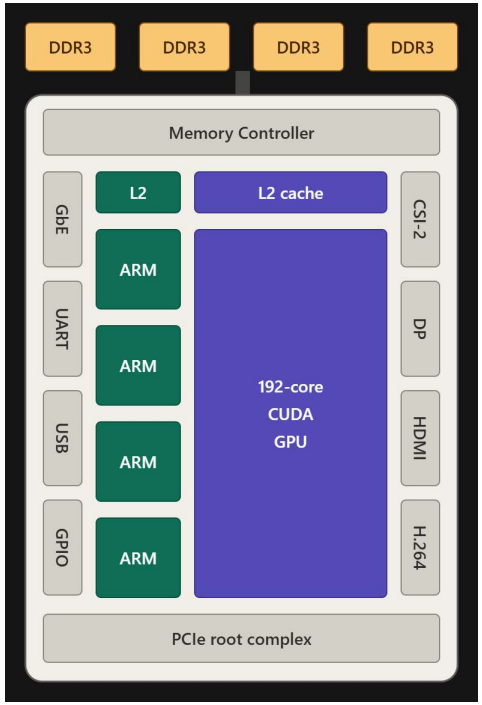
[9]



Maxwell (2014) (Cont'd)

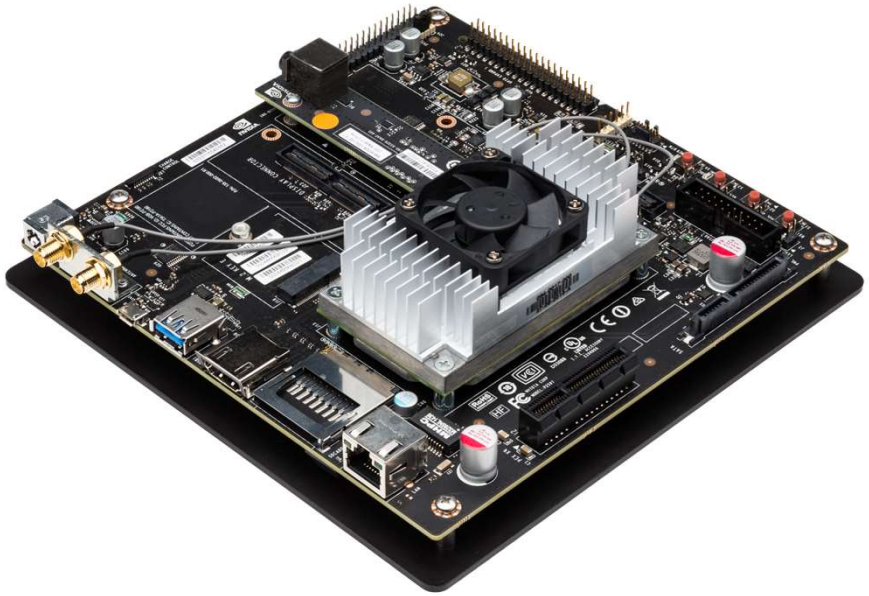
- **Architecture**
  - Embedded GPU Chips
    - Tegra K1 (Kepler), Tegra X1(Maxwell)

[*]



Tegra K1

[34]

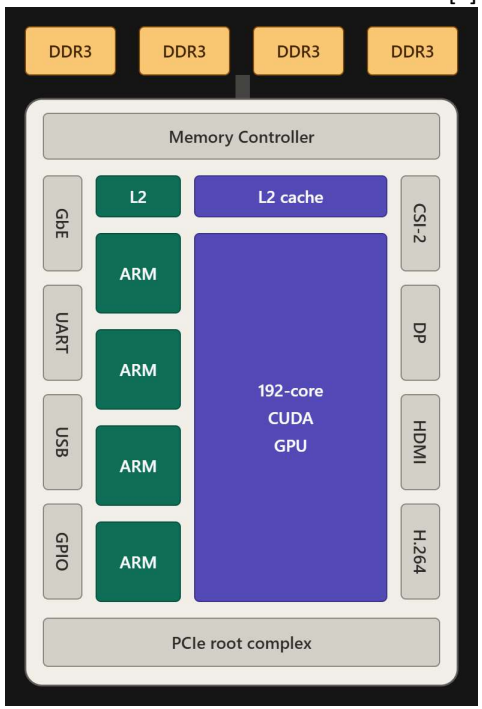


Tegra X1

Maxwell (2014) (Cont'd)

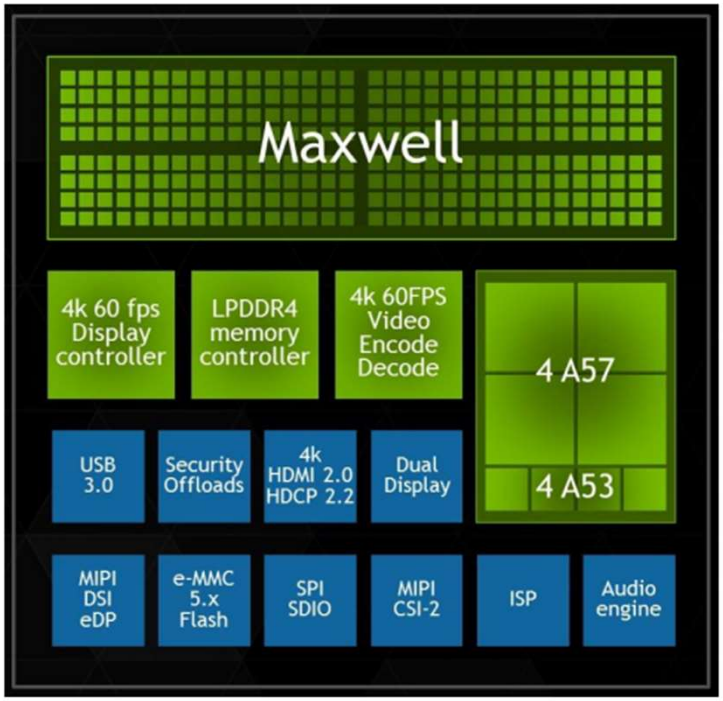
- **Architecture**
  - Embedded GPU Chips
    - Tegra K1 (Kepler), Tegra X1(Maxwell)

[*]



Tegra K1

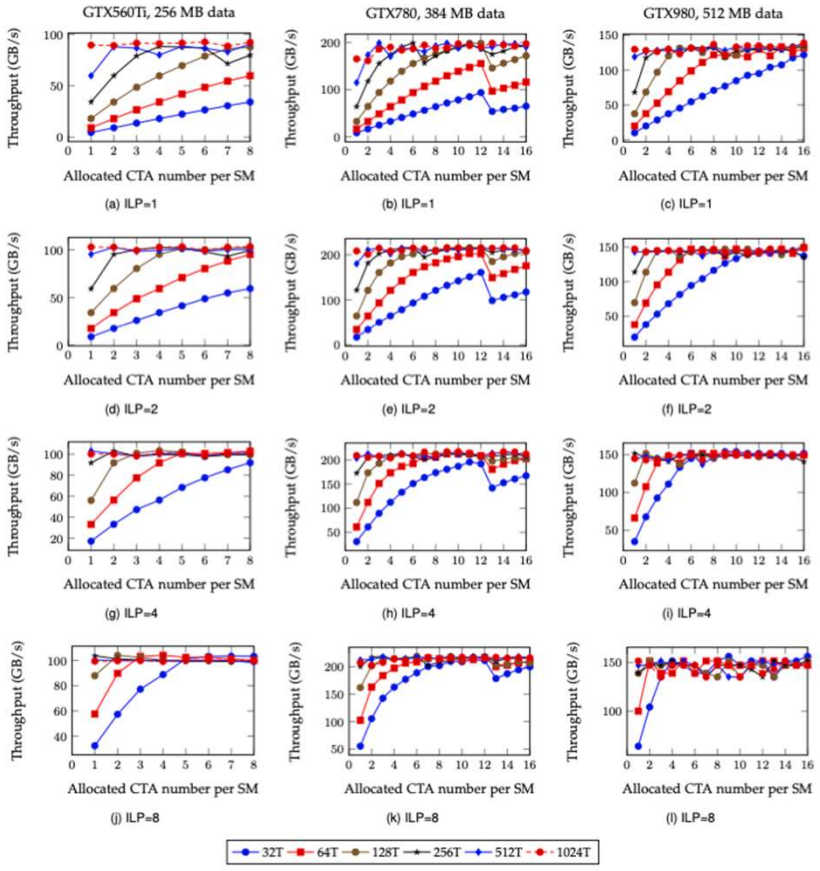
[9]



Tegra X1

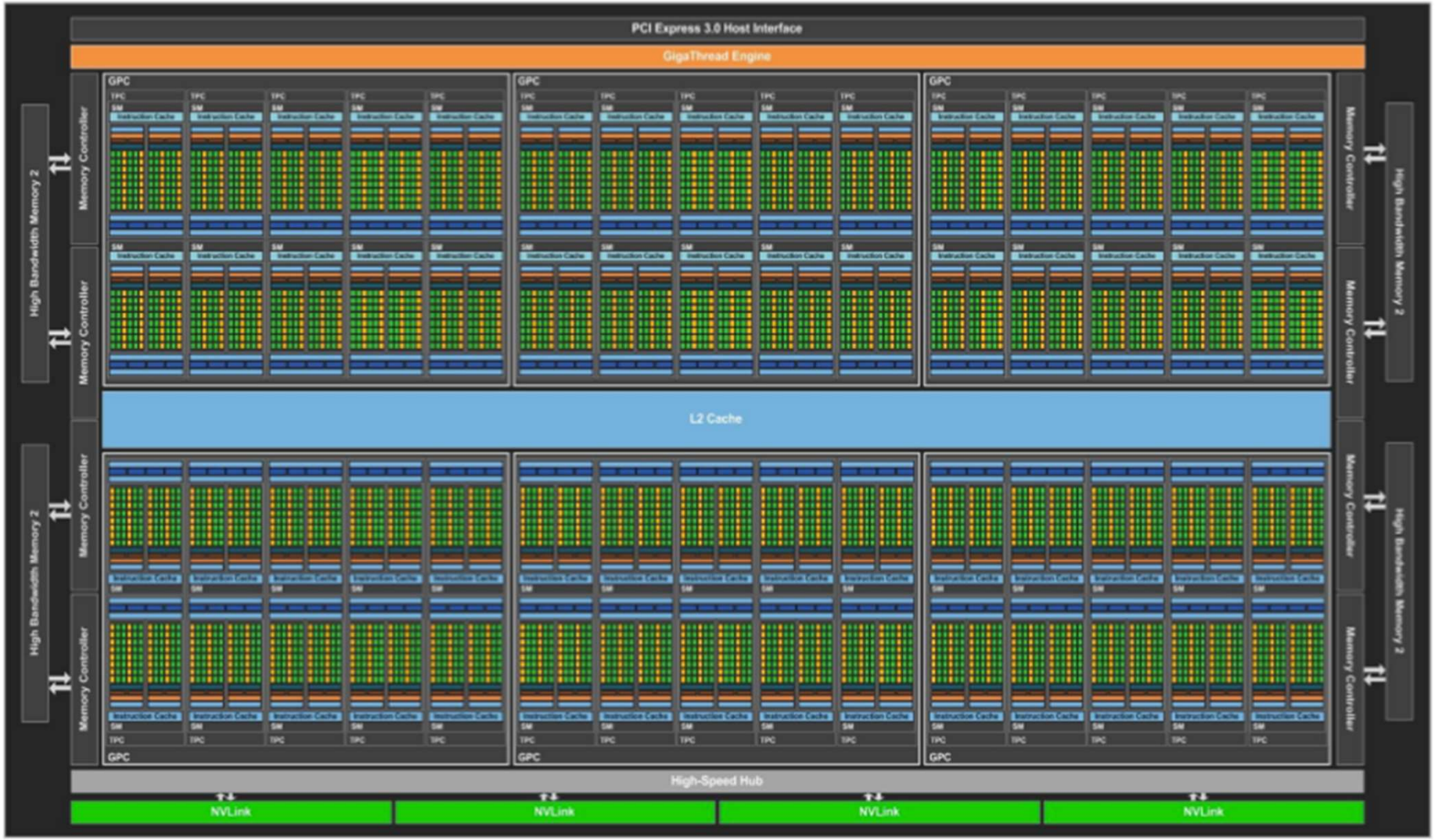
Maxwell (2014) (Cont'd)

- **Relates Studies: Memory Hierarchy analysis**
  - *Dissecting GPU Memory Hierarchy through Microbenchmarking [37]*



Pascal (2016)

[10]



Pascal (2016)

- **Updated specs**
  - TPC as subpart of GPC

Archi.	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core/ SM	Total # of Cores	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Maxwell	GM200	28nm	6	—	24	—	4	128	3,072	4	64	2,048	1,536	32	24KB ×2	96KB	3,072KB
Pascal	GP100	16nm	6	30	60	2	10	64	3,840	2	64	2,048	3,840	32	24KB x2	64KB	4,096KB
Pascal	GP102	16nm	6	30	30	1	5	128	3,840	4	64	2,048	1,920	32	24KB ×2	96KB	3,072KB
Pascal	GP104	16nm	4	20	20	1	5	128	2,560	4	64	2,048	1,280	32	24KB ×2	96KB	2,048KB

Pascal (2016) (Cont'd)

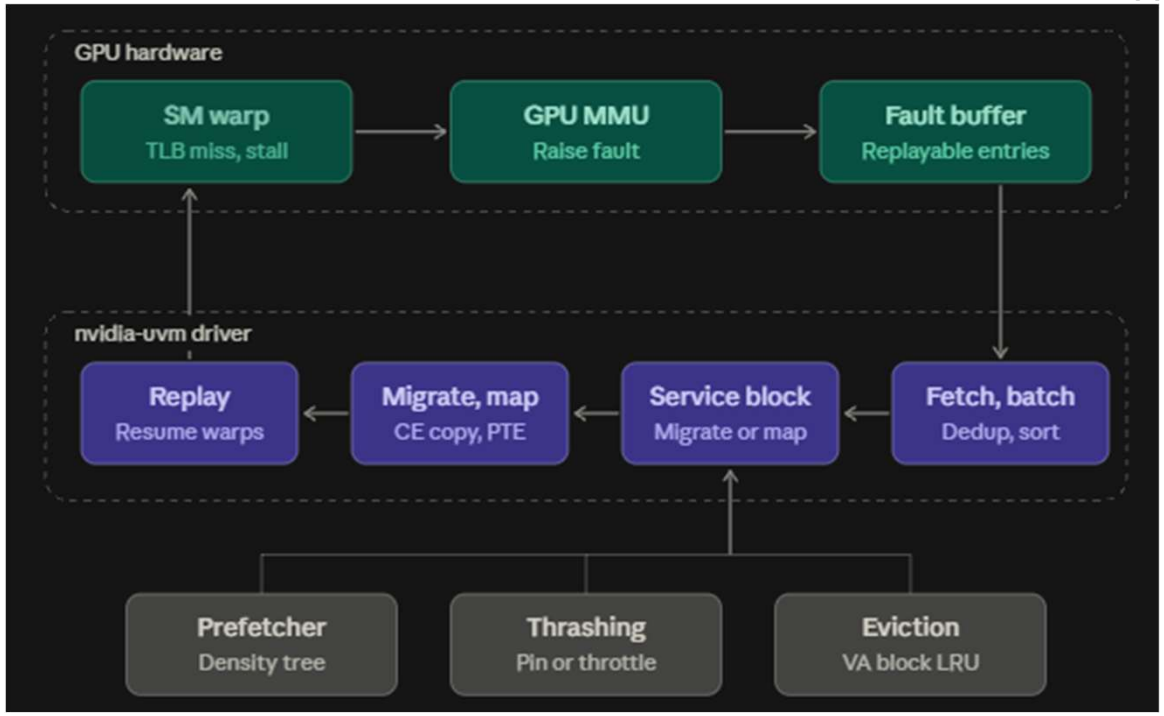
- **SM**
  - 64 FP32 cores
  - 32 FP64 cores (double-precision)
    - 3:1 (Kepler) -> 2:1 ratio
  - FP16 supprot

[10]



Pascal (2016) (Cont'd)

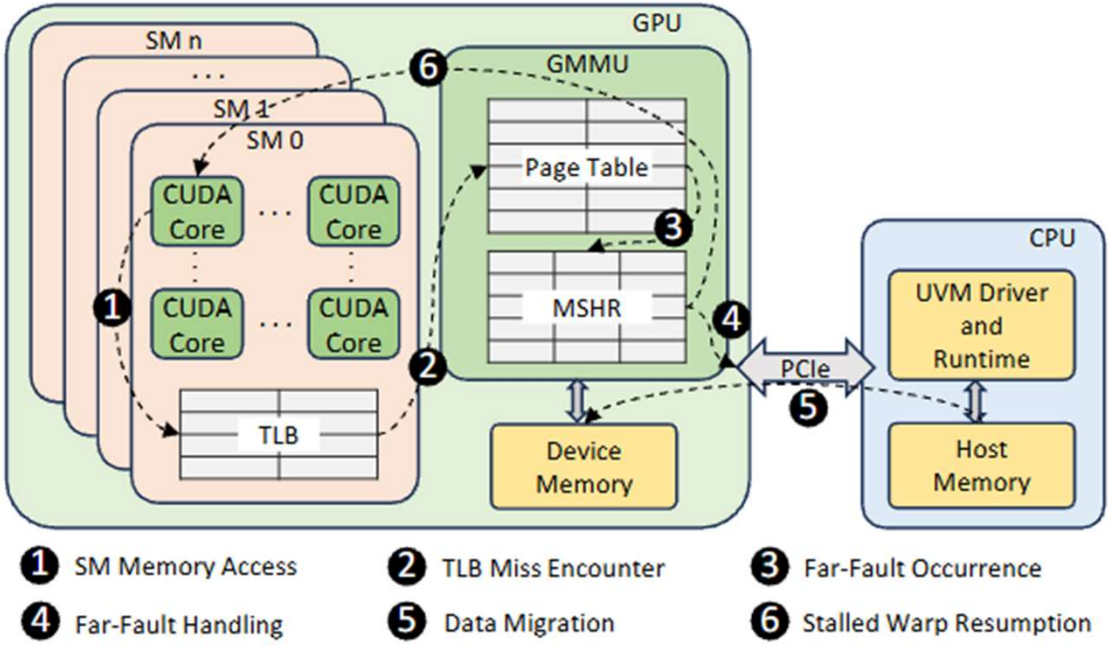
- **Architecture**
  - HBM2 channel
  - Page Migration Engine
    - Support for Unified Memory
    - Enabling Page Fault on GPU, Prefetch, Thrashing, Eviction [*]



Pascal (2016) (Cont'd)

- **Architecture**
  - HBM2 channel
  - Page Migration Engine
    - Support for Unified Memory
    - Enabling Page Fault on GPU, Prefetch, Thrasing, Eviction
    - Huge cost for Page Fault

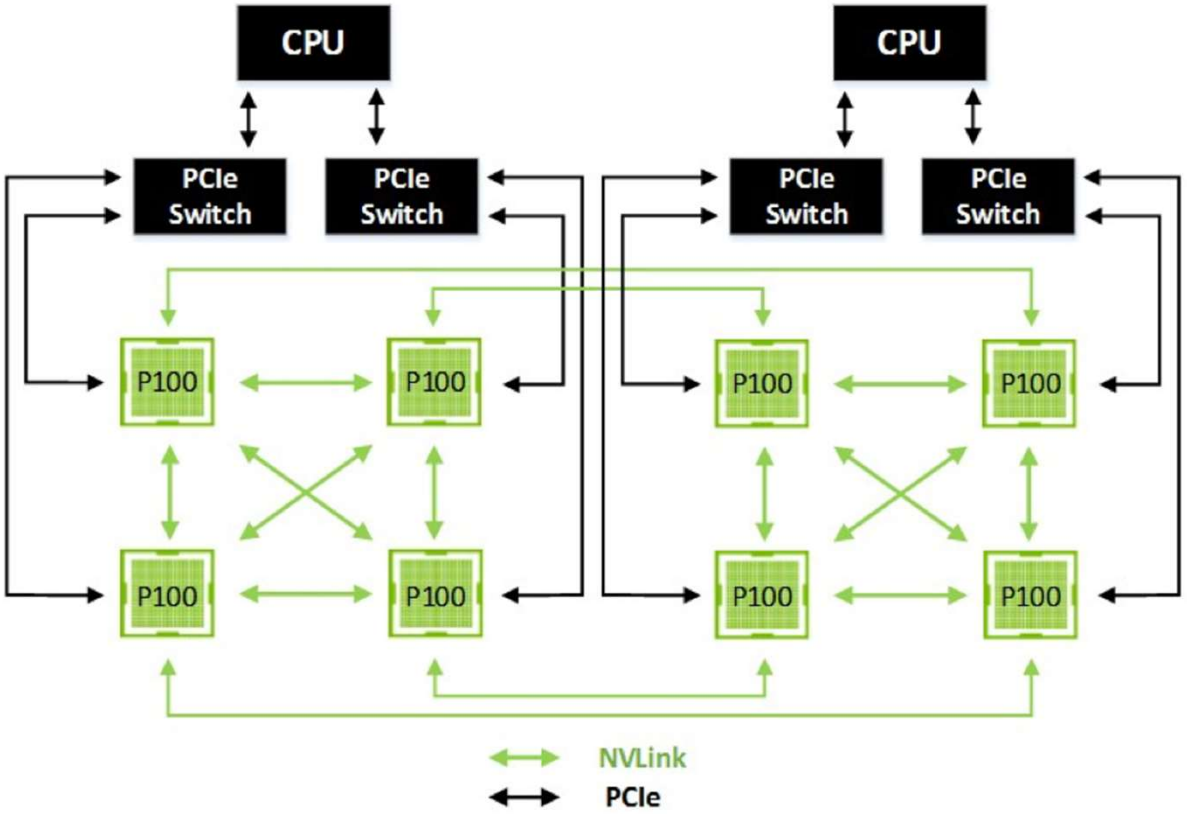
[26]



Pascal (2016) (Cont'd)

- **Architecture**
  - NVLINK

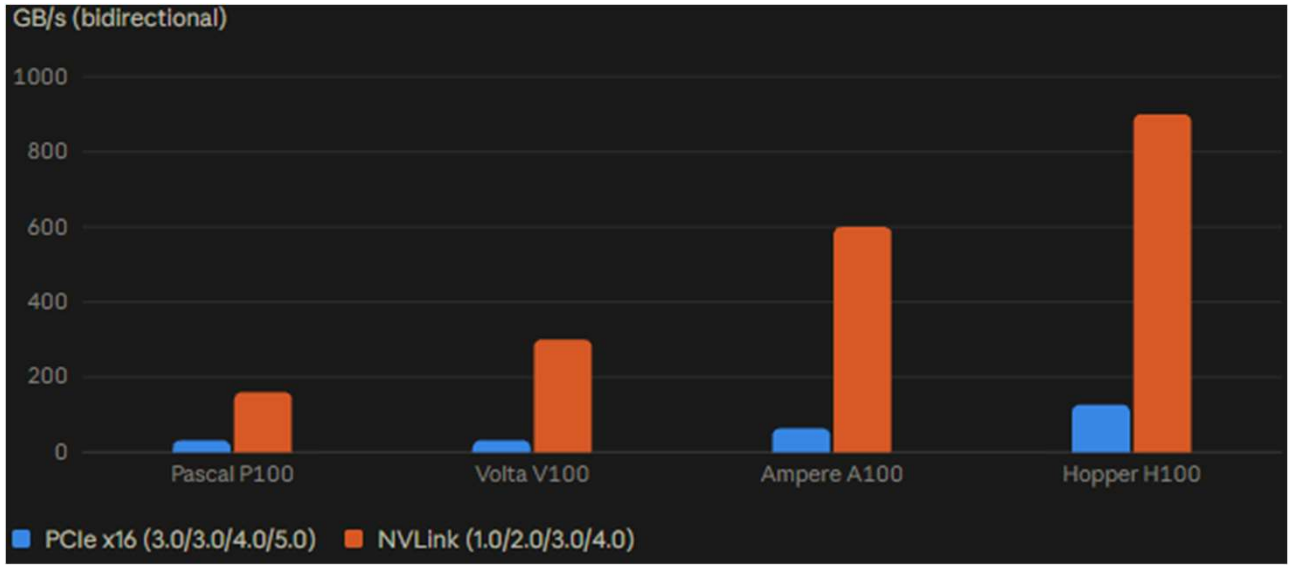
[11]



Pascal (2016) (Cont'd)

- **Architecture**
  - NVLINK

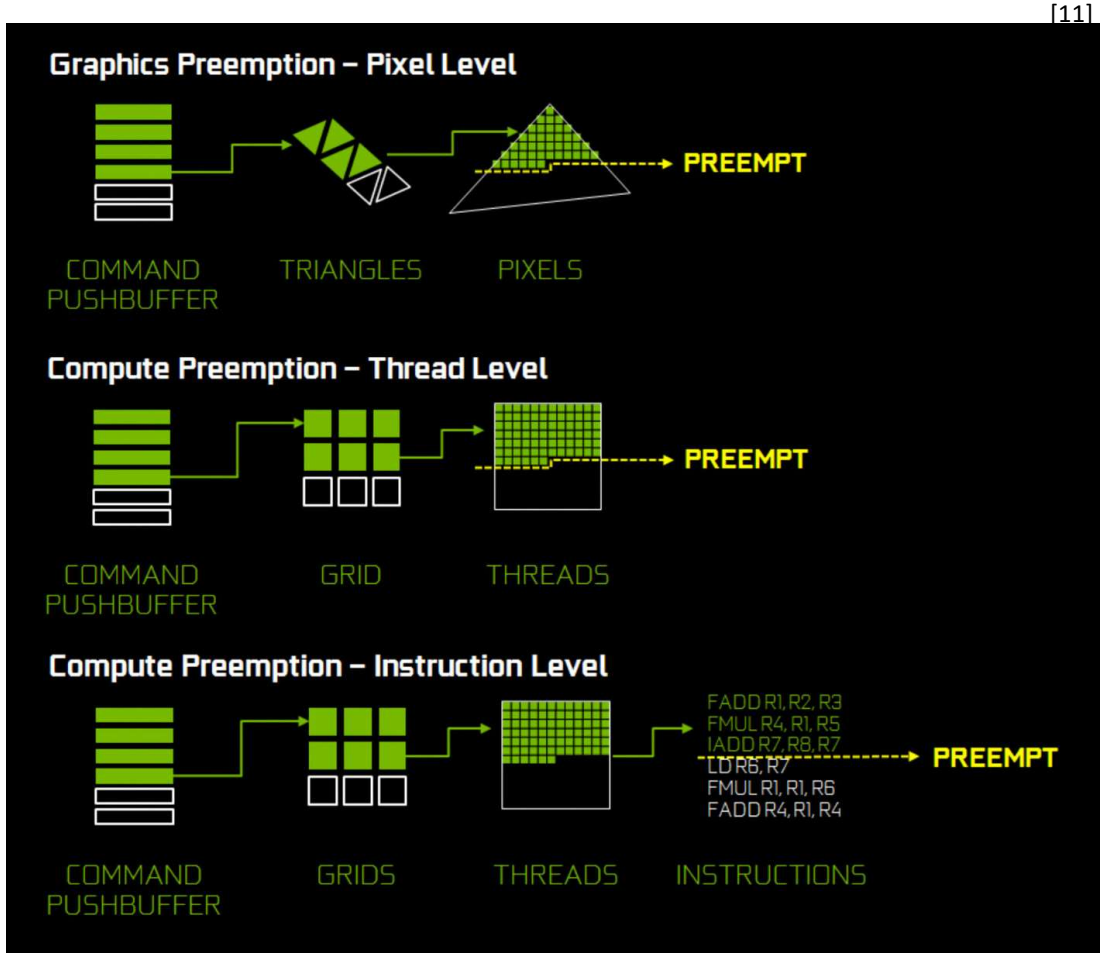
[*]



Pascal (2016) (Cont'd)

- **Architecture**
  - Compute Preemption (Hardware, Instruction level)

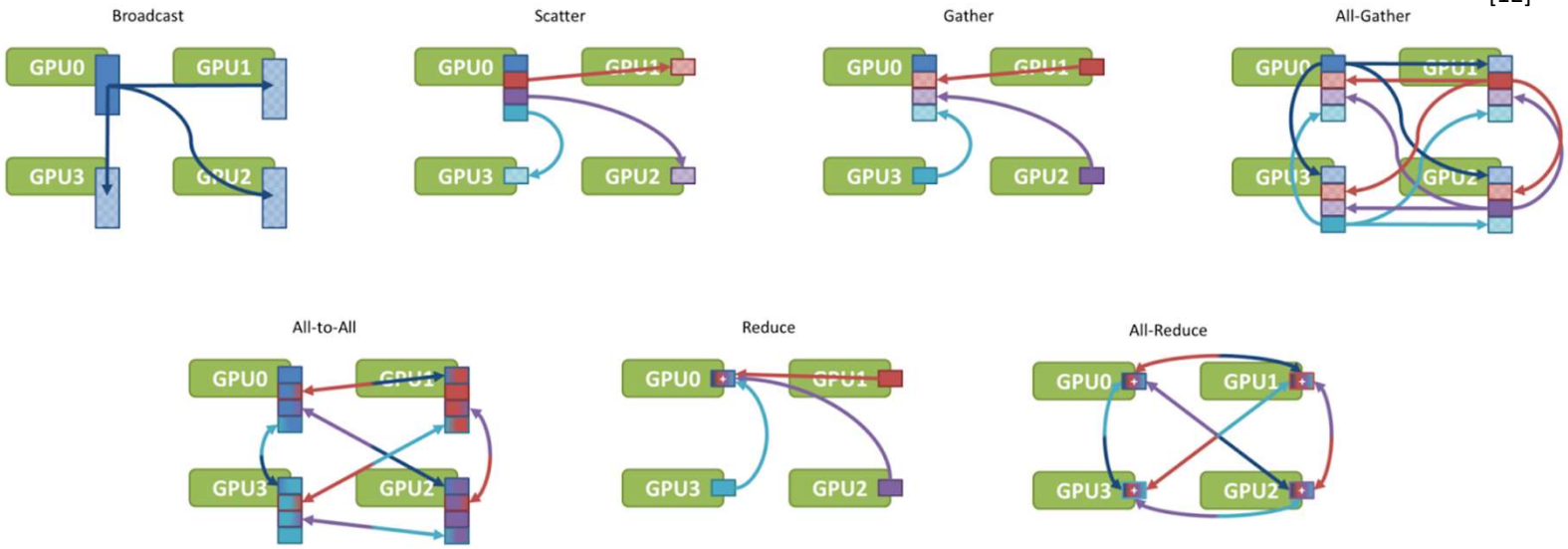
[11]



Pascal (2016) (Cont'd)

- **CUDA**
  - NVIDIA Collective Communication Library (NCCL) 1.0

[12]



Related Studies (Pascal)

- **Instruction-level preemption**
  - FLEP: Enabling Flexible and Efficient Preemption on GPUs [16]

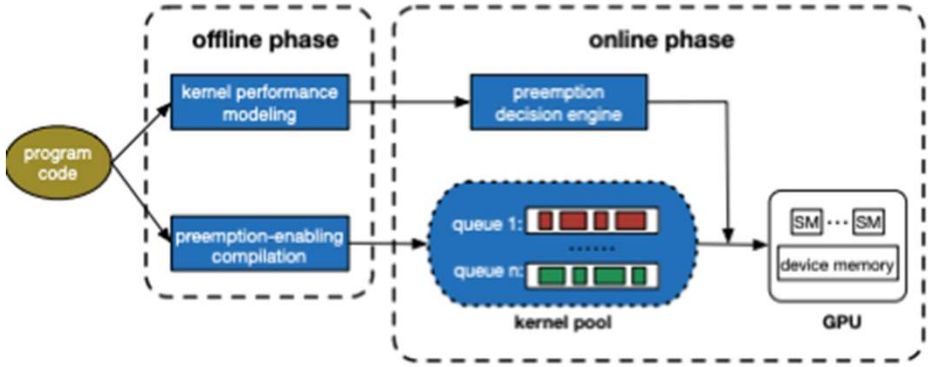


Figure 3: Overview of the FLEP framework.

```
0:  // Executed when a new kernel (Kn) is invoked
1:
2:  if (kernel Kr is running on GPU) {
3:     if (Kr.priority < Kn.priority) {
4:        Preempt(Kr)
5:        Enqueue(Kr.priority)
6:        Schedule_to_GPU(Kn) }
7:     else if (Kr.priority > Kn.priority) {
8:        Enqueue(Kn, Kn.priority) }
9:     else //Kr.priority == Kn.priority {
10:       Schedule_for_queue(Kn.priority)
11:    }
12: } else {
13:    Enqueue(Kn, Kn.priority)
14:    Schedule_for_queue(Kn.priority)
15: }
16:
17: //Executed when one kernel is finished
18:
19: //find the non-empty queue of highest priority hp
20: Schedule_for_queue(hp)
21:
22: //Definition of the key scheduling function
23:
24: schedule_for_queue(p){
25:
26:    //find the kernel Ks in the queue of priority level p
27:    //with smallest Tr
28:    if (kernel Kr is running on GPU){
29:       if(Kr.Tr > Ks.Tr + preemption_overhead) {
30:          Preempt(Kr)
31:          Schedule_to_GPU(Ks) }
32:    } else {
33:       Schedule_to_GPU(Ks) }
34: }
```

Related Studies (Pascal) (Cont'd)

- **Instruction-level preemption**
  - FLEP: Enabling Flexible and Efficient Preemption on GPUs [16]

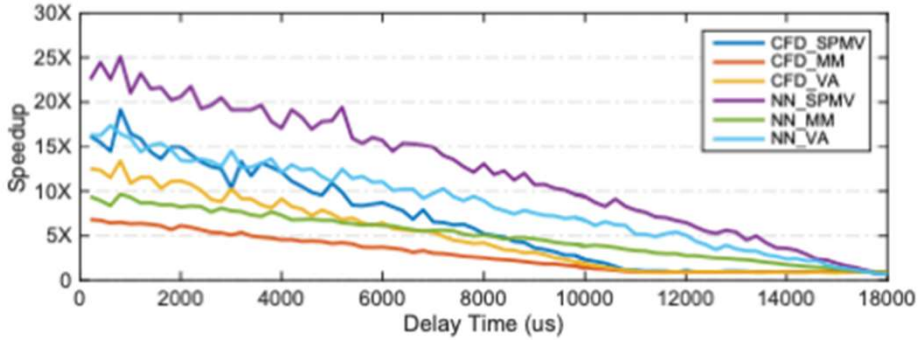
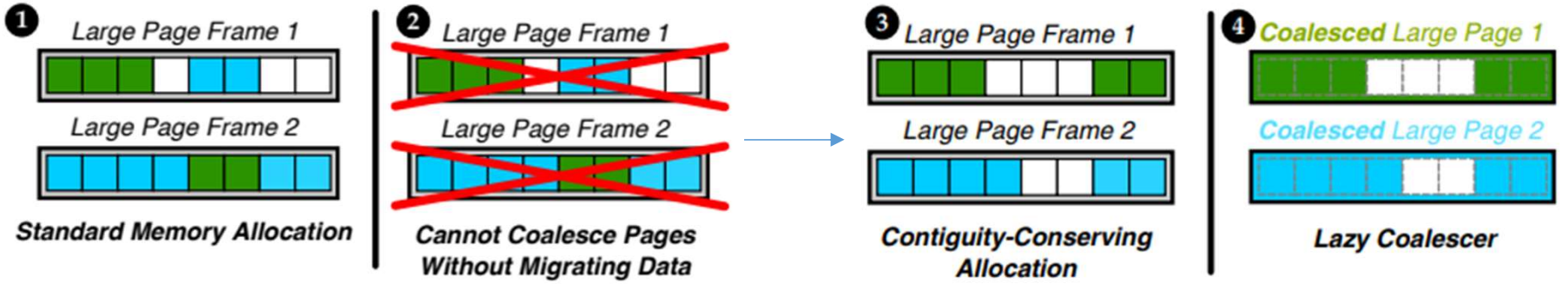
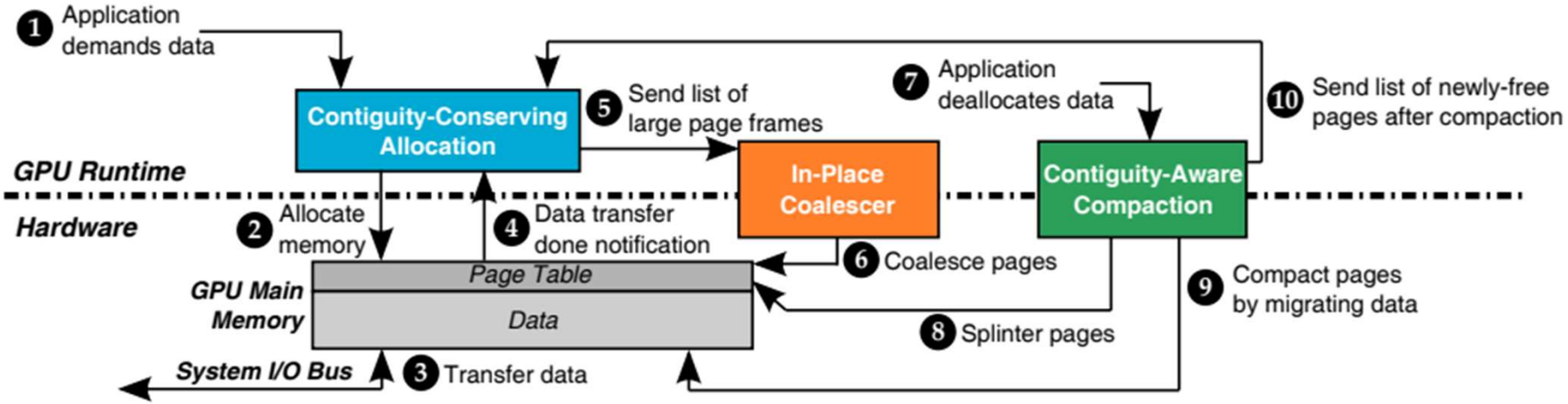


Figure 9: Performance improvement for high-priority kernels over their execution in MPS-based co-runs when delay is introduced between kernel invocations.

Related Studies (Pascal) (Cont'd)

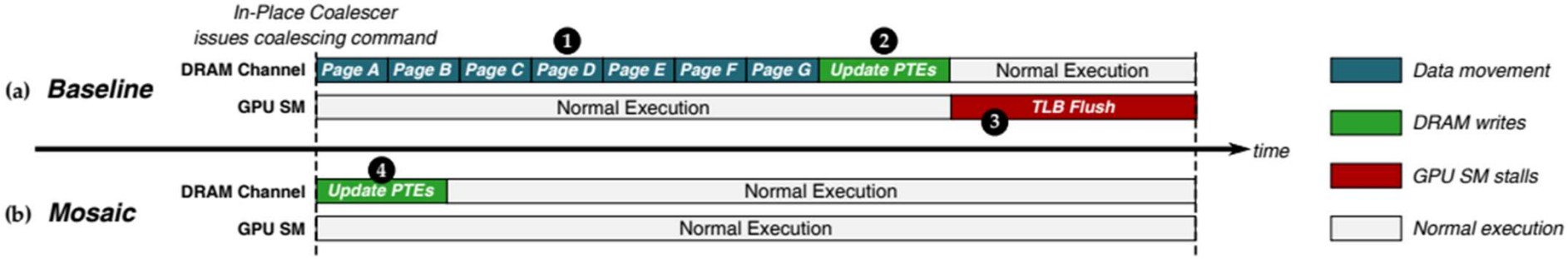
- **Memory Management**
  - Mosaic [38]

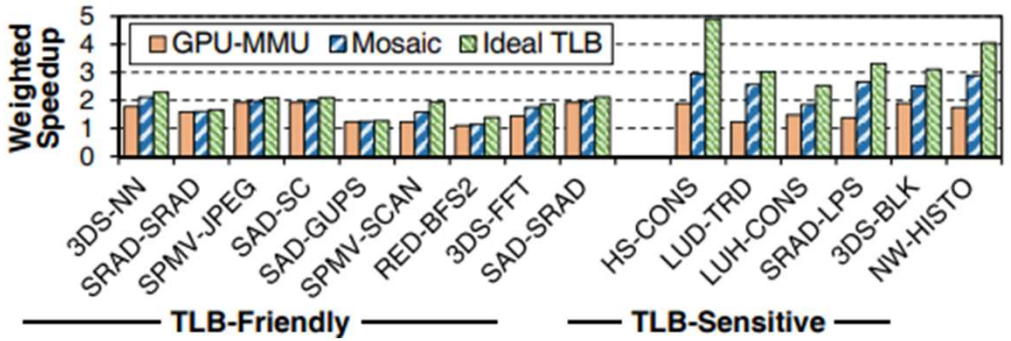




Related Studies (Pascal) (Cont'd)

- **Memory Management**
  - Mosaic [38]

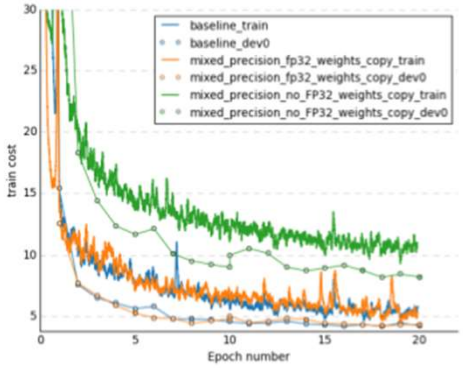




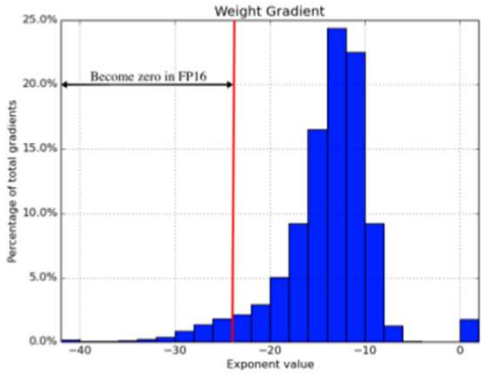
Related Studies (Pascal) (Cont'd)

- **Deep Learning**
  - Mixed precision Training [17]





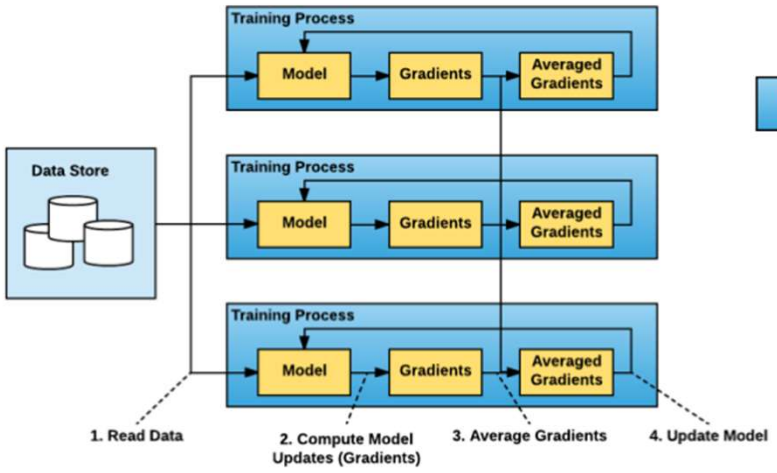
(a) Training and validation (dev0) curves for Mandarin speech recognition model

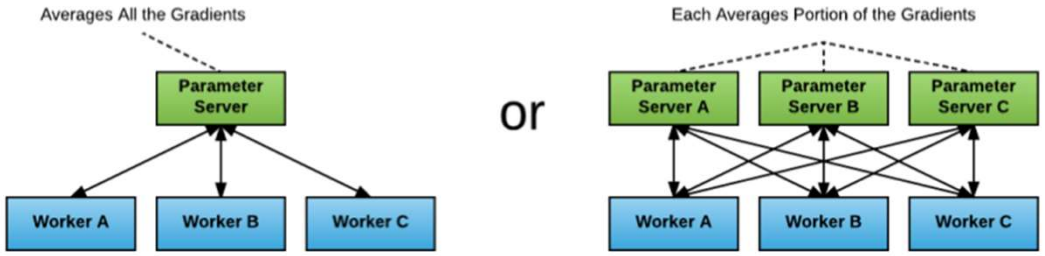


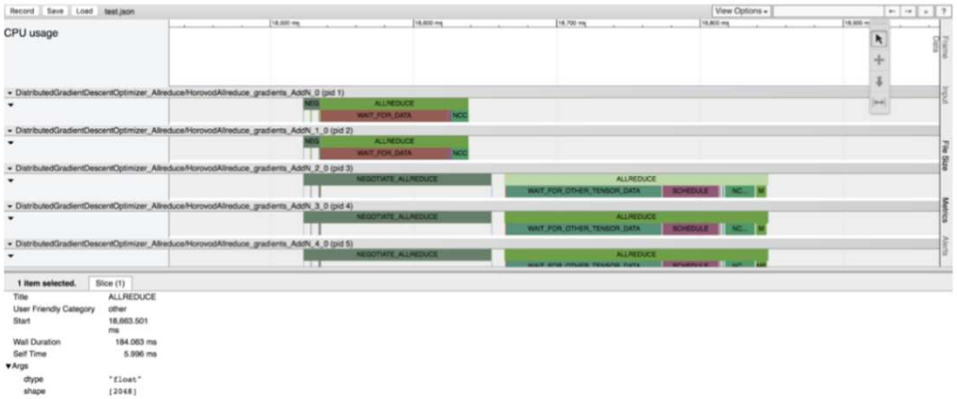
(b) Gradient histogram for Mandarin training run

Related Studies (Pascal) (Cont'd)

- **Deep Learning**
  - Horovod [18]

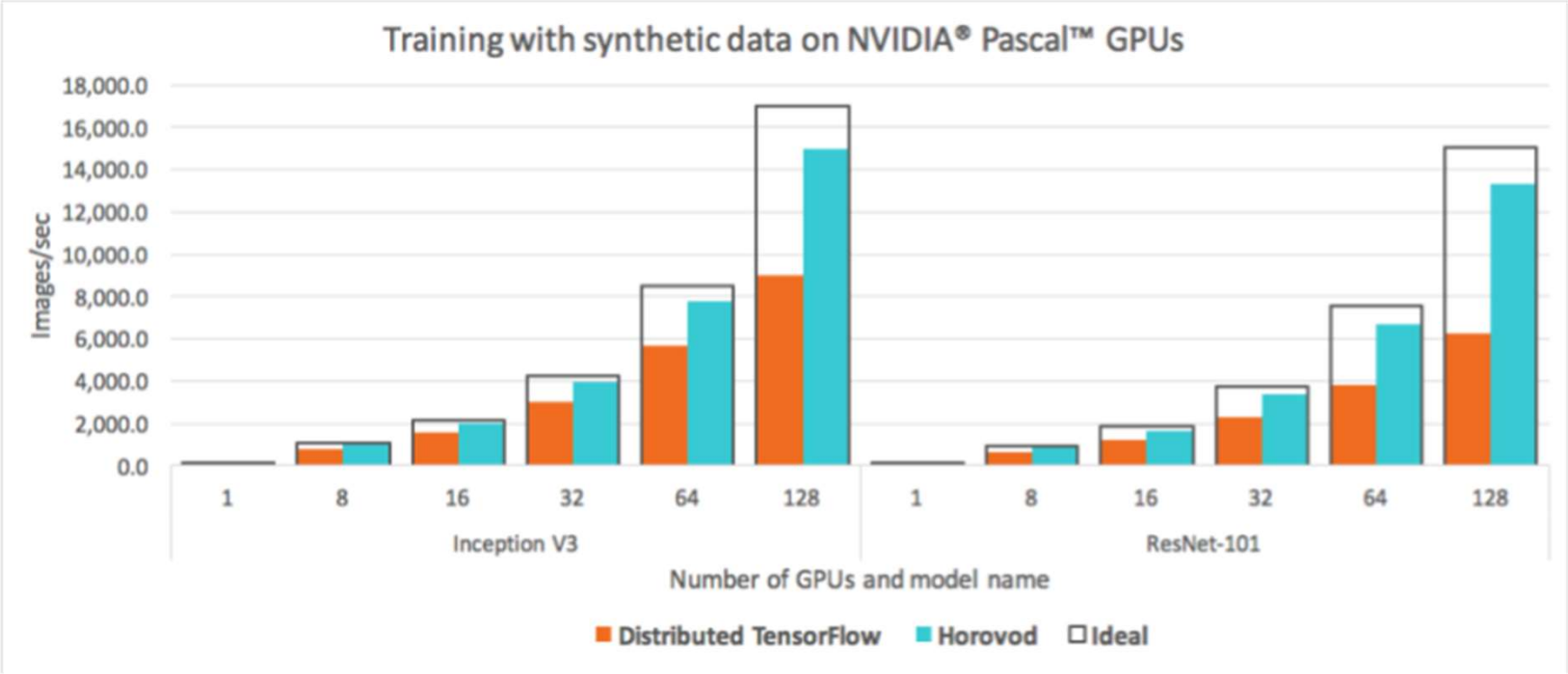






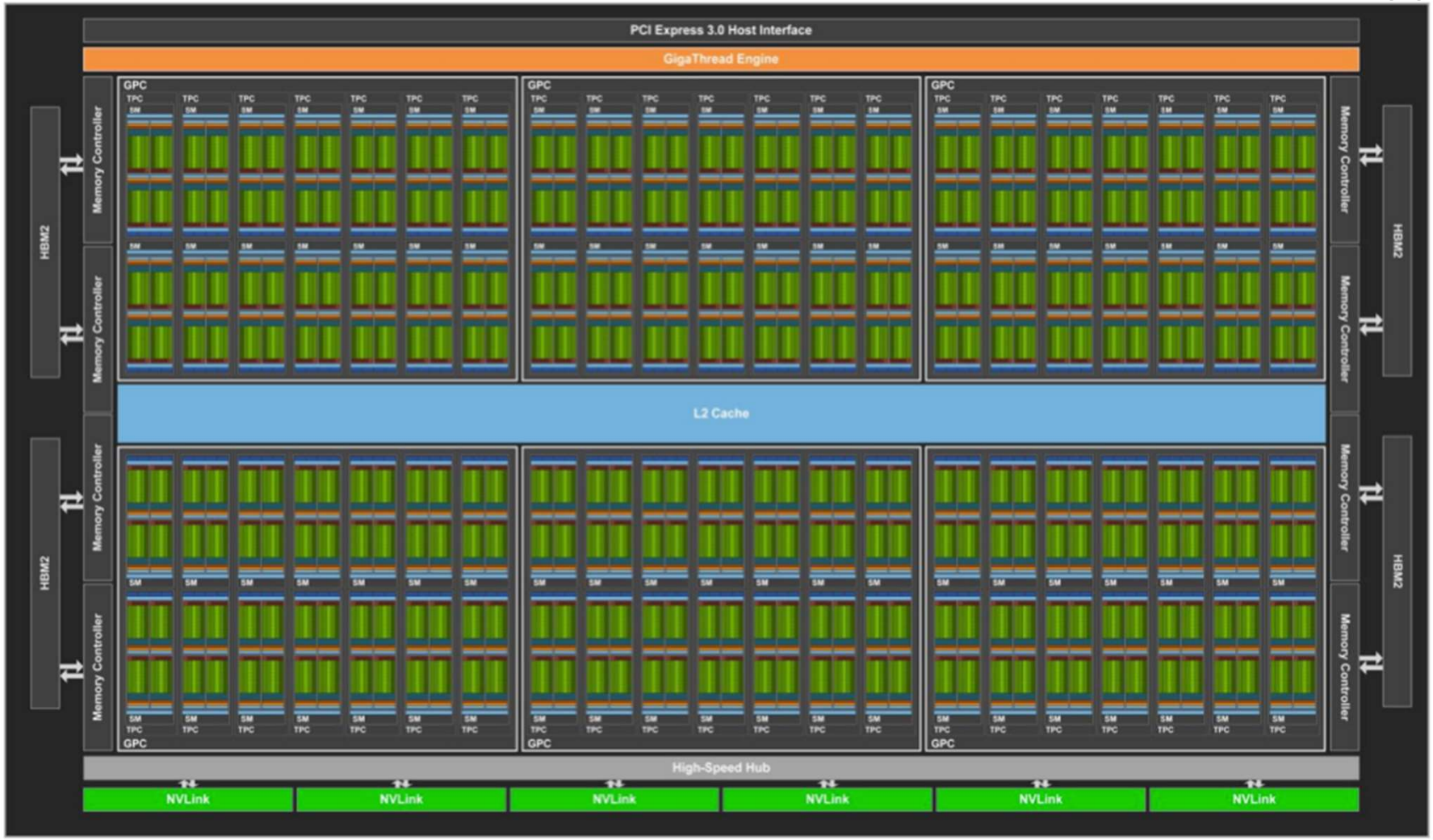
Related Studies (Pascal) (Cont'd)

- **Deep Learning**
  - Horovod [18]



Volta (2017)

[13]



Volta (2017)

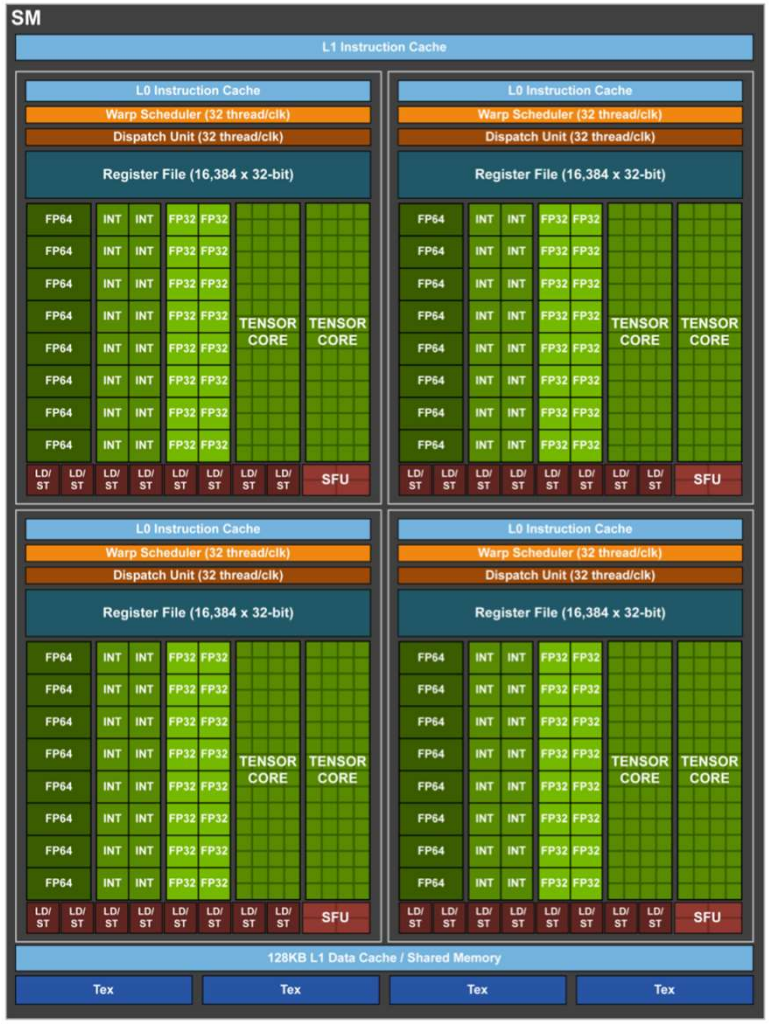
- **Updated specs**
  - L1, shared memory reunified
  - Tensor cores

Archi.s	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core /SM	Total # of Cores	Tensor core / SM	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Pascal	GP104	16nm	4	20	20	—	1	5	128	2,560	4	64	2,048	1,280	32	24KB ×2	96KB	2,048KB
Volta	GV100	12nm FFN	6	42	84	2	2	14	64	5,376	4	64	2,048	5,376	32	128KB (Shared memory 96KB Max)		6,144KB

Volta (2017) (Cont'd)

- **SM**
  - 64 FP32 cores
  - 32 FP64 cores
  - 64 INT32 cores
  - 8 Tensor cores

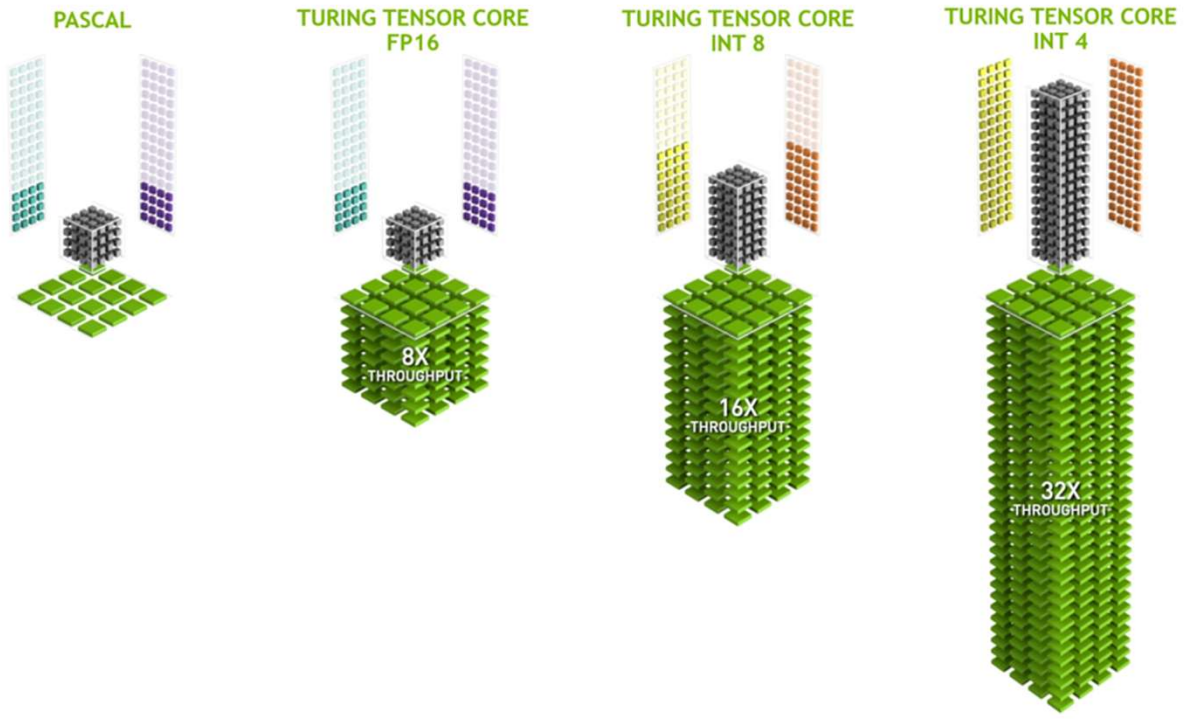
[13]



Volta (2017) (Cont'd)

- **Architecture**
  - 1st Gen Tensor Core
    - Extreme performance for AI & HPC

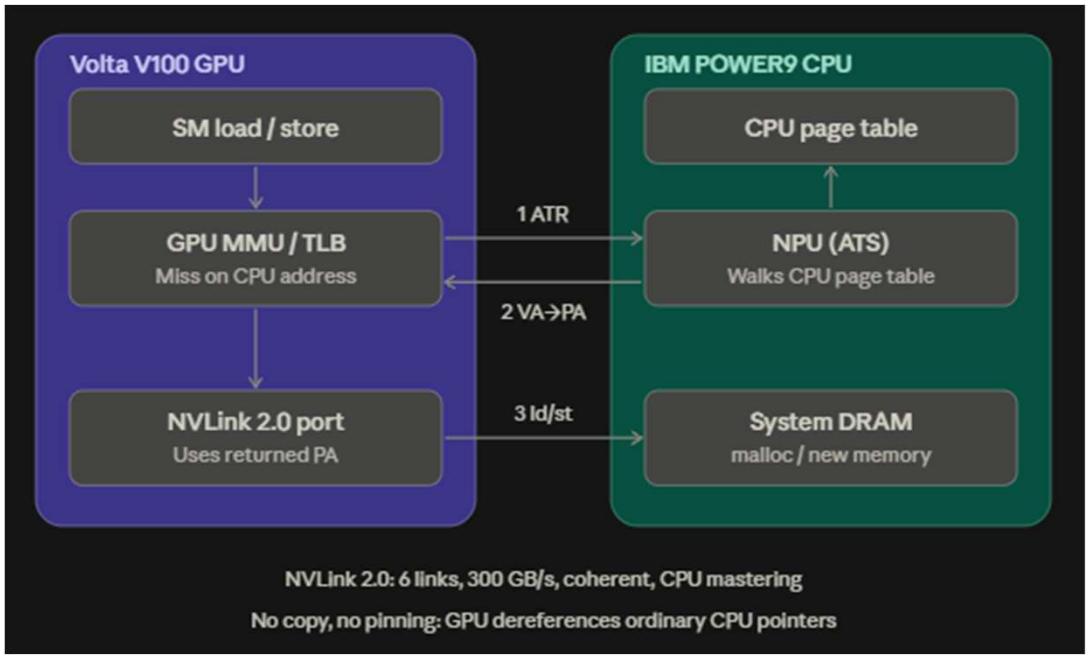
[13]



Volta (2017) (Cont'd)

- **Architecture**
  - Grid-wide sync
    - Synchronization between Thread Blocks
  - Address Translation Service
    - Under NVLink

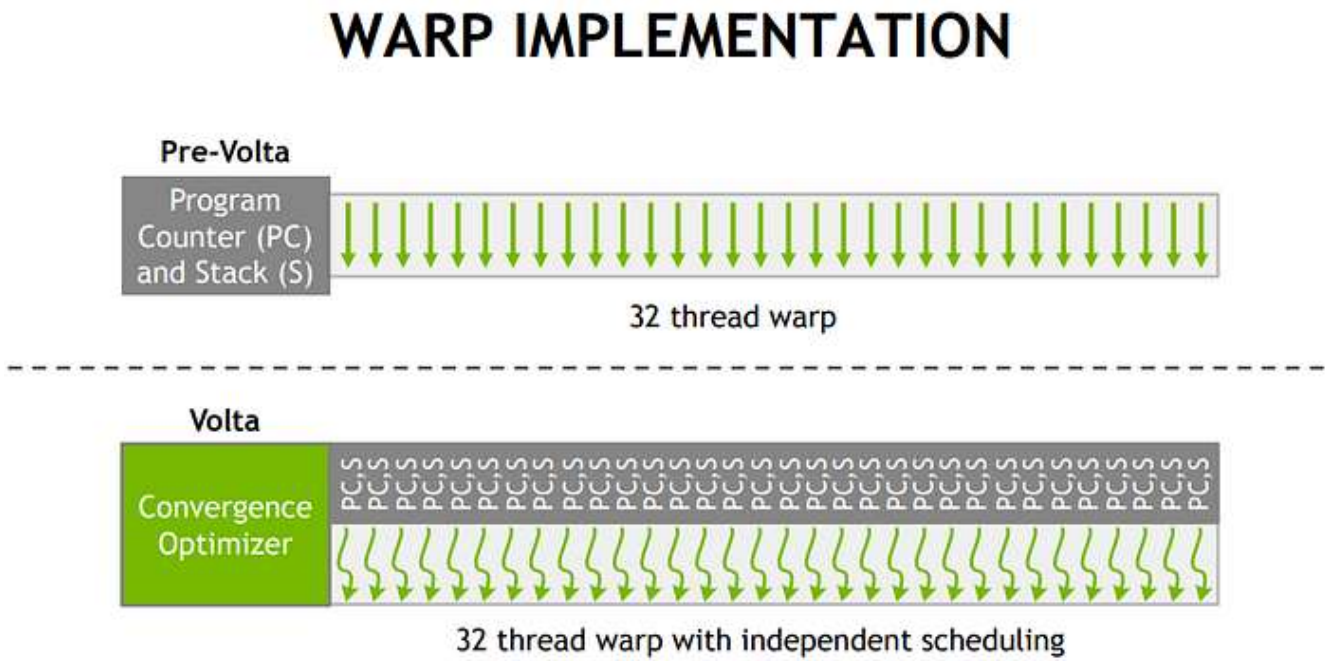
[*]



Volta (2017) (Cont'd)

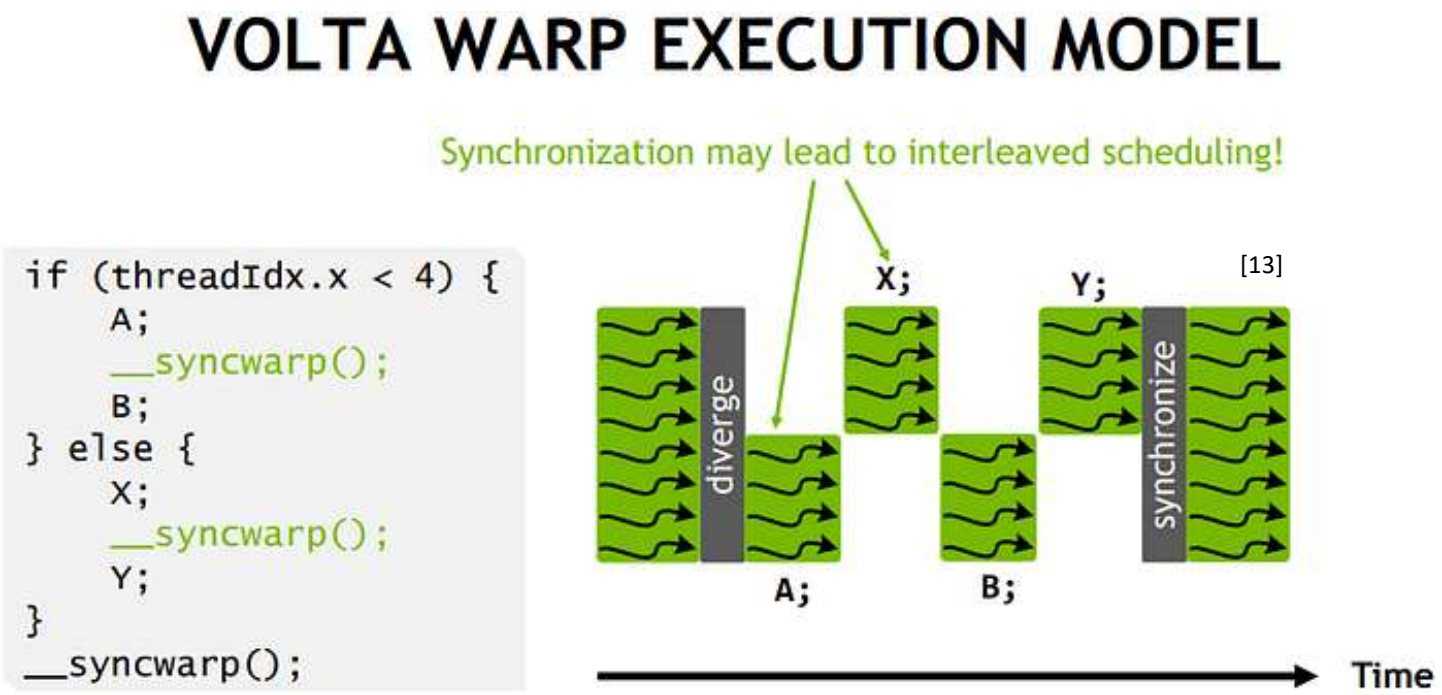
- **Architecture**
  - Independent Thread Scheduling

[13]



Volta (2017) (Cont'd)

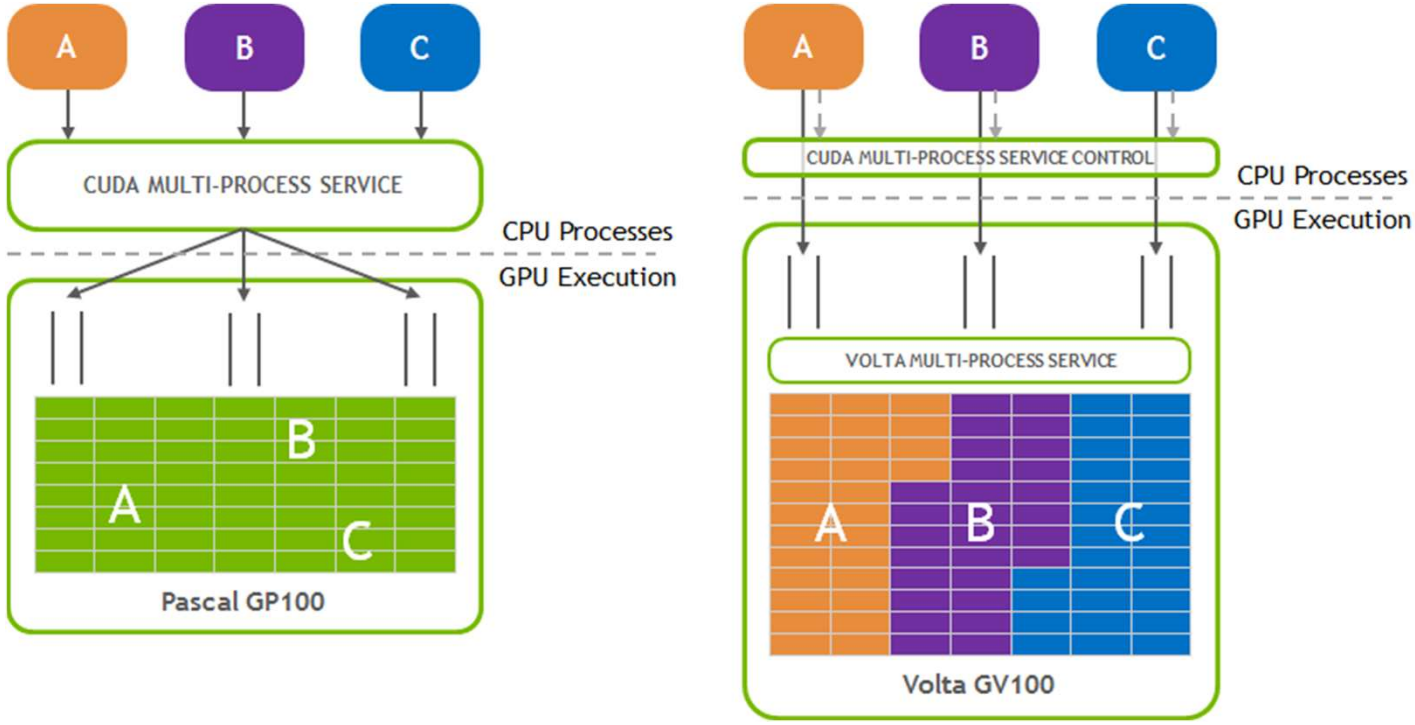
- **Architecture**
  - Independent Thread Scheduling



Volta (2017) (Cont'd)

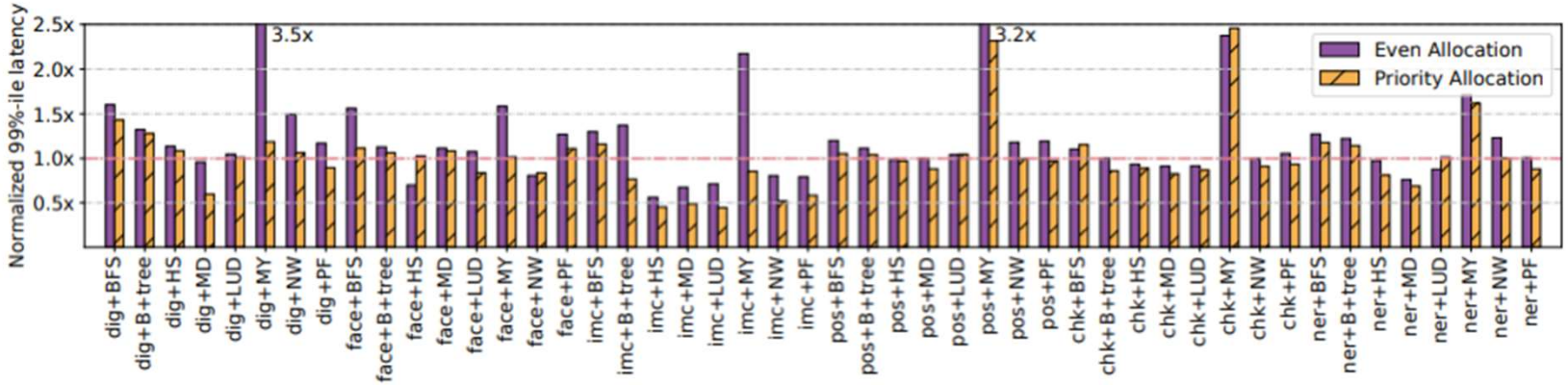
- **CUDA**
  - Volta MPS

[13]



Related Studies (Volta)

- **Volta MPS**
  - Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters [19]



Related Studies (Volta) (Cont'd)

- **Volta MPS**
  - Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters [19]

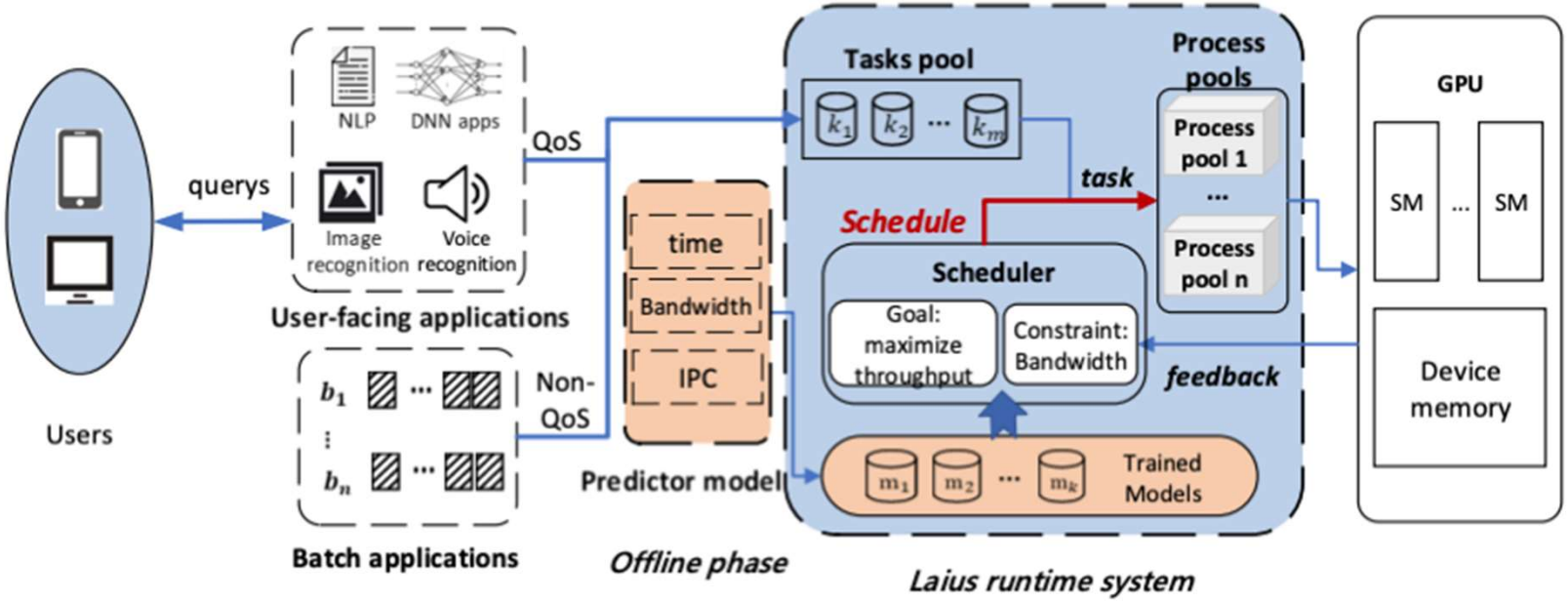


Figure 4: Design overview of Laius.

Related Studies (Volta) (Cont'd)

- **Volta MPS**
  - Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters [19]

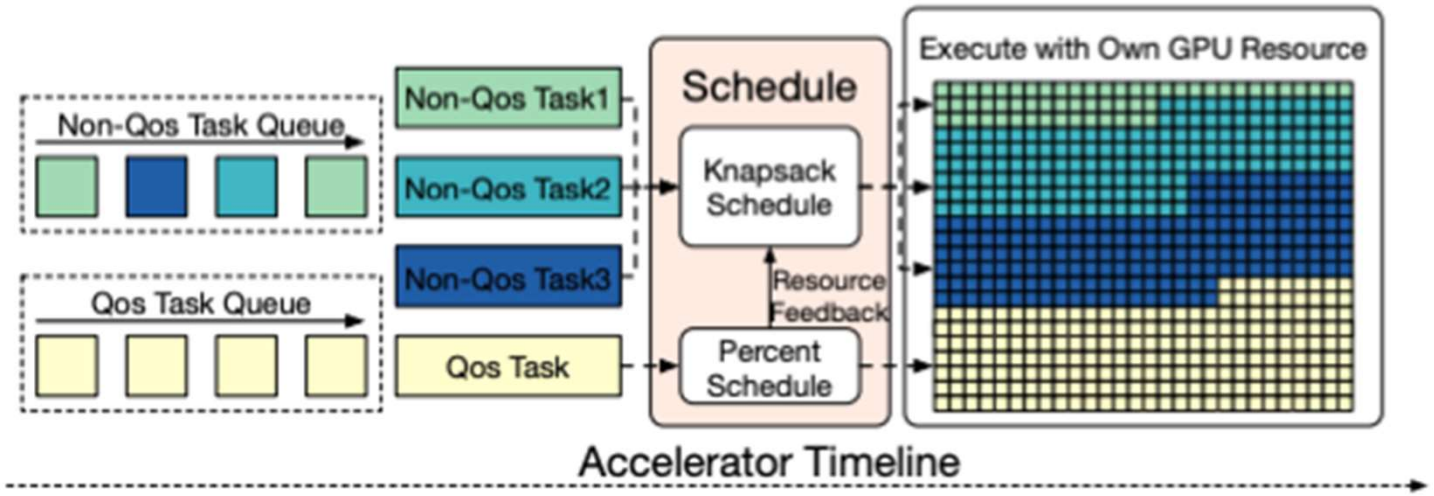
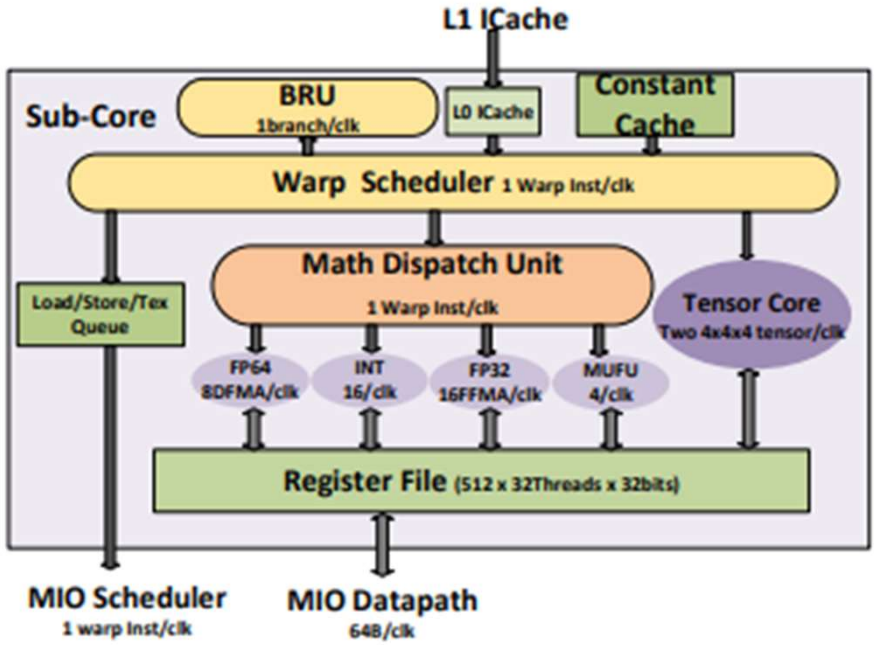


Figure 9: Contention-aware resource allocation in Laius.

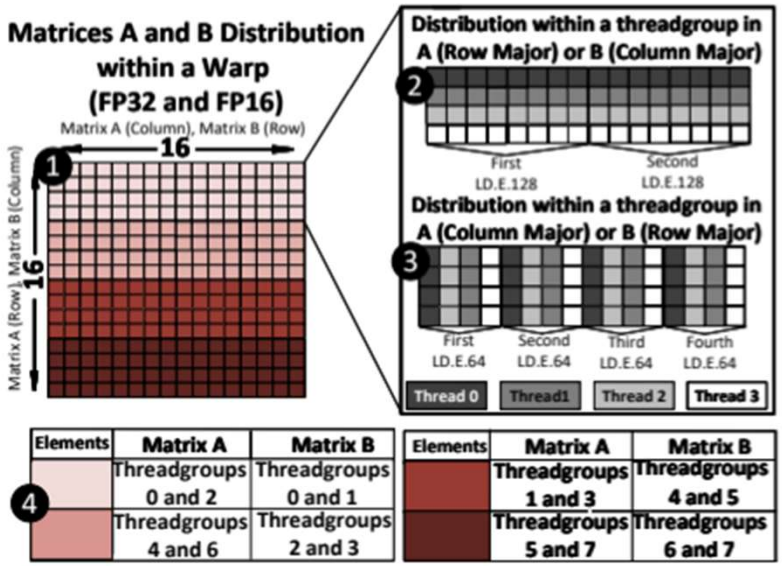
Related Studies (Volta) (Cont'd)

- **Tensor Core**
  - Modeling Deep Learning Accelerator Enabled GPUs [20]

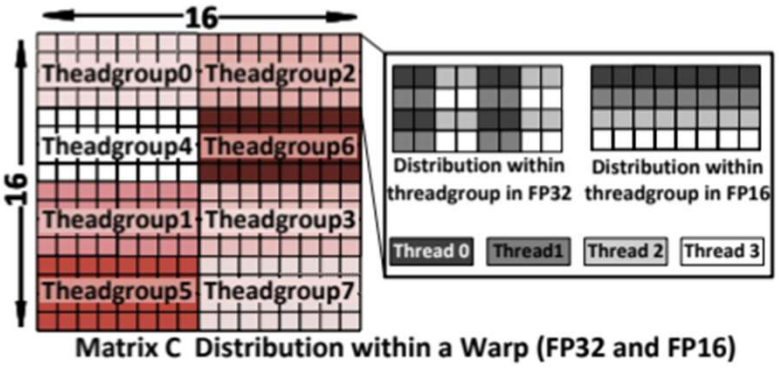


Related Studies (Volta) (Cont'd)

- **Tensor Core**
  - Modeling Deep Learning Accelerator Enabled GPUs [20]



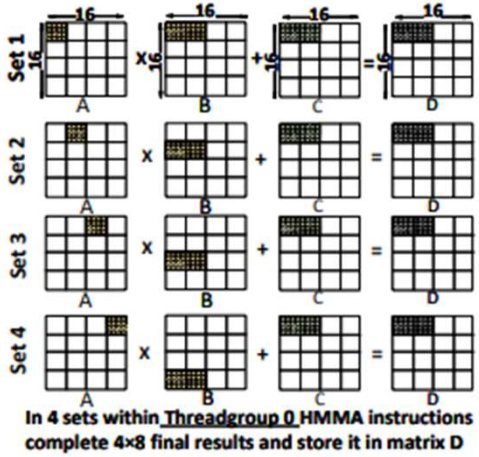
(a) Operand matrices A and B.



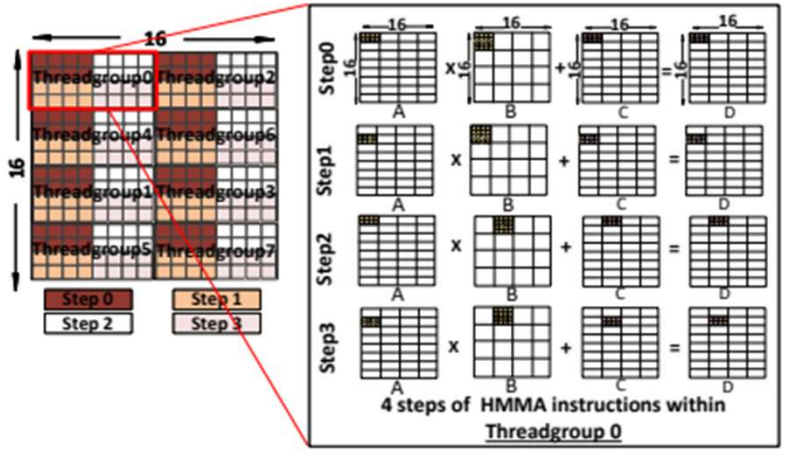
(b) Operand matrix C.

Related Studies (Volta) (Cont'd)

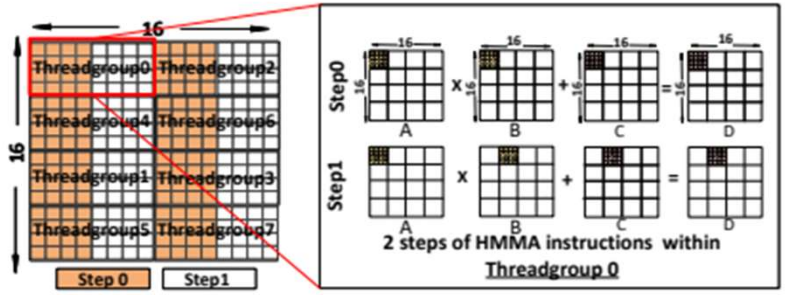
- **Tensor Core**
  - Modeling Deep Learning Accelerator Enabled GPUs [20]



(a) Elements accessed in each "Set"



(b) Elements accessed in each "Step" (mixed-precision mode).



(c) Elements accessed in each "Step" (FP16 mode).

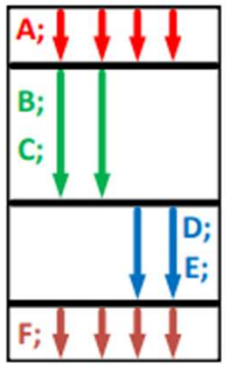
Related Studies (Volta) (Cont'd)

- **Independent Thread Scheduling**
  - Control Flow Management in Modern GPUs [40]

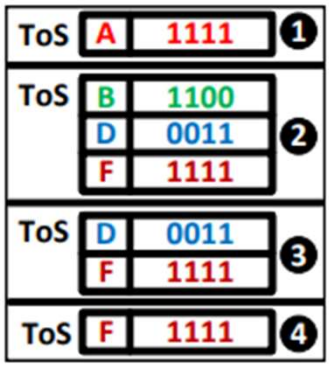
Pre-Volta

```
1: A;
2: if (tid % 4 < 2) {
3:     B;
4:     C;
5: } else {
6:     D;
7:     E;
8: }
9: F;
```

(a)



(b)

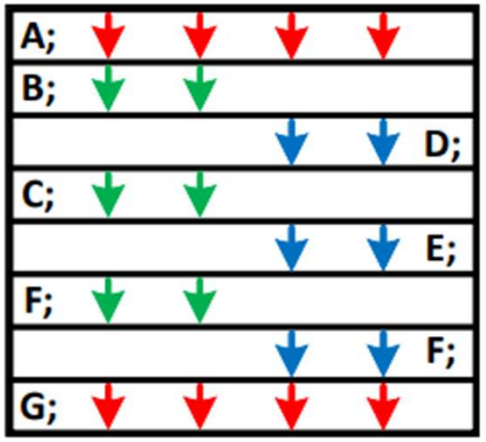


(c)

Volta

```
1: A;
2: if (tid % 4 < 2) {
3:     B;
4:     C;
5: } else {
6:     D;
7:     E;
8: }
9: F;
10: __syncwarp();
11: G;
```

(a)

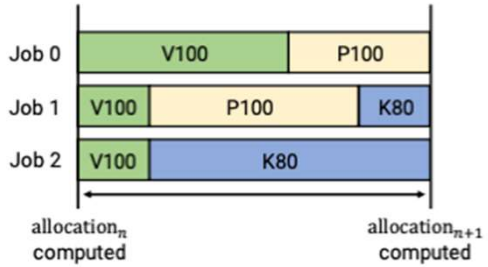


(b)

Related Studies (Volta) (Cont'd)

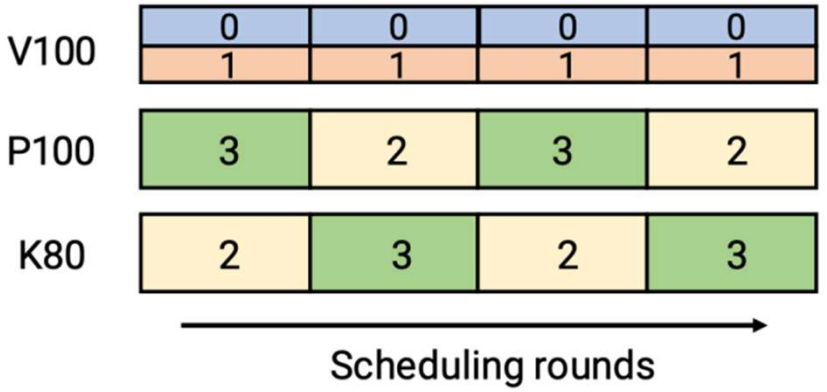
- **Multi tenancy Heterogeneous GPU sharing**
  - Gavel: Heterogeneity-Aware Cluster Scheduling [21]

$$X^{\text{example}} = \begin{array}{c} \begin{matrix} V100 & P100 & K80 \end{matrix} \\ \begin{pmatrix} 0.6 & 0.4 & 0.0 \\ 0.2 & 0.6 & 0.2 \\ 0.2 & 0.0 & 0.8 \end{pmatrix} \end{array} \begin{matrix} \text{job 0} \\ \text{job 1} \\ \text{job 2} \end{matrix}$$



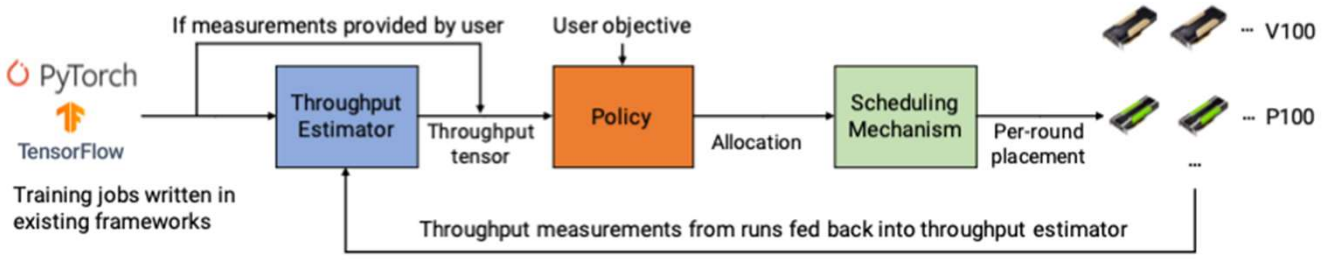
V100 | P100 | K80

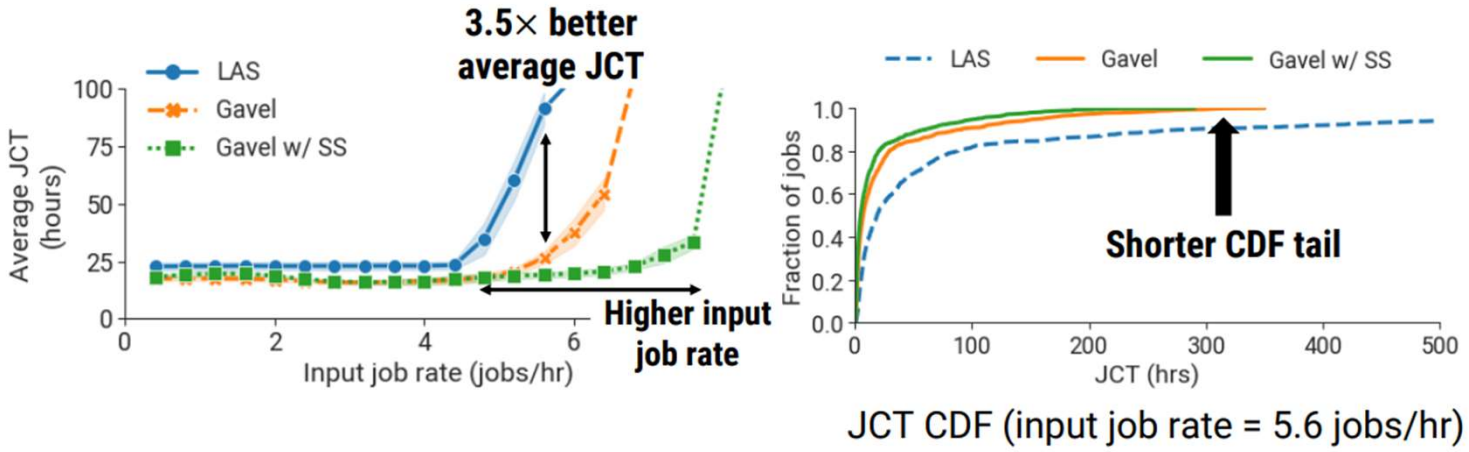
$$X^{\text{het.+SS}} = \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 0.5 & 0.5 \\ 0.0 & 0.5 & 0.5 \end{pmatrix} \begin{matrix} \text{jobs 0+1} \\ \text{job 2} \\ \text{job 3} \end{matrix}$$



Related Studies (Volta) (Cont'd)

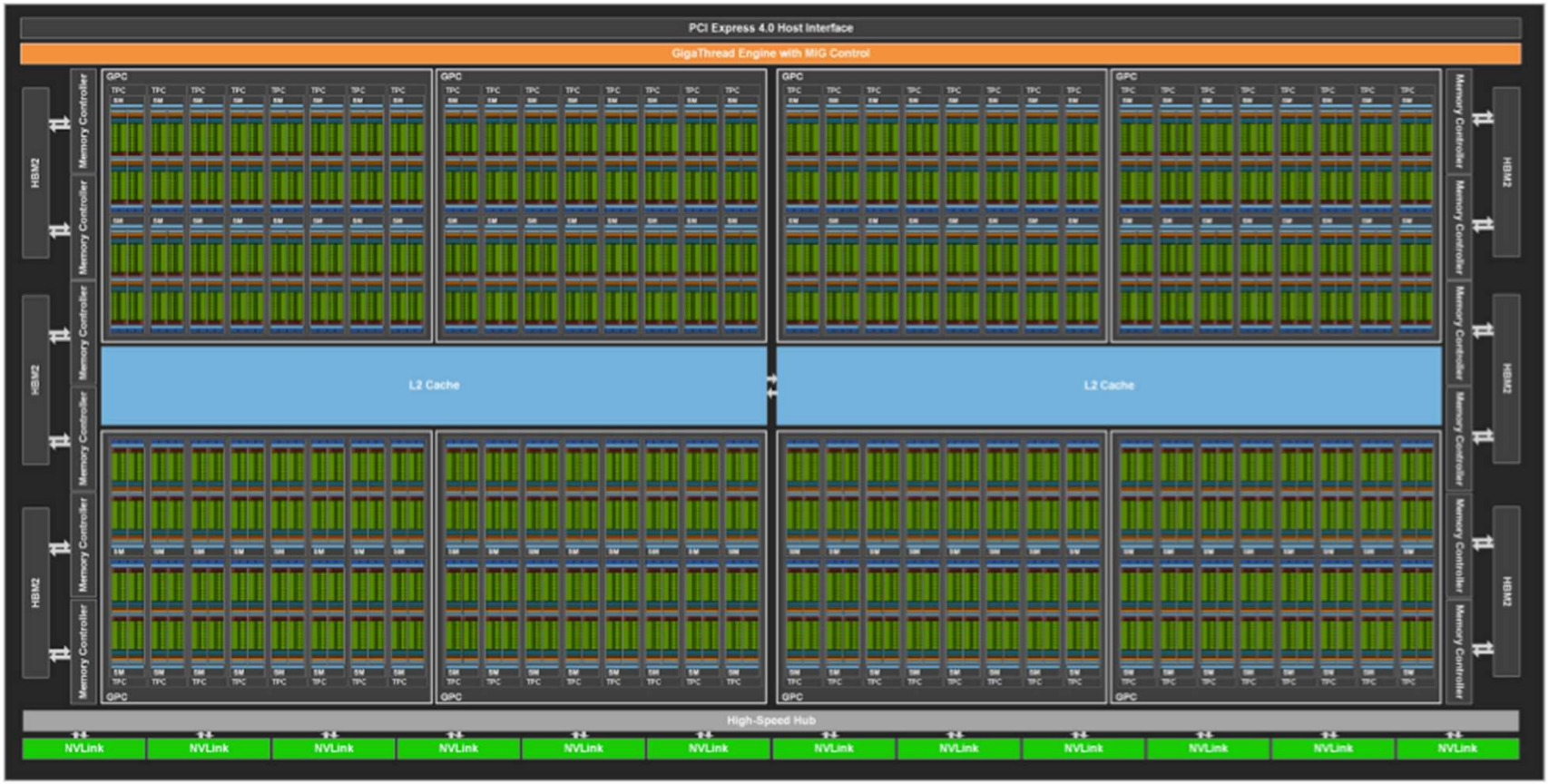
- **Multi tenancy Heterogeneous GPU sharing**
  - Gavel: Heterogeneity-Aware Cluster Scheduling [21]





Ampere (2020)

[14]



Ampere (2020)

- **Updated specs**

Archi.	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core/ SM	Total # of Cores	Tensor core / SM	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Volta	GV100	12nm FFN	6	42	84	2	14	64	5,376	8	4	64	2,048	5,376	32	128KB (Shared memory 96KB Max)		6,144KB
Ampere	GA100	TSMC 7nm	8	64	128	2	16	64	8,192	4	4	64	2,048	8,192	32	192KB (Shared memory 164KB Max)		49,152KB
Ampere	GA102	Samsung 8nm	7	42	84	2	12	192	10,752	4	4	48	1,536	4,032	16	128KB (Shared memory 100KB Max)		6,144KB
Ampere	GA104	Samsung 8nm	6	24	48	2	8	192	6,144	4	4	48	1,536	2,304	16	128KB (Shared memory 100KB Max)		4,096KB

Ampere (2020) (Cont'd)

- **SM**
  - 64 FP32 cores
  - 64 INT32 cores
  - 32 FP64 cores
  - 3rd Gen Tensor core
    - 4 Tensor cores

[14]



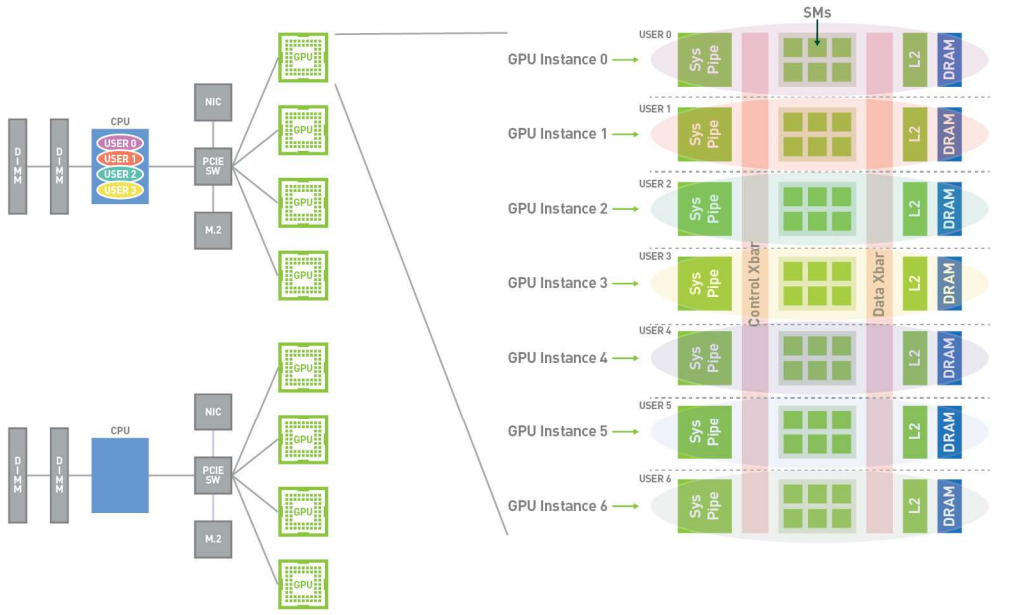
Shared memory / L1 cache

Ampere (2020) (Cont'd)

- **Architecture**
  - Multi-Instance GPU (MIG)

MULTI-INSTANCE GPU ("MIG")

[14]



Ampere (2020) (Cont'd)

- **Architecture**
  - Multi-Instance GPU (MIG)

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 450.80.02    Driver Version: 450.80.02    CUDA Version: 11.0     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  A100-PCIE-40GB      Off  | 00000000:01:00.0 Off |                   On |
| N/A   46C    P0    37W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   1  A100-PCIE-40GB      Off  | 00000000:25:00.0 Off |                   On |
| N/A   64C    P0    40W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   2  A100-PCIE-40GB      Off  | 00000000:41:00.0 Off |                   On |
| N/A   47C    P0    42W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   3  A100-PCIE-40GB      Off  | 00000000:61:00.0 Off |                   On |
| N/A   47C    P0    40W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   4  A100-PCIE-40GB      Off  | 00000000:81:00.0 Off |                   On |
| N/A   55C    P0    45W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   5  A100-PCIE-40GB      Off  | 00000000:A1:00.0 Off |                   On |
| N/A   58C    P0    44W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   6  A100-PCIE-40GB      Off  | 00000000:C1:00.0 Off |                   On |
| N/A   52C    P0    39W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+
|   7  A100-PCIE-40GB      Off  | 00000000:E1:00.0 Off |                   On |
| N/A   52C    P0    41W / 250W |      0MiB / 40537MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| MIG devices:                                                                |
+------------------+----------------------+-----------+-----------------------+
| GPU  GI  CI  MIG |         Memory-Usage |        Vol|         Shared        |
|      ID  ID  Dev |           BAR1-Usage | SM     Unc| CE  ENC  DEC  OFA  JPG|
|                  |                      |        ECC|                       |
|==================+======================+===========+=======================|
|  No MIG devices found                                                       |
+-----------------------------------------------------------------------------+
```

```
|  4    7    0    0  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4    8    0    1  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4    9    0    2  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4   10    0    3  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4   11    0    4  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4   12    0    5  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  4   13    0    6  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5    7    0    0  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5    8    0    1  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5    9    0    2  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5   10    0    3  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5   11    0    4  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5   12    0    5  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
|  5   13    0    6  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                     |      0MiB /  8191MiB |           |                       |
+---------------------+----------------------+-----------+-----------------------+
```

Ampere (2020) (Cont'd)

- **Architecture**
  - Multi-Instance GPU (MIG)

[31]

Config	Slice #0	Slice #1	Slice #2	Slice #3	Slice #4	Slice #5	Slice #6	OFA	NVDEC	NVJPEG	P2P	GPU Direct RDMA
1	7							1	7	7	No	Supported MemBW proportional to size of the instance
2	4				3			0	4+3	4+3	No	
3	4				2		1	0	4+2+1	4+2+1	No	
4	4				1	1	1	0	4+1+1+1	4+1+1+1	No	
5	3			3				0	3+3	3+3	No	
6	3			2		1		0	3+2+1	3+2+1	No	
7	3			1	1	1		0	3+1+1+1	3+1+1+1	No	
8	2		2		3			0	2+2+3	2+2+3	No	
9	2		1	1	3			0	2+1+1+3	2+1+1+3	No	
10	1	1	2		3			0	1+1+2+3	1+1+2+3	No	
11	1	1	1	1	3			0	1+1+1+1+3	1+1+1+1+3	No	
12	2		2		2		1	0	2+2+2+2+1	2+2+2+2+1	No	
13	2		1	1	2		1	0	2+1+1+2+1	2+1+1+2+1	No	
14	1	1	2		2		1	0	1+1+2+2+1	1+1+2+2+1	No	
15	2		1	1	1	1	1	0	2+1+1+1+1+1	2+1+1+1+1+1	No	
16	1	1	2		1	1	1	0	1+1+2+1+1+1	1+1+2+1+1+1	No	
17	1	1	1	1	2		1	0	1+1+1+1+2+1	1+1+1+1+2+1	No	
18	1	1	1	1	1	2		0	1+1+1+1+1+2	1+1+1+1+1+2	No	
19	1	1	1	1	1	1	1	0	1+1+1+1+1+1+1	1+1+1+1+1+1+1	No	

MIG Configs for Blackwell B200

Ampere (2020) (Cont'd)

- **Architecture**
  - Multi-Instance GPU (MIG)

[31]

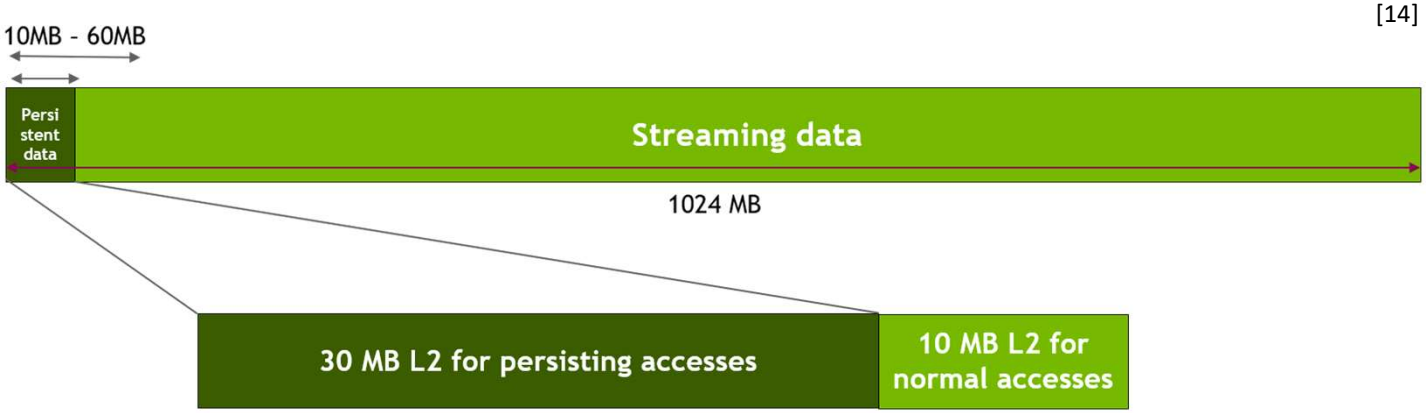
Config	Slice #0	Slice #1	Slice #2	Slice #3	Slice #4	Slice #5	Slice #6	OFA	NVDEC	NVJPEG	P2P	GPU Direct RDMA
1	7							1	7	7	No	Supported MemBW proportional to size of the instance
2	4				3			0	4+3	4+3	No	
3	4				2		1	0	4+2+1	4+2+1	No	
4	4				1	1	1	0	4+1+1+1	4+1+1+1	No	
5	3			3				0	3+3	3+3	No	
6	3			2		1		0	3+2+1	3+2+1	No	
7	3			1	1	1		0	3+1+1+1	3+1+1+1	No	
8	2		2		3			0	2+2+3	2+2+3	No	
9	2		1	1	3			0	2+1+1+3	2+1+1+3	No	
10	1	1	2		3			0	1+1+2+3	1+1+2+3	No	
11	1	1	1	1	3			0	1+1+1+1+3	1+1+1+1+3	No	
12	2		2		2		1	0	2+2+2+2+1	2+2+2+2+1	No	
13	2		1	1	2		1	0	2+1+1+2+1	2+1+1+2+1	No	
14	1	1	2		2		1	0	1+1+2+2+1	1+1+2+2+1	No	
15	2		1	1	1	1	1	0	2+1+1+1+1+1	2+1+1+1+1+1	No	
16	1	1	2		1	1	1	0	1+1+2+1+1+1	1+1+2+1+1+1	No	
17	1	1	1	1	2		1	0	1+1+1+1+2+1	1+1+1+1+2+1	No	
18	1	1	1	1	1	2		0	1+1+1+1+1+2	1+1+1+1+1+2	No	
19	1	1	1	1	1	1	1	0	1+1+1+1+1+1+1	1+1+1+1+1+1+1	No	

MIG Configs for Blackwell B200

Ampere (2020) (Cont'd)

- **CUDA**
  - L2 persistency API
    - Controlling which data will persistently remain in L2 cache

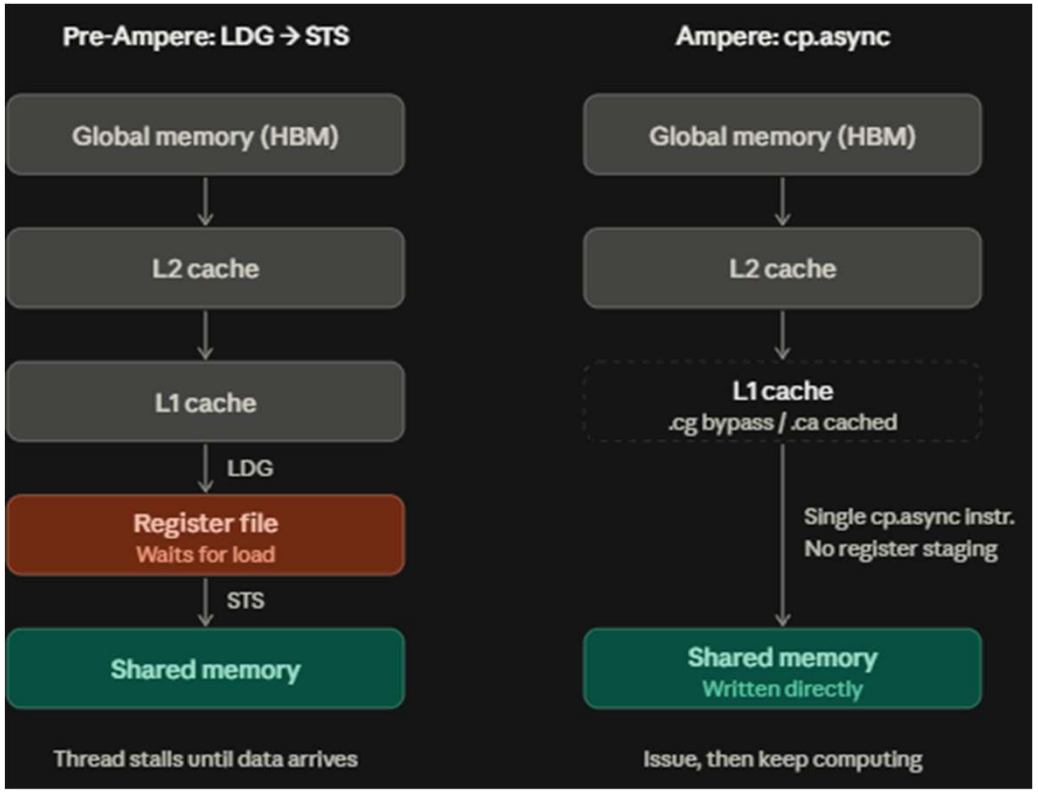
[14]



Ampere (2020) (Cont'd)

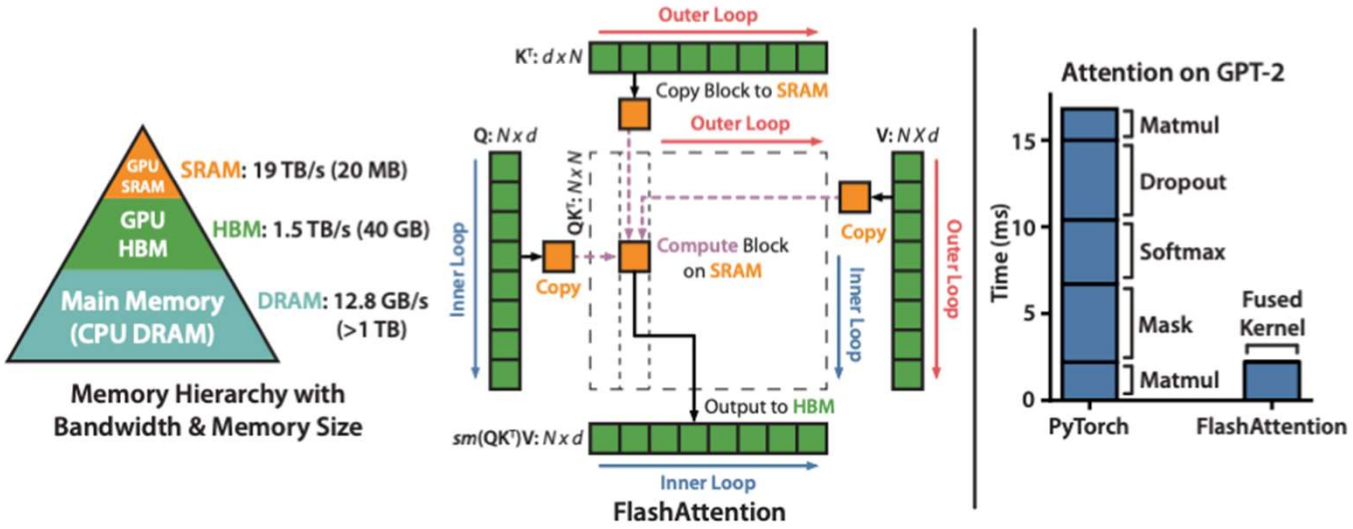
- **CUDA**
  - Asynchronous Memcpy
  - cp.async

[*]



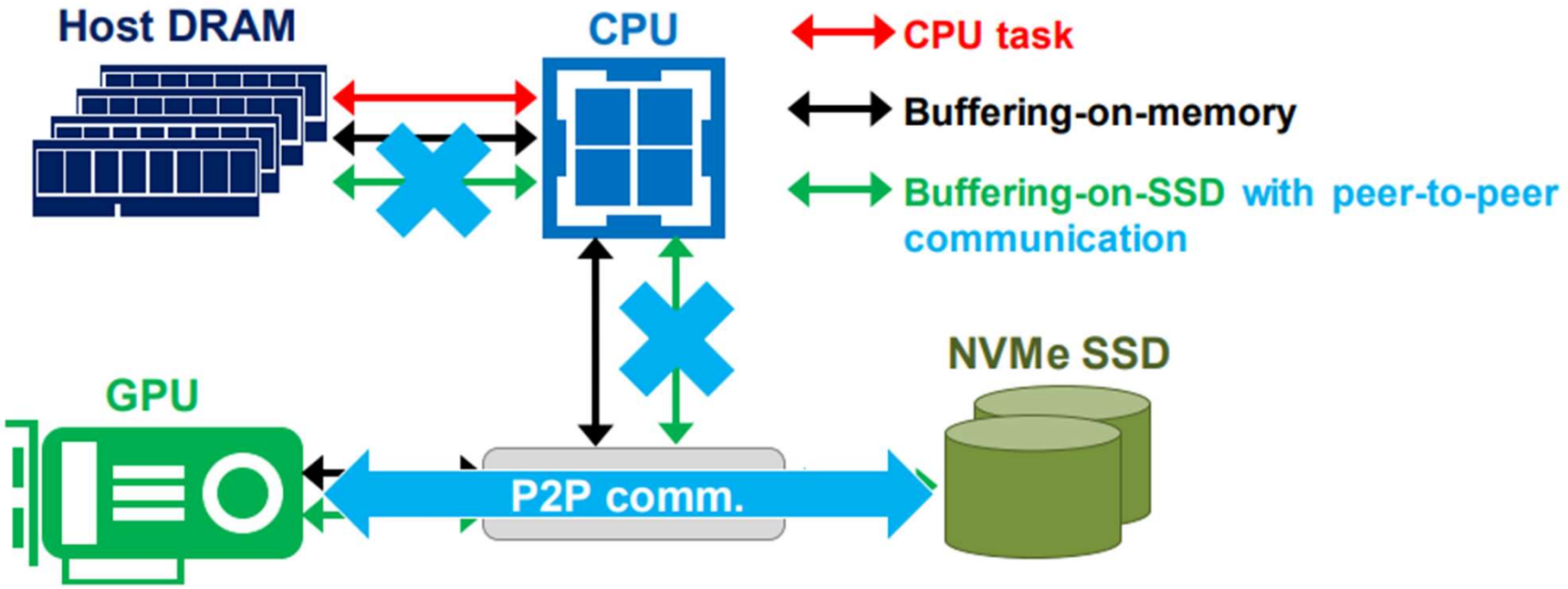
Related Studies (Ampere) (Cont'd)

- **LLM**
  - FlashAttention, NeurIPS 2022 [22]
    - Recomputation is better than Data Transfer due to Tensor Core
    - Kernel fusion of MM, softmax, mask, dropout



Related Studies (Ampere) (Cont'd)

- **Data Transfer Optimization**
  - FlashNeuron, FAST 2021 [23]



Related Studies (Ampere) (Cont'd)

- **LLM Training / Serving**
  - Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM [24]

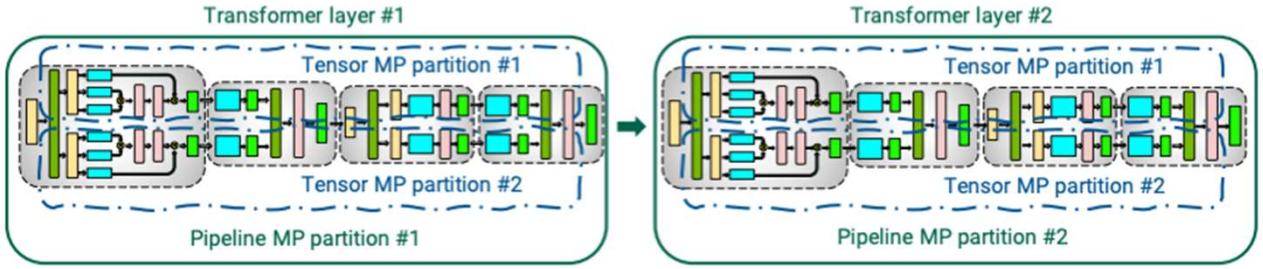
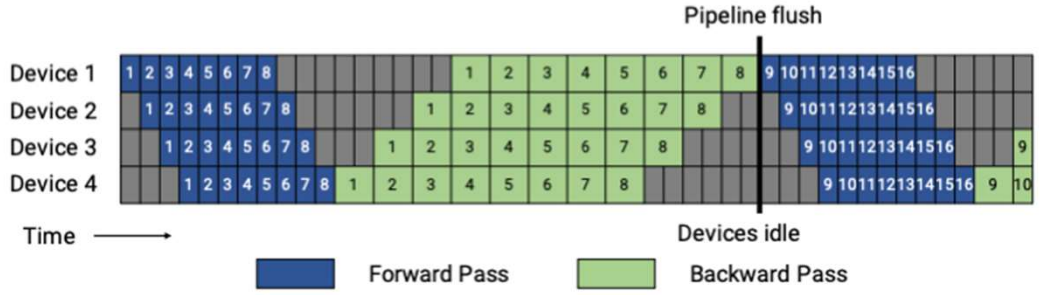
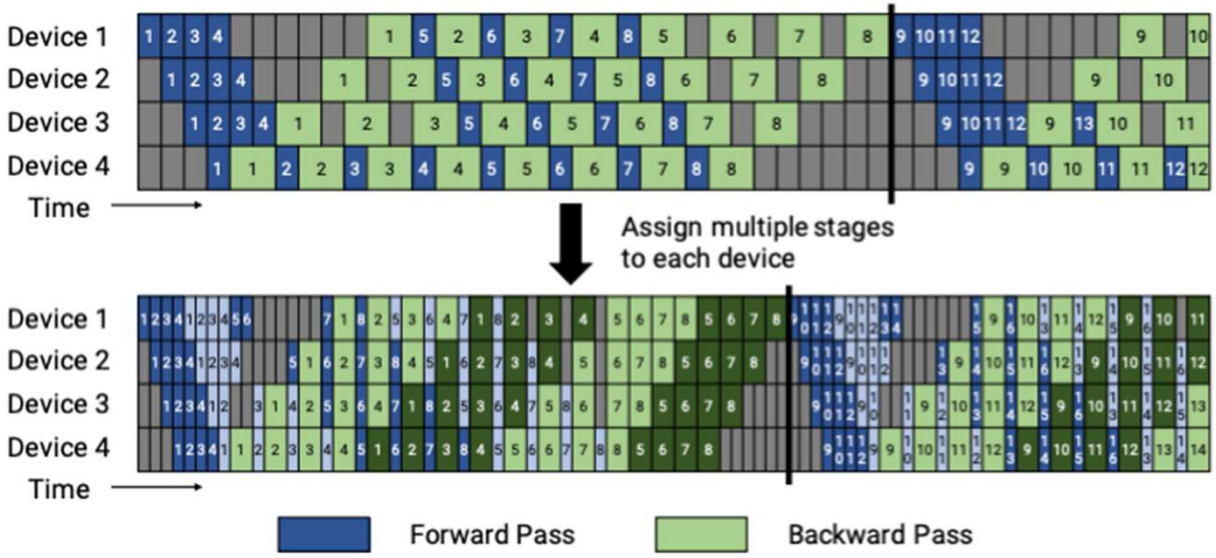


Figure 2: Combination of tensor and pipeline model parallelism (MP) used in this work for transformer-based models.



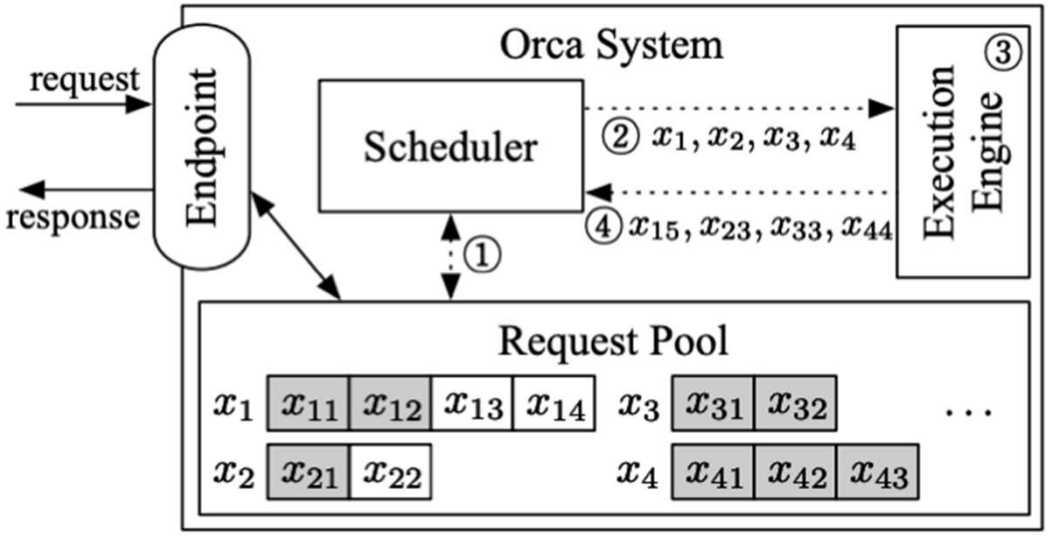
Related Studies (Ampere) (Cont'd)

- **LLM Training / Serving**
  - Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM [24]



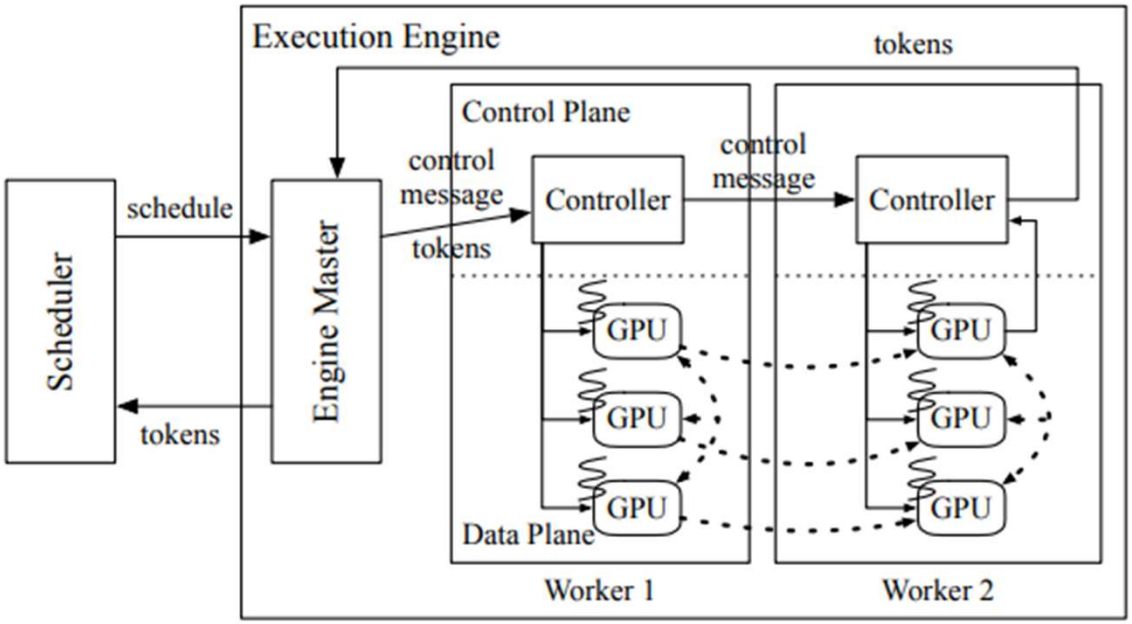
Related Studies (Ampere) (Cont'd)

- **LLM Training / Serving**
  - *Orca: A Distributed Serving System for Transformer-Based Generative Models* [25]



Related Studies (Ampere) (Cont'd)

- **LLM Training / Serving**
  - *Orca: A Distributed Serving System for Transformer-Based Generative Models* [25]



Hopper (2022) (Cont'd)

[29]



Hopper (2022)

- **Updated specs**

Archi.	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core/ SM	Total # of Cores	Tensor core / SM	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Ampere	GA104	Samsung 8nm	6	24	48	2	8	192	6,144	4	4	48	1,536	2,304	16	128KB (Shared memory 100KB Max)		4,096KB
Hopper	GH100	TSMC 4N	8	72	144	2	18	256	18,432	4	4	64	2,048	9,216	32	256KB (Shared memory 228KB max)		61,440KB

Hopper (2022) (Cont'd)

- **SM**
  - Support for FP8
  - 4th Gen Tensor Core
  - Tensor Memory Accelerator

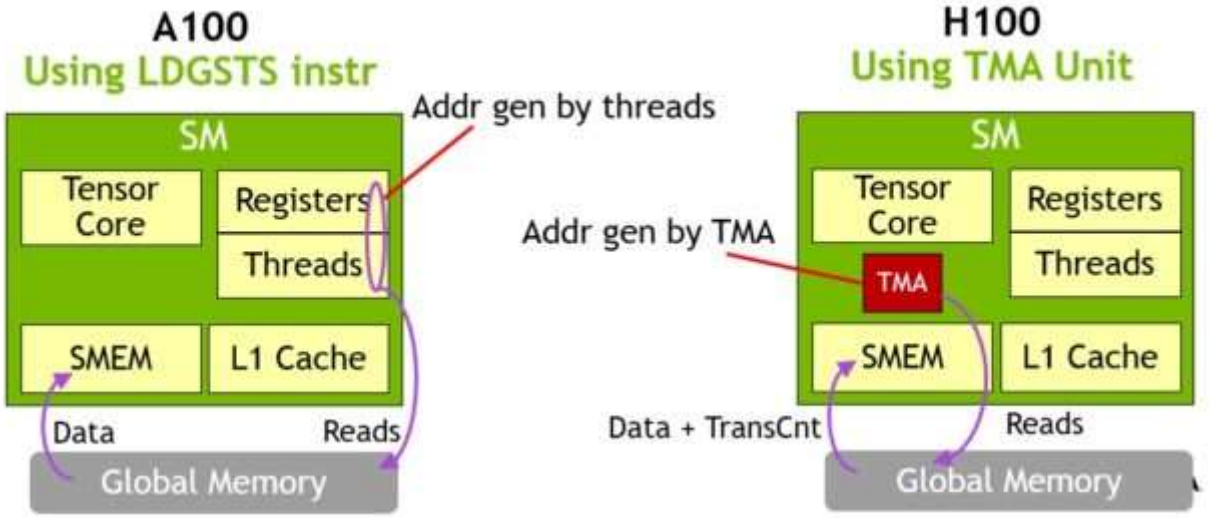
[29]



Hopper (2022) (Cont'd)

- **Architecture**
  - Tensor Memory Accelerator (TMA)

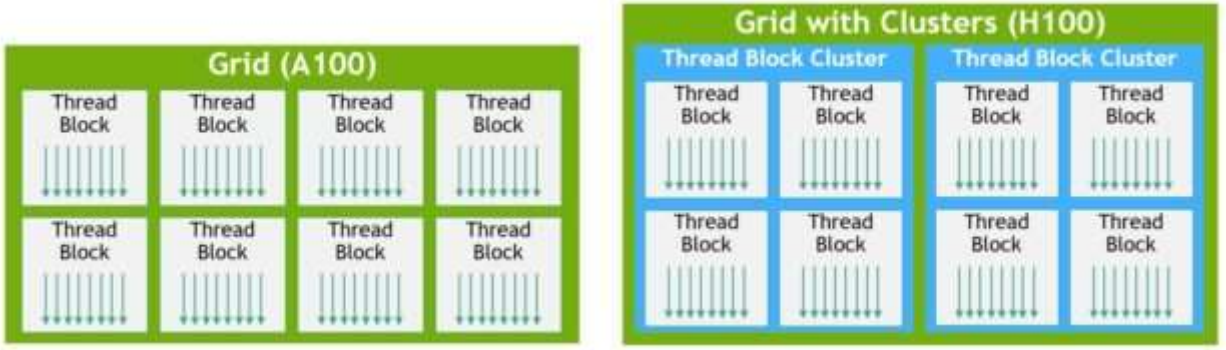
[29]



Hopper (2022) (Cont'd)

- **Architecture**
  - Thread block cluster
    - TBs in the same cluster will be deployed to the same GPC
  - Distributed Shared Memory
    - TB can load/store/atomic other TB's shared memory in the same cluster

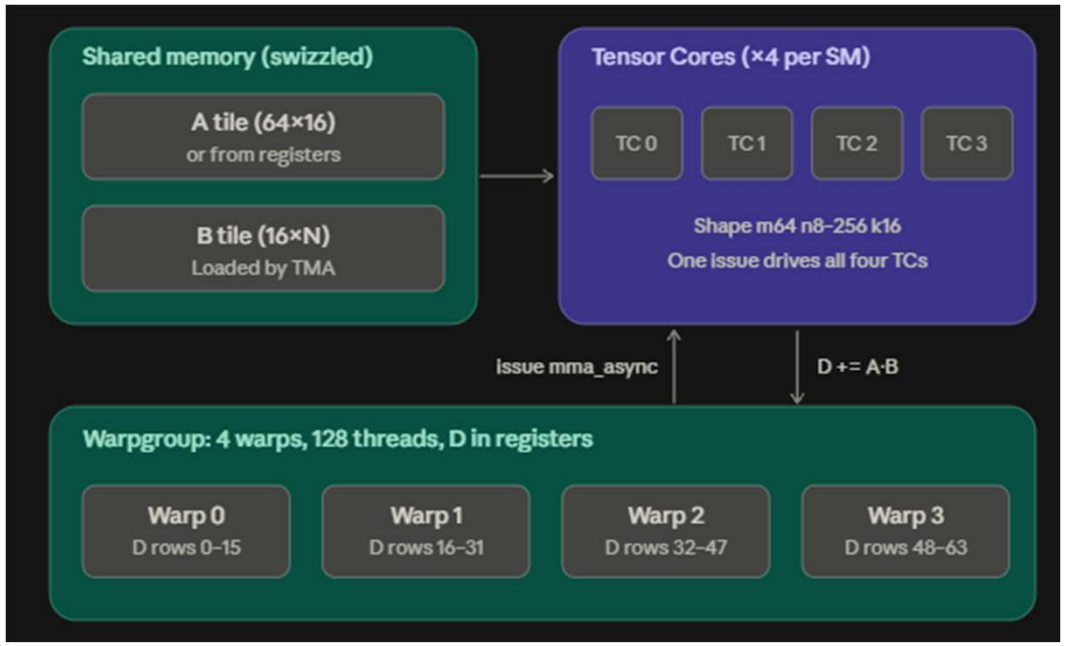
[29]



Hopper (2022) (Cont'd)

- **CUDA**
  - Asynchronous WarpGroup Matrix Multiply-Accumulate (WGMMA)
    - Command for optimized multiply-add
    - Directly referencing shared memory
    - Warp level process

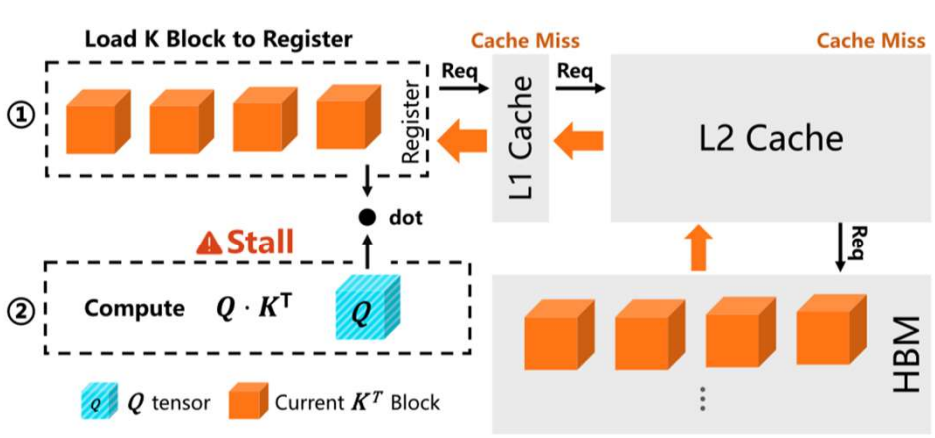
[*]



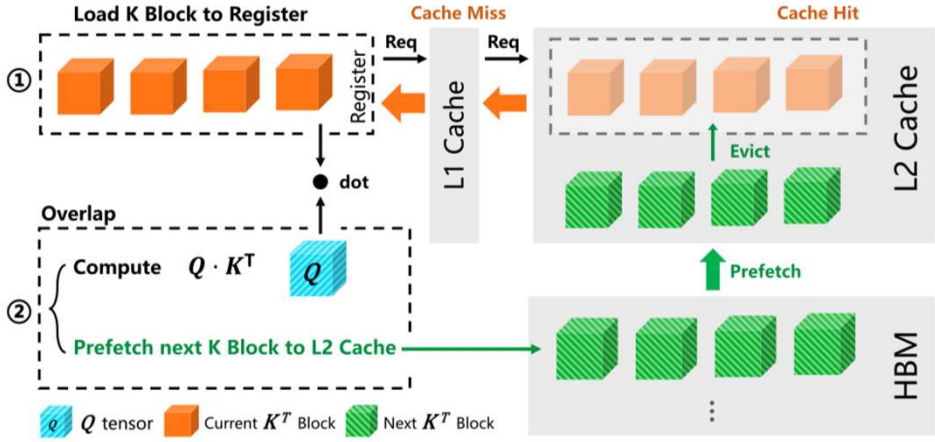
Hopper (2022) (Cont'd)

- **CUDA**
  - Asynchronous L2 cache prefetching
    - cp.async.bulk.prefetch.L2

[42]



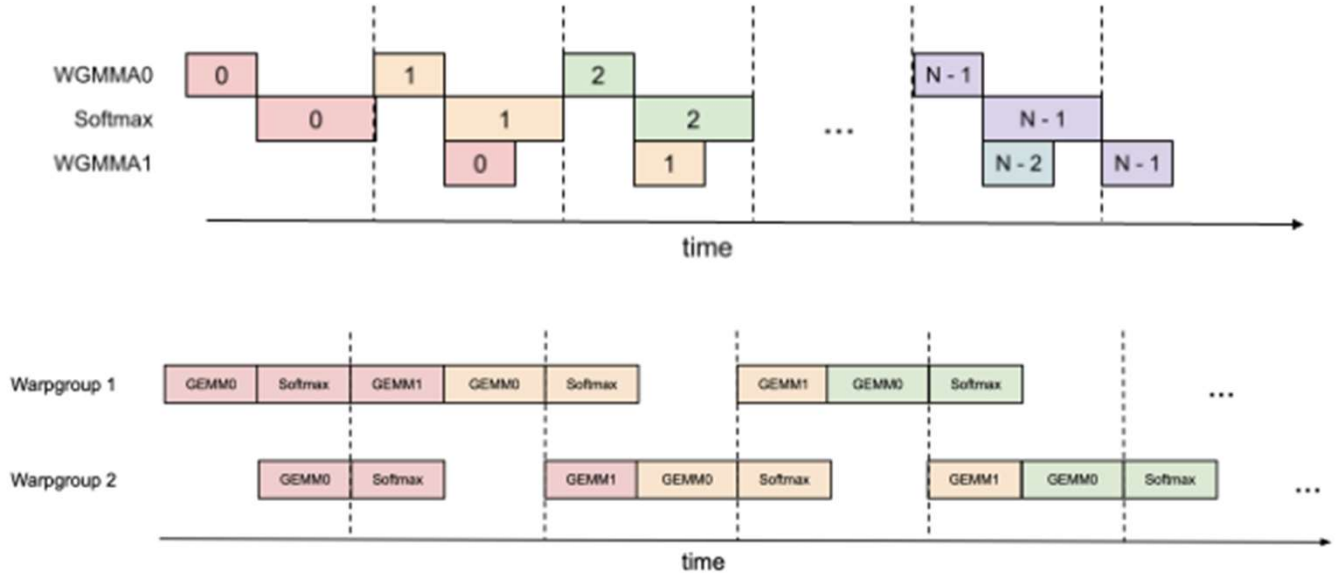
(a) Native XFormers



(b) The proposed method

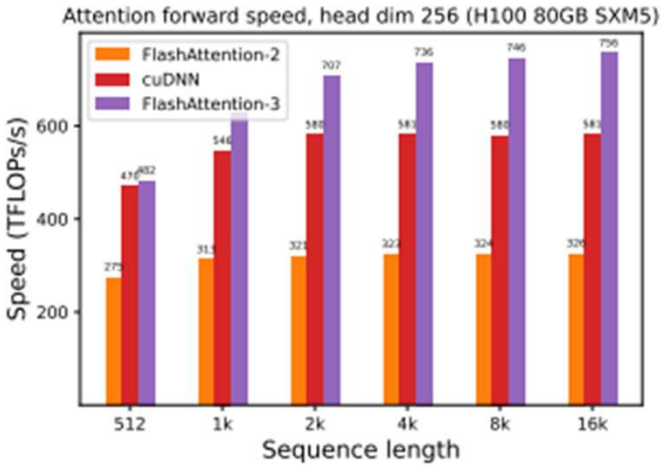
Related Studies (Hopper) (Cont'd)

- **LLM**
  - FlashAttention-3 [41]
    - Interleaving WGMMA & softmax

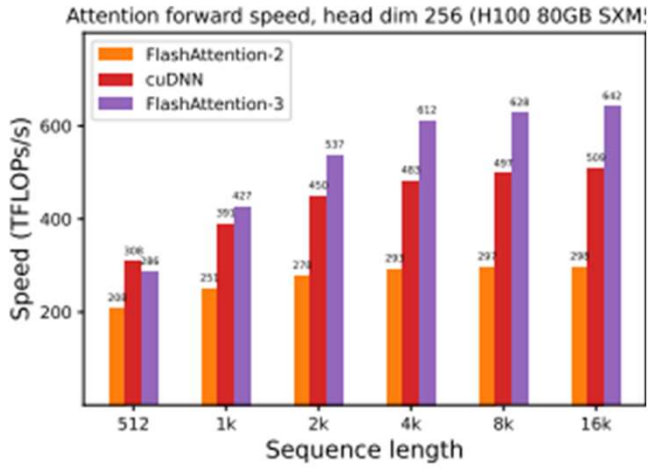


Related Studies (Hopper) (Cont'd)

- **LLM**
  - FlashAttention-3 [41]



(e) Forward, without causal mask, head dim 256



(f) Forward, with causal mask, head dim 256

Blackwell (2024)

[30]



Blackwell (2024)

- **Updated specs**

Archi.	Chip	Fabric	GPC #	TPC #	Total SMs	SM / TPC	SM / GPC	Core/ SM	Total # of Cores	Tensor core / SM	Warp scheduler / SM	Max # of warps in SM	Max # of threads in SM	Max # of warps in GPU (Total)	Max # of TBs in SM	L1 Cache (per SM)	Shared Memory (per SM)	L2 Cache
Hopper	GH100	TSMC 4N	8	72	144	2	18	256	18,432	4	4	64	2,048	9,216	32	256KB (Shared memory 228KB max)		61,440KB
Blackwell	GB202	TSMC 4N	12	96	192	2	16	*	24,576	4	4	48	1,536	9,216	32	128KB (Shared memory 100KB Max)		131,072KB

Blackwell (2024) (Cont'd)

- **SM**
  - FP4, FP6 cores included
  - 5th Gen Tensor Cores
  - Tensor Memory
    - On-chip 256KB memory per SM
    - Tcgen05 command
    - Accumulation on Tensor Memory
      - Register File before

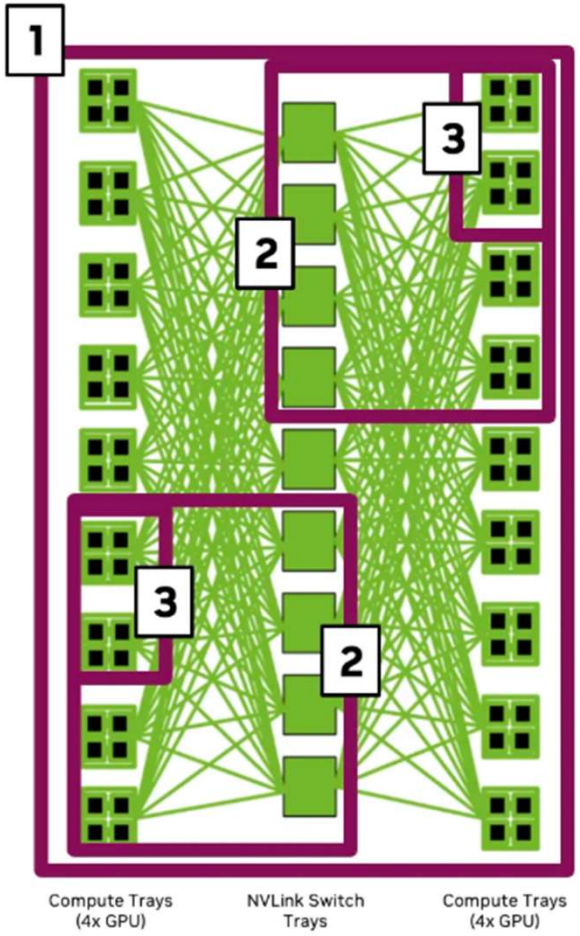
[30]



Blackwell (2024) (Cont'd)

- **Architecture**
  - Rack-scale NVLINK & NVLINK Switch

[30]



NVLink Domain 1
Defines an entire rack of physically connected GPUs over NVLink

NVLink Partition 2
Defines the groups of compute trays within an NVLink Domain that are allowed to communicate over MNNVL

IMEX Domain 3
Defines GPU memory sharing domains for GPUs in the same NVLink Partition and enables applications to use MNNV

Thank You!

(Prof. Hyeonsang Eom
hseom@snu.ac.kr)

References

1. E. Lindholm, J. Nickolls, S. Oberman and J. Montrym, "NVIDIA Tesla: A Unified Graphics and Computing Architecture," in IEEE Micro, vol. 28, no. 2, pp. 39-55, March-April 2008, doi: 10.1109/MM.2008.31.
2. David Kanter, NVIDIA's GT200: Inside a Parallel Processor, https://www.realworldtech.com/gt200/10/
3. NVIDIA, Fermi Whitepaper, https://www.nvidia.com/content/pdf/fermi_white_papers/nvidia_fermi_compute_architecture_whitepaper.pdf
4. NVIDIA, Peer-to-Peer & Unified Virtual Addressing Webinar, https://developer.download.nvidia.com/CUDA/training/cuda_webinars_GPUDirect_uva.pdf
5. NVIDIA, Kepler Whitepaper, https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/NVIDIA-Kepler-GK110-GK210-Architecture-Whitepaper.pdf
6. NVIDIA, GTC2012 Keynote, https://developer.download.nvidia.com/GTC/PDF/GTC2012/PresentationPDF/S0338-GTC2012-CUDA-Programming-Model.pdf
7. NVIDIA, MPS Architecture, https://docs.nvidia.com/deploy/mps/latest/architecture.html
8. Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, & Evan Shelhamer. (2014). cuDNN: Efficient Primitives for Deep Learning.
9. NVIDIA, GeForce GTX 750 Ti Whitepaper, https://international.download.nvidia.com/geforce-com/international/pdfs/GeForce-GTX-750-Ti-Whitepaper.pdf?_gl=1*178nw1o*_gcl_au*NzY0MDUzNDU2LjE3ODQ1NDY5MDkuLS4tLjE3ODQ1NDY5MDkuNTU3ODQ2NTkwLjE3OTExMTQ1MDcuMTc5MTE2NDUxMw..
10. NVIDIA, Pascal P100 Whitepaper, https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf
11. NVIDIA, Inside-Pascal blog, https://developer.nvidia.com/blog/inside-pascal/
12. NVIDIA, NCCL, https://developer.nvidia.com/nccl
13. NVIDIA, Volta Whitepaper, https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf
14. NVIDIA, Ampere GA102 Whitepaper, https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf
15. A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," 2009 IEEE International Symposium on Performance Analysis of Systems and Software, Boston, MA, USA
16. Bo Wu, Xu Liu, Xiaobo Zhou, and Changjun Jiang. 2017. FLEP: Enabling Flexible and Efficient Preemption on GPUs. SIGPLAN Not. 52, 4 (April 2017), 483–496. https://doi.org/10.1145/3093336.3037742

References

17. Micikevicius, Paulius et al. “Mixed Precision Training.” ArXiv abs/1710.03740 (2017): n. pag.

18. Alexander Sergeev, & Mike Del Balso. (2018). Horovod: fast and easy distributed deep learning in TensorFlow.

19. Wei Zhang, Weihao Cui, Kaihua Fu, Quan Chen, Daniel Edward Mawhirter, Bo Wu, Chao Li, and Minyi Guo. 2019. Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters. In Proceedings of the ACM International Conference on Supercomputing (ICS '19). Association for Computing Machinery, New York, NY, USA, 58–68. https://doi.org/10.1145/3330345.3330351Modeling Deep Learning Accelerator Enabled GPUs

20. M. A. Raihan, N. Goli and T. M. Aamodt, "Modeling Deep Learning Accelerator Enabled GPUs," 2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Madison, WI, USA, 2019, pp. 79-92, doi: 10.1109/ISPASS.2019.00016.

21. Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, & Matei Zaharia (2020). Heterogeneity-Aware Cluster Scheduling Policies for Deep Learning Workloads. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20) (pp. 481–498). USENIX Association.

22. Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1189, 16344–16359.

23. Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, & Jae W. Lee (2021). FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In 19th USENIX Conference on File and Storage Technologies (FAST 21) (pp. 387–401). USENIX Association.

24. Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, & Matei Zaharia. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM.

25. Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, & Byung-Gon Chun (2022). Orca: A Distributed Serving System for Transformer-Based Generative Models. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22) (pp. 521–538). USENIX Association.

26. Mao Lin, Yuan Feng, Guilherme Cox, and Hyeran Jeon. 2025. Forest: Access-aware GPU UVM Management. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25). Association for Computing Machinery, New York, NY, USA, 137–152. https://doi.org/10.1145/3695053.3731047

27. https://blog.grijjy.com/2021/01/14/shader-programming/

28. NVIDIA. CUDA Programming Guide. https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html

29. NVIDIA. Hopper Whitepaper. https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c

30. NVIDIA. Blackwell Whitepaper. https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf

References

31. NVIDIA Multi-Instance GPU User Guide, Table of Contenhttps://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/supported-mig-profiles.html

32. NVIDIA GeForce256 Spec, https://www.techpowerup.com/gpu-specs/geforce-256-ddr.c734

33. NVIDIA GeForce3 Spec, https://www.techpowerup.com/gpu-specs/geforce3.c738

34. NVIDIA Tegra X1, https://www.techpowerup.com/gpu-specs/docs/nvidia-tegra-x1-architecture.pdf

35. I. Tanasic, I. Gelado, J. Cabezas, A. Ramirez, N. Navarro and M. Valero, "Enabling preemptive multiprogramming on GPUs," 2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA), Minneapolis, MN, USA, 2014, pp. 193-204, doi: 10.1109/ISCA.2014.6853208.

36. Bharath Pichai, Lisa Hsu, and Abhishek Bhattacharjee. 2014. Architectural support for address translation on GPUs: designing memory management units for CPU/GPUs with unified address spaces. In Proceedings of the 19th international conference on Architectural support for programming languages and operating systems (ASPLOS '14).

37. X. Mei and X. Chu, "Dissecting GPU Memory Hierarchy Through Microbenchmarking," in IEEE Transactions on Parallel and Distributed Systems, vol. 28, no. 1, pp. 72-86, 1 Jan. 2017

38. Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, & Onur Mutlu. (2018). Mosaic: An Application-Transparent Hardware-Software Cooperative Memory Manager for GPUs.

39. Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, & Stephen W. Keckler. (2016). vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design.

40. Control Flow Management in Modern GPUs

41. Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, & Tri Dao. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision.

42. Yanhao Dong, Yubo Miao, Weinan Li, Xiao Zheng, Chao Wang, Jiesheng Wu, & Feng Lyu. (2025). Accelerating LLM Inference Throughput via Asynchronous KV Cache Prefetching.

[*] This image has been generated by Claude Opus 5.5. Anthropic. (2026). Introducing Claude Opus 5.5. Anthropic. https://www.anthropic.com/claude-opus-5-5